



Escuela
Politécnica
Superior

Comparativa de frameworks de Federated Learning aplicados a clasificación de imágenes



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Raúl Luján Domenech

Tutores:

Francisco Antonio Pujol López

Tamai Ramírez Gordillo

Junio 2026



Universitat d'Alacant
Universidad de Alicante

Comparativa de frameworks de Federated Learning aplicados a clasificación de imágenes

Autor

Raúl Luján Domenech

Tutores

Francisco Antonio Pujol López

Departamento de Tecnología Informática y Computación

Tamai Ramírez Gordillo

Departamento de Tecnología Informática y Computación



Grado en Ingeniería Informática



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Junio 2026

A Raúl Ruiz Flores, mi mejor amigo y compañero de universidad.

Agradecimientos

Muchas gracias a mi familia y, sobre todo, a mi madre, por apoyarme tanto durante estos años de carrera. Nada de esto habría sido posible sin ella, que me insistió tantas veces en que terminara el TFG y me dio ánimos justo cuando menos me apetecía seguir.

Muchas gracias a mi mejor amigo, Raúl Ruiz Flores, por todos estos años de amistad y compañerismo, por las risas en la universidad y por habernos lanzado juntos a todo lo que pudimos. Gracias, además, por toda la ayuda que me ha dado a lo largo de la carrera.

Muchas gracias a mi hermano, por apoyarme incondicionalmente en todo, aunque no tuviera demasiada idea de lo que yo estaba haciendo.

Muchas gracias a Ginés, Alex, Juan, Nico, David y Raúl, mis amigos de la carrera, por todas esas risas y noches de estudio compartidas.

Muchas gracias a mis tutores, Paco y Tamai, por estas dos etapas que hemos vivido, la de profesor-alumno y la de tutor-alumno. Gracias por lo atentos y cercanos que han sido conmigo y por el buen trato que he recibido en todo momento.

Muchas gracias a mi grupo de amigos actual, que confió en que sería capaz de hacerlo y de terminar la carrera a pesar de los baches del camino.

Resumen

El aprendizaje federado permite entrenar modelos de forma colaborativa sin centralizar los datos, lo que resulta apropiado para escenarios distribuidos o con información sensible. Para construir este tipo de sistemas se recurre a *frameworks* especializados, pero la variedad de herramientas disponibles dificulta decidir cuál conviene en cada caso, ya que cada una adopta decisiones de diseño distintas y puede comportarse de otra manera ante la misma carga de trabajo.

Este Trabajo de Fin de Grado compara de forma experimental tres *frameworks* de aprendizaje federado —Flower, NVIDIA FLARE y FedML— sobre un mismo escenario controlado: clasificación de imágenes con el conjunto CIFAR-10, una red ResNet-18 adaptada, el algoritmo *Federated Averaging* (FedAvg) y una configuración *cross-silo* horizontal. La evaluación se articula en tres ejes: el rendimiento cuantitativo del modelo (precisión, tiempo de ejecución y consumo de recursos), la usabilidad entendida como experiencia de desarrollo y la flexibilidad como propiedad arquitectónica.

Los resultados muestran que, bajo idénticas condiciones, los tres *frameworks* alcanzan precisiones muy similares (en torno al 90,7–92,0 %), de modo que la elección de la herramienta influye poco en la calidad del modelo y mucho en el coste de obtenerlo, en la experiencia de uso y en la estabilidad del modo concurrente. En usabilidad, Flower ofrece la puesta en marcha más sencilla, NVIDIA FLARE destaca por su trazabilidad a costa de una curva de aprendizaje elevada y FedML ocupa una posición intermedia. En flexibilidad, las tres herramientas obtienen una valoración equivalente y se diferencian sobre todo en su perfil cualitativo. El trabajo aporta así una comparación razonada y reproducible que orienta la selección de un *framework* de aprendizaje federado en proyectos similares.

Palabras clave: aprendizaje federado, Flower, NVIDIA FLARE, FedML, FedAvg, CIFAR-10, comparativa experimental.

Abstract

Federated learning makes it possible to train models collaboratively without centralising the data, which suits distributed scenarios or settings with sensitive information. Building such systems relies on specialised frameworks, but the range of available tools makes it hard to decide which one is appropriate in each case, since each adopts different design choices and may behave differently under the same workload.

This Bachelor’s Thesis carries out an experimental comparison of three federated learning frameworks —Flower, NVIDIA FLARE and FedML— on a single controlled scenario: image classification on the CIFAR-10 dataset, an adapted ResNet-18 network, the Federated Averaging (FedAvg) algorithm and a horizontal cross-silo setup. The evaluation is organised along three axes: the quantitative performance of the model (accuracy, execution time and resource usage), usability understood as the development experience, and flexibility as an architectural property.

The results show that, under identical conditions, the three frameworks reach very similar accuracies (around 90.7–92.0%), so the choice of tool has little effect on model quality and a large effect on the cost of obtaining it, on the user experience and on the stability of the concurrent mode. Regarding usability, Flower offers the most straightforward setup, NVIDIA FLARE stands out for its traceability at the cost of a steep learning curve, and FedML sits in an intermediate position. Regarding flexibility, the three tools obtain an equivalent score and differ mainly in their qualitative profile. The work thus provides a reasoned and reproducible comparison that guides the selection of a federated learning framework in similar projects.

Keywords: federated learning, Flower, NVIDIA FLARE, FedML, FedAvg, CIFAR-10, experimental comparison.

Índice general

Agradecimientos	v
Resumen	vii
Abstract	ix
1 Introducción	1
1.1 Contextualización del problema	2
1.2 Estructura de la memoria	2
2 Objetivos	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
3 Fundamentos teóricos	5
3.1 Aprendizaje automático y aprendizaje profundo	5
3.2 Del entrenamiento centralizado al aprendizaje federado	5
3.3 Aprendizaje federado	6
3.3.1 Funcionamiento: la ronda federada y el algoritmo <i>FedAvg</i>	6
3.3.2 Variantes de <i>FedAvg</i>	8
3.3.3 Tipos de aprendizaje federado	9
3.3.4 Retos del aprendizaje federado	10
4 Estado del arte	11
4.1 Panorama de herramientas existentes	11
4.2 Criterios de selección	12
4.3 Frameworks seleccionados	12
4.3.1 Flower	13
4.3.2 NVIDIA FLARE	13
4.3.3 FedML	14
4.4 Comparativa de los frameworks seleccionados	14
4.5 Estudios comparativos previos	15
4.6 Síntesis y posicionamiento	15
5 Metodología	17
5.1 Entorno de trabajo	17
5.2 Comparativa de rendimiento: métricas e instrumentación	17
5.3 Comparativa de usabilidad: marco y rúbrica	18
5.4 Análisis de flexibilidad: criterios	20
5.5 Reproducibilidad y control de variables	20

5.6	Alcance y limitaciones del estudio	22
6	Diseño experimental e implementación	23
6.1	Diseño del experimento común	23
6.1.1	Conjunto de datos y particionamiento	23
6.1.2	Modelo	24
6.1.3	Algoritmo y configuración de entrenamiento	24
6.2	Matriz de experimentos	25
6.3	Modos de ejecución: secuencial y concurrente	25
7	Resultados	27
7.1	Consideraciones previas	27
7.2	Flower	29
7.2.1	Rendimiento de Flower	29
7.2.2	Usabilidad de Flower	29
7.2.2.1	Utilidad funcional (<i>useful</i>)	30
7.2.2.2	Facilidad de uso (<i>usable</i>)	30
7.2.2.3	Experiencia de desarrollo (<i>desirable</i>)	31
7.2.2.4	Accesibilidad (<i>accessible</i>)	31
7.2.2.5	Valoración de la usabilidad de Flower	31
7.2.3	Flexibilidad de Flower	32
7.2.3.1	Adaptaciones para habilitar la evaluación	32
7.2.3.2	Selección dinámica de la estrategia de agregación	33
7.2.3.3	Mini-experimento de flexibilidad	34
7.2.3.4	Integración de modelo, datos y métricas	35
7.2.3.5	Capacidades avanzadas según la documentación	35
7.2.3.6	Ejecución, despliegue y operación según la documentación	36
7.2.3.7	Limitaciones observadas	37
7.2.3.8	Valoración de la flexibilidad de Flower	37
7.3	NVIDIA FLARE	38
7.3.1	Rendimiento de NVIDIA FLARE	38
7.3.2	Usabilidad de NVIDIA FLARE	40
7.3.2.1	Utilidad funcional (<i>useful</i>)	40
7.3.2.2	Facilidad de uso (<i>usable</i>)	40
7.3.2.3	Experiencia de desarrollo (<i>desirable</i>)	41
7.3.2.4	Accesibilidad (<i>accessible</i>)	41
7.3.2.5	Valoración de la usabilidad de NVIDIA FLARE	41
7.3.3	Flexibilidad de NVIDIA FLARE	41
7.3.3.1	Diseño modular: jobs, componentes y workflows	42
7.3.3.2	Adaptaciones para soportar las estrategias	42
7.3.3.3	Mini-experimento de flexibilidad	43
7.3.3.4	Capacidades avanzadas según la documentación	44
7.3.3.5	Limitaciones observadas	45
7.3.3.6	Valoración de la flexibilidad de NVIDIA FLARE	46
7.4	FedML	47
7.4.1	Rendimiento de FedML	47

7.4.2	Usabilidad de FedML	48
7.4.2.1	Utilidad funcional (<i>useful</i>)	48
7.4.2.2	Facilidad de uso (<i>usable</i>)	49
7.4.2.3	Experiencia de desarrollo (<i>desirable</i>)	49
7.4.2.4	Accesibilidad (<i>accessible</i>)	49
7.4.2.5	Valoración de la usabilidad de FedML	49
7.4.3	Flexibilidad de FedML	50
7.4.3.1	Configuración y componentes personalizados	50
7.4.3.2	Sustitución de la estrategia federada	51
7.4.3.3	Mini-experimento de flexibilidad	51
7.4.3.4	Capacidades avanzadas según la documentación	52
7.4.3.5	Limitaciones observadas	53
7.4.3.6	Valoración de la flexibilidad de FedML	53
7.5	Comparativa entre frameworks	53
7.5.1	Comparativa de rendimiento	54
7.5.1.1	Precisión final	54
7.5.1.2	Tiempo de ejecución	54
7.5.1.3	Escalabilidad	55
7.5.1.4	Consumo de recursos	56
7.5.1.5	Comportamiento en modo concurrente	57
7.5.2	Comparativa de usabilidad	57
7.5.3	Comparativa de flexibilidad	58
7.6	Desglose gráfico por métrica	60
7.6.1	Desglose por métrica: Flower	60
7.6.2	Desglose por métrica: NVIDIA FLARE	60
7.6.3	Desglose por métrica: FedML	63
7.6.4	Valores máximos y mínimos entre frameworks	63
7.7	Síntesis global de resultados	66
8	Conclusiones	69
8.1	Consecución de los objetivos	69
8.2	Conclusiones principales	70
8.2.1	Rendimiento	70
8.2.2	Usabilidad	70
8.2.3	Flexibilidad	70
8.3	Aportaciones del trabajo	71
8.4	Limitaciones del estudio	71
8.5	Líneas de trabajo futuro	72
8.6	Reflexión final	73
	Bibliografía	75
	Lista de Acrónimos y Abreviaturas	79
	Glosario de Términos	81

Índice de figuras

3.1	Comparación entre el entrenamiento centralizado, en el que los datos se reúnen en una infraestructura común, y el aprendizaje federado, en el que solo se intercambian el modelo global (línea continua) y los modelos locales entrenados (línea discontinua). Elaboración propia a partir de McMahan et al. (2017) y Kairouz et al. (2021).	6
3.2	Ciclo de una ronda federada en <i>FedAvg</i> : el servidor distribuye el modelo global, cada cliente lo entrena con sus datos locales, devuelve su modelo local entrenado y el servidor agrega esos modelos para generar una nueva versión del modelo global (McMahan et al., 2017).	7
4.1	Arquitectura por capas común a los <i>frameworks</i> de aprendizaje federado comparados. Las herramientas se diferencian en cómo exponen cada capa al desarrollador y en el énfasis que ponen en la simulación, el despliegue o la seguridad. Elaboración propia.	13
7.1	Convergencia de la precisión global de Flower en el experimento principal.	30
7.2	Convergencia de la precisión centralizada del modelo global de NVIDIA FLARE en el experimento principal, evaluando sobre el conjunto de prueba completo el modelo agregado de cada ronda.	39
7.3	Convergencia de la precisión centralizada del modelo global de FedML en el experimento principal, evaluada sobre el conjunto de prueba completo al final de cada ronda.	48
7.4	Precisión final por experimento y <i>framework</i> (modo secuencial).	55
7.5	Tiempo de ejecución por experimento y <i>framework</i> (modo secuencial).	55
7.6	Precisión final frente al número de clientes en los experimentos de escalabilidad (modo secuencial).	56
7.7	Energía estimada por experimento (modo secuencial).	56
7.8	Flower: precisión y pérdida final por experimento (modo secuencial).	60
7.9	Flower: tiempo de ejecución y energía estimada por experimento (modo secuencial).	61
7.10	Flower: memoria de GPU y memoria RAM por experimento (modo secuencial). La RAM corresponde al proceso principal; el motor de simulación Ray reparte el trabajo en procesos hijo que esta métrica no contabiliza, por lo que infravalora el consumo real.	61
7.11	Flower: utilización de GPU y temperatura por experimento (modo secuencial).	61
7.12	NVIDIA FLARE: precisión y pérdida final por experimento (modo secuencial).	62
7.13	NVIDIA FLARE: tiempo de ejecución y energía estimada por experimento (modo secuencial).	62
7.14	NVIDIA FLARE: memoria de GPU y memoria RAM por experimento (modo secuencial). La RAM corresponde al proceso principal.	62

7.15	NVIDIA FLARE: utilización de GPU y temperatura por experimento (modo secuencial).	63
7.16	FedML: precisión y pérdida final por experimento (modo secuencial). . . .	63
7.17	FedML: tiempo de ejecución y energía estimada por experimento (modo secuencial).	64
7.18	FedML: memoria de GPU y memoria RAM por experimento (modo secuencial). En este modo FedML se ejecuta en un único proceso (<i>backend sp</i>), por lo que la RAM del proceso principal refleja el consumo real del <i>framework</i>	64
7.19	FedML: utilización de GPU y temperatura por experimento (modo secuencial). No se registró la temperatura de FedML en e0.	64
7.20	Memoria de GPU máxima y mínima por <i>framework</i> (modo secuencial), con el experimento en el que se alcanzó. En NVIDIA FLARE y FedML el extremo se repite en varios experimentos.	65
7.21	Precisión máxima y mínima por <i>framework</i> (modo secuencial). El máximo de los tres se dio en e1 (configuración principal) y el mínimo en e2-c8 (ocho clientes).	65
7.22	Memoria RAM máxima y mínima por <i>framework</i> (modo secuencial). La RAM corresponde al proceso principal medido con <code>/usr/bin/time</code> . En este modo, NVIDIA FLARE y FedML (<i>backend sp</i>) se ejecutan en un único proceso, mientras que Flower delega en Ray, que reparte el trabajo en procesos hijo no contabilizados por esta métrica; por ello la cifra de Flower aparece artificialmente baja y la comparación entre <i>frameworks</i> debe interpretarse con cautela.	66

Índice de tablas

4.1	Comparación cualitativa de los tres <i>frameworks</i> seleccionados.	15
5.1	Versiones de software empleadas en cada entorno de ejecución.	17
5.2	Métricas de rendimiento recogidas en cada ejecución y herramienta de medida.	18
5.3	Rúbrica de usabilidad. Cada subcriterio se valora con una escala de 1 a 5.	19
5.4	Escala de valoración empleada en la rúbrica de usabilidad.	20
5.5	Escala de valoración empleada en el análisis de flexibilidad.	20
5.6	Criterios de flexibilidad evaluados. Cada criterio se valora de 0 a 2.	21
6.1	Matriz de experimentos del estudio. Cada configuración se ejecuta en los tres <i>frameworks</i> y en los dos modos de ejecución.	25
7.1	Resultados de Flower en modo secuencial. La RAM corresponde al proceso principal; en Flower, el motor de simulación Ray ejecuta procesos hijo, por lo que esta cifra infravalora el consumo real de memoria del sistema.	29
7.2	Valoración de la usabilidad de Flower según las cuatro dimensiones de la rúbrica (escala 1–5).	31
7.3	Configuración del mini-experimento de flexibilidad de Flower.	34
7.4	Resultados del mini-experimento de flexibilidad de Flower con tres estrategias de agregación. Las cifras son auxiliares y no deben interpretarse como una comparación de rendimiento. Se reportan dos medidas de memoria de GPU: la registrada por <i>nvidia-smi</i> a nivel de sistema (<i>GPU sist.</i>), afectada por otras aplicaciones del equipo, y el pico reservado por PyTorch al proceso del cliente (<i>GPU proc.</i>), métrica comparable entre estrategias.	34
7.5	Valoración de la flexibilidad de Flower según los criterios F1–F10. El tipo de evidencia distingue las capacidades ejecutadas en los experimentos (experimental) de las descritas en la documentación oficial (documental). . .	38
7.6	Resultados de NVIDIA FLARE en modo secuencial. La RAM corresponde al proceso principal.	39
7.7	Valoración de la usabilidad de NVIDIA FLARE según las cuatro dimensiones de la rúbrica (escala 1–5).	41
7.8	Resultados del mini-experimento de flexibilidad de NVIDIA FLARE. Las cifras de aprendizaje son auxiliares (tres rondas, una época, una semilla) y no constituyen una comparación de rendimiento. El dispositivo real procede del registro de ejecución, no del fichero de configuración del cliente.	44
7.9	Valoración de la flexibilidad de NVIDIA FLARE según los criterios F1–F10. El tipo de evidencia distingue las capacidades ejecutadas en los experimentos de las descritas en la documentación oficial.	46

7.10	Resultados de FedML en modo secuencial. La RAM corresponde al proceso principal.	47
7.11	Resultados de FedML en modo concurrente (backend MPI). No se dispone de precisión final extraída; solo se reportan tiempo y consumo de recursos. . .	48
7.12	Valoración de la usabilidad de FedML según las cuatro dimensiones de la rúbrica (escala 1–5).	50
7.13	Resultados del mini-experimento de flexibilidad de FedML. Las cifras de aprendizaje son auxiliares (dos rondas, una época, una semilla) y constituyen una comprobación funcional, no una comparación de rendimiento entre estrategias.	51
7.14	Valoración de la flexibilidad de FedML según los criterios F1–F10. El tipo de evidencia distingue las capacidades ejecutadas en los experimentos de las descritas en la documentación oficial.	54
7.15	Comparativa de la valoración de usabilidad de los tres frameworks según las cuatro dimensiones de la rúbrica (escala 1–5).	57
7.16	Comparativa de la valoración de flexibilidad de los tres frameworks según los criterios F1–F10 definidos en la metodología.	58

Índice de Códigos

3.1	Pseudocódigo de Federated Averaging (FedAvg)	8
6.1	Modos de ejecución en NVIDIA FLARE	26
6.2	Modos de ejecución en FedML	26
6.3	Modos de ejecución en Flower	26
7.1	Configuración de la federación en Flower (run_flower_experiments.sh)	32
7.2	Configuración de la aplicación en Flower (pyproject.toml)	32
7.3	Selección dinámica de la estrategia en Flower (server_app.py)	33
7.4	Configuración FedOpt del servidor en NVIDIA FLARE (config_fed_server.conf)	43
7.5	Configuración FedAdam vía FedOpt en FedML (fedml_config.yaml)	51

1 Introducción

El aprendizaje automático se utiliza hoy en muchos ámbitos de la ingeniería informática, como la visión por computador, el procesamiento del lenguaje natural, la medicina, la industria o la ciberseguridad. Buena parte de estos avances se apoya en el aprendizaje profundo (Goodfellow et al., 2016), cuyo rendimiento depende de la cantidad y la diversidad de los datos con que se entrena el modelo. Esos datos, además, no siempre están donde se necesitan: rara vez residen en un único repositorio y, con frecuencia, no pueden trasladarse sin restricciones a una infraestructura central.

El enfoque habitual ha sido el entrenamiento centralizado, que reúne en un servidor común los datos de las distintas fuentes. En muchos casos reales, en cambio, esos datos están repartidos entre dispositivos, usuarios u organizaciones, y centralizarlos es difícil o poco conveniente. La privacidad lo limita, transferir grandes volúmenes de información tiene un coste elevado y concentrar datos sensibles en un solo punto añade riesgo (Kairouz et al., 2021; Li, Sahu, Talwalkar & Smith, 2020). La colaboración entre entidades sigue siendo deseable, pero compartir los datos en bruto no siempre es una opción.

El aprendizaje federado (*Federated Learning*, FL) responde a esa necesidad: entrena modelos de forma colaborativa sin sacar los datos de los dispositivos o entidades donde se generan (Kairouz et al., 2021). El modelo se entrena localmente en cada cliente y, en lugar de los datos, solo viajan las actualizaciones resultantes, que un servidor central combina para formar un modelo global. El algoritmo de referencia es *Federated Averaging* (FedAvg), propuesto por McMahan et al. (2017), que organiza el proceso en rondas sucesivas de distribución del modelo, entrenamiento local y agregación.

Implementar un sistema federado, sin embargo, exige bastante más que entrenar una red neuronal. Hay que gestionar la comunicación entre servidor y clientes, definir estrategias de agregación, controlar las rondas, registrar métricas y dejar los experimentos en condiciones de reproducirse (Bonawitz et al., 2019). Para no rehacer toda esa infraestructura cada vez, han aparecido *frameworks* especializados que ofrecen componentes reutilizables con los que construir y evaluar este tipo de sistemas (Bonawitz et al., 2019; Caldas et al., 2018). El problema es que hay varios y no siempre está claro cuál conviene para un caso concreto, porque cada uno toma decisiones de diseño distintas y puede comportarse de otra manera ante la misma carga de trabajo.

Ahí se sitúa este Trabajo de Fin de Grado. Se propone comparar varios *frameworks* de aprendizaje federado de forma experimental: implementar en todos ellos un mismo escenario, en condiciones equivalentes, y medir tanto el aprendizaje del modelo (precisión y pérdida) como el coste en recursos (tiempo de ejecución, memoria, uso de GPU, potencia y energía estimada), la reproducibilidad y la facilidad de uso. No se busca decidir qué herramienta es mejor en términos absolutos, sino ofrecer una comparación razonada y reproducible que aclare sus diferencias prácticas y sirva de orientación en proyectos parecidos.

1.1 Contextualización del problema

Durante mucho tiempo, entrenar modelos de aprendizaje automático ha dependido de tener los datos centralizados. Cuando todos pertenecen a una misma organización y pueden almacenarse y procesarse sin restricciones, este enfoque funciona bien y simplifica el control del preprocesamiento y la evaluación del modelo. En muchos sistemas actuales, en cambio, la situación es distinta. Los datos están repartidos entre varias fuentes y llevarlos a un mismo lugar suele ser problemático (Kairouz et al., 2021).

El motivo no es solo la privacidad. La protección de datos personales o sensibles es una de las razones de peso para no centralizar, pero hay además limitaciones técnicas y organizativas. El volumen de datos que generan dispositivos y sensores puede hacer poco eficiente su transferencia continua; el almacenamiento centralizado encarece la infraestructura y convierte al servidor en un punto crítico de seguridad; y, en el plano organizativo, dos entidades independientes pueden no estar dispuestas a compartir sus bases de datos completas (Li, Sahu, Talwalkar & Smith, 2020).

Frente a este panorama, el aprendizaje federado permite entrenar un modelo de forma colaborativa sin reunir todos los datos en un mismo sitio. Lo que se comparte son las actualizaciones del modelo, no los datos originales, y eso lo hace apropiado para escenarios distribuidos o con datos sensibles. Conviene matizar, eso sí, que no elimina del todo los riesgos de privacidad (Kairouz et al., 2021). Estos fundamentos se desarrollan en el Capítulo 3.

1.2 Estructura de la memoria

El resto de la memoria se organiza de la siguiente manera. El Capítulo 2 concreta el objetivo general y los objetivos específicos que guían el trabajo. El Capítulo 3 reúne los fundamentos teóricos del aprendizaje automático y del aprendizaje federado, incluido el algoritmo *FedAvg*, los tipos de aprendizaje federado y los retos que plantea la heterogeneidad de datos y clientes. El Capítulo 4 revisa el panorama de herramientas disponibles y justifica la selección de los *frameworks* comparados. El Capítulo 5 describe la metodología experimental: el marco de evaluación en tres ejes —rendimiento, usabilidad y flexibilidad—, el control de variables, la reproducibilidad, las métricas empleadas y el alcance y las limitaciones del trabajo. El Capítulo 6 concreta el escenario común a las tres herramientas, la matriz de experimentos y los modos de ejecución secuencial y concurrente. El Capítulo 7 presenta los resultados de los tres ejes y su análisis comparativo entre *frameworks*. Por último, el Capítulo 8 recapitula la consecución de los objetivos, las aportaciones, las limitaciones y las líneas de trabajo futuro.

2 Objetivos

En este capítulo se concretan los objetivos que orientan el Trabajo de Fin de Grado. A partir de la motivación y el contexto expuestos en el Capítulo 1, se formula primero el objetivo general y, a continuación, se desglosa en una serie de objetivos específicos que estructuran el desarrollo experimental y guían la evaluación de los *frameworks* comparados.

2.1 Objetivo general

El objetivo general de este Trabajo de Fin de Grado es comparar de forma experimental varios *frameworks* de aprendizaje federado. Para ello se ejecuta un mismo escenario de entrenamiento en todos y se observa tanto el rendimiento del modelo como el consumo de recursos y la facilidad de uso.

2.2 Objetivos específicos

Para alcanzar este objetivo general se plantean los siguientes objetivos específicos:

1. **OE1:** Estudiar los fundamentos del aprendizaje federado, su funcionamiento, sus diferencias respecto al entrenamiento centralizado y sus principales retos técnicos.
2. **OE2:** Analizar la necesidad de utilizar *frameworks* especializados para implementar sistemas federados e identificar los aspectos que influyen en su elección.
3. **OE3:** Diseñar un escenario experimental común que permita comparar herramientas bajo condiciones equivalentes, manteniendo constantes el conjunto de datos, el modelo, los hiperparámetros y el entorno de ejecución en la medida de lo posible.
4. **OE4:** Implementar el experimento federado en los *frameworks* seleccionados, adaptando el flujo de entrenamiento a la arquitectura propia de cada herramienta.
5. **OE5:** Recoger métricas de aprendizaje, rendimiento y consumo de recursos: precisión, pérdida, tiempo de ejecución, uso de GPU, memoria empleada, potencia consumida y energía estimada.
6. **OE6:** Evaluar la experiencia práctica de uso de cada *framework*: instalación, configuración, documentación, claridad de los registros, facilidad de depuración y flexibilidad.
7. **OE7:** Comparar los resultados de forma crítica y señalar ventajas, limitaciones y escenarios en los que cada herramienta puede resultar más adecuada.

3 Fundamentos teóricos

Este capítulo reúne los conceptos en los que se apoya el estudio experimental. Empieza por las nociones de aprendizaje automático y aprendizaje profundo y pasa después al aprendizaje federado: su algoritmo de referencia, sus variantes y los retos que condicionan su evaluación. No se pretende repasar todo el campo de la inteligencia artificial, sino dejar la base necesaria para entender el diseño de los experimentos y la comparación entre *frameworks*.

3.1 Aprendizaje automático y aprendizaje profundo

El aprendizaje automático es la rama de la inteligencia artificial que busca que un sistema aprenda patrones a partir de datos, en lugar de seguir reglas programadas de antemano (Goodfellow et al., 2016). En el enfoque supervisado, el más extendido en este trabajo, el modelo se entrena con ejemplos etiquetados (pares de entrada y salida esperada) y aprende una función que asocia cada nueva entrada con la salida correcta, en tareas como la clasificación de imágenes o el reconocimiento de voz.

El aprendizaje profundo es la subárea que utiliza redes neuronales con varias capas, capaces de aprender representaciones complejas de los datos (Goodfellow et al., 2016). Durante el entrenamiento, el modelo genera una predicción para cada entrada, la compara con la salida esperada mediante una función de pérdida y ajusta sus parámetros para ir reduciendo ese error a lo largo de varias épocas. En clasificación, las dos métricas habituales son la precisión (*accuracy*), que indica la proporción de ejemplos bien clasificados, y la pérdida (*loss*), que cuantifica el error del modelo.

En visión por computador, las redes neuronales convolucionales han tenido un papel destacado por su capacidad para procesar datos con estructura espacial. Entre ellas, las redes residuales de K. He et al. (2016) facilitaron el entrenamiento de redes muy profundas gracias a las conexiones residuales; su arquitectura, ResNet, se emplea en este trabajo como modelo común para comparar los *frameworks*.

3.2 Del entrenamiento centralizado al aprendizaje federado

Desde un punto de vista técnico conviene distinguir tres enfoques de entrenamiento que a menudo se confunden. El entrenamiento centralizado es el más directo: todos los datos se reúnen en una misma infraestructura y el modelo se entrena sobre el conjunto completo, lo que simplifica el control del experimento y la evaluación. El entrenamiento distribuido reparte la carga de cálculo entre varios nodos para acelerar el proceso, pero suele asumir que los datos pueden accederse o repartirse de forma controlada dentro de una misma infraestructura. El aprendizaje federado, en cambio, parte de una premisa distinta: la descentralización de los datos no es un recurso para ganar velocidad, sino una condición impuesta por el problema (Kairouz et al., 2021).

Esta diferencia de premisa tiene consecuencias técnicas. En el aprendizaje federado el servidor nunca observa los datos, solo modelos entrenados sobre ellos, de modo que la distribución de los datos entre clientes deja de ser una decisión del diseñador y pasa a ser un dato del problema, normalmente desconocido y desigual. La Figura 3.1 contrasta los dos esquemas extremos: en el centralizado los datos se trasladan al servidor, mientras que en el federado permanecen en cada cliente y solo se intercambian el modelo global y los modelos locales resultantes.

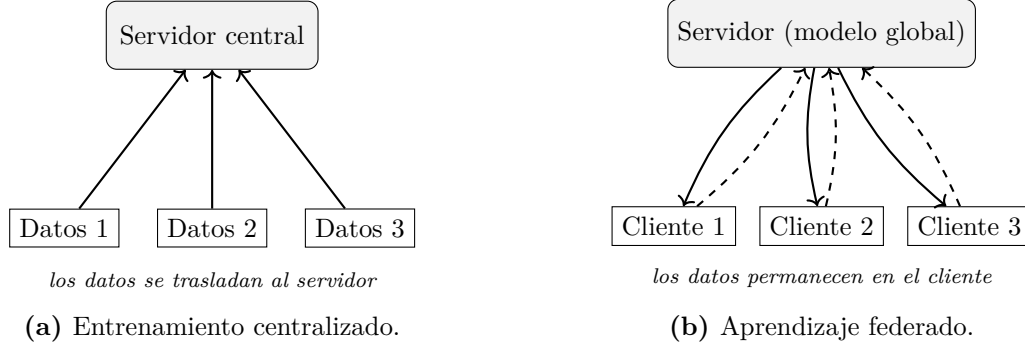


Figura 3.1: Comparación entre el entrenamiento centralizado, en el que los datos se reúnen en una infraestructura común, y el aprendizaje federado, en el que solo se intercambian el modelo global (línea continua) y los modelos locales entrenados (línea discontinua). Elaboración propia a partir de McMahan et al. (2017) y Kairouz et al. (2021).

3.3 Aprendizaje federado

El aprendizaje federado organiza el entrenamiento en torno a un servidor central y un conjunto de clientes que conservan sus datos locales (McMahan et al., 2017). El servidor coordina el proceso, mantiene el modelo global y agrega los modelos que recibe; los clientes entrenan ese modelo con sus propios datos. Las siguientes subsecciones describen cómo funciona una ronda y cuál es su algoritmo de referencia, las variantes algorítmicas que lo extienden, los tipos de aprendizaje federado que se distinguen y los retos que condicionan su evaluación.

3.3.1 Funcionamiento: la ronda federada y el algoritmo *FedAvg*

El proceso arranca con un modelo global inicializado en el servidor. En cada ronda, el servidor escoge un subconjunto de clientes y les envía el modelo actual; cada cliente lo entrena con sus datos locales durante un número fijo de épocas y devuelve su modelo local ya entrenado; el servidor combina esos modelos con una estrategia de agregación y obtiene una nueva versión del modelo global (McMahan et al., 2017). El ciclo, que recoge la Figura 3.2, se repite hasta completar el número de rondas previsto o hasta cumplir un criterio de parada.

En este proceso intervienen varios conceptos. Una *ronda federada* es una iteración completa del ciclo de distribución, entrenamiento local, recepción de los modelos y agregación. Una *época local* indica cuántas veces recorre cada cliente su conjunto de datos durante el entrenamiento. El *tamaño de lote* (*batch size*) fija cuántos ejemplos se procesan antes de actualizar

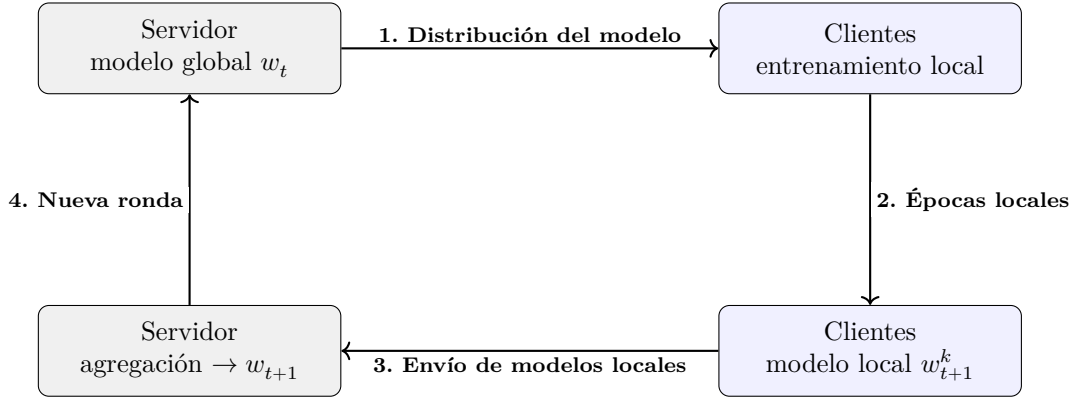


Figura 3.2: Ciclo de una ronda federada en *FedAvg*: el servidor distribuye el modelo global, cada cliente lo entrena con sus datos locales, devuelve su modelo local entrenado y el servidor agrega esos modelos para generar una nueva versión del modelo global (McMahan et al., 2017).

los parámetros, y la *tasa de aprendizaje* regula la magnitud de cada paso de actualización. Su configuración influye directamente en el coste y la estabilidad del entrenamiento: subir el número de épocas locales reduce la comunicación, pero puede aumentar la divergencia entre clientes cuando sus datos son muy heterogéneos (McMahan et al., 2017).

La terminología requiere una precisión adicional, ya que «actualización» se emplea a menudo de forma ambigua. En sentido estricto, en cada ronda el servidor recibe de cada cliente su *modelo local* ya entrenado, es decir, el vector de pesos w_{t+1}^k resultante de aplicar varias épocas de entrenamiento al modelo global recibido. Esto contrasta con otros esquemas en los que lo que se transmite es una *actualización* entendida como la diferencia (o gradiente acumulado) respecto al modelo de partida. El algoritmo de referencia del aprendizaje federado, *Federated Averaging* (FedAvg), opera sobre los modelos locales: agrega los pesos w_{t+1}^k ponderándolos por el número de ejemplos de cada cliente, y es precisamente la estrategia de agregación que se ejecuta dentro de la ronda descrita (McMahan et al., 2017).

Formalmente, *FedAvg* busca minimizar una función de pérdida global definida como la media de las pérdidas locales, ponderada por el número de ejemplos de cada cliente. Para K clientes, donde el cliente k dispone de n_k ejemplos y $n = \sum_{k=1}^K n_k$, el objetivo es:

$$\min_w F(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w), \quad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} \ell(w; x_i, y_i), \quad (3.1)$$

donde $F_k(w)$ es la pérdida media en el conjunto local $\mathcal{P}_k$ del cliente k y $\ell(\cdot)$ es la función de pérdida por ejemplo. En cada ronda t , el servidor selecciona un subconjunto de clientes S_t , les envía el modelo global w_t y cada cliente realiza varias épocas de descenso de gradiente estocástico sobre sus datos, con lo que obtiene su modelo local w_{t+1}^k . El servidor agrega después esos modelos locales mediante el promedio ponderado por el número de muestras:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{n_{S_t}} w_{t+1}^k, \quad n_{S_t} = \sum_{j \in S_t} n_j. \quad (3.2)$$

De este modo, los clientes con más datos pesan proporcionalmente más en el modelo global

resultante. El procedimiento completo se resume en el pseudocódigo del Código 3.1, que separa la lógica del servidor de la del cliente (McMahan et al., 2017).

Pseudocódigo de Federated Averaging (FedAvg)

```

1 # --- Servidor ---
2 inicializar modelo global  $w_0$ 
3 para  $t = 0, 1, \dots, T-1$ :
4      $S_t$  = seleccionar subconjunto de clientes
5     para cada cliente  $k$  en  $S_t$  (en paralelo):
6          $w_{\text{local}}[k] = \text{ClienteActualiza}(k, w_t)$       # modelo local
6         ↪ entrenado
7     # Agregacion: promedio ponderado de los MODELOS locales por  $n_k$ 
8      $n_{St} = \text{sumar } n_k \text{ para todo } k \text{ en } S_t$ 
9      $w_{(t+1)} = \text{sumar } (n_k / n_{St}) * w_{\text{local}}[k] \text{ para todo } k \text{ en } S_t$ 
10 devolver  $w_T$ 
11
12 # --- Cliente  $k$  ---
13 funcion  $\text{ClienteActualiza}(k, w)$ :
14     para cada epoca  $e = 1, \dots, E$ :
15         para cada lote  $b$  en  $\text{datos\_locales}[k]$ :
16              $w = w - \text{eta} * \text{gradiente}(\text{perdida}(w, b))$ 
17     devolver  $w$       # pesos locales
17     ↪ actualizados

```

La sencillez de *FedAvg* lo hace muy adecuado para experimentos comparativos: al reducir la complejidad algorítmica, deja el foco en el comportamiento de los *frameworks* y en el consumo de recursos. Su rendimiento, eso sí, puede empeorar cuando los datos se distribuyen de forma no IID entre los clientes, porque los modelos locales llegan a diferir bastante y dificultan la convergencia (Li, Sahu, Talwalkar & Smith, 2020). Aun con esa limitación, sigue siendo la referencia frente a la que se comparan métodos más sofisticados, y es el algoritmo que se emplea de manera uniforme en todos los *frameworks* evaluados en este trabajo.

3.3.2 Variantes de *FedAvg*

El carácter de referencia de *FedAvg* no impide reconocer sus limitaciones; de hecho, ha servido de punto de partida a varias variantes que buscan mejorar su convergencia, sobre todo en presencia de datos heterogéneos. La más directa es *Federated Proximal* (FedProx), propuesta por Li, Sahu, Zaheer et al. (2020), que conserva el esquema de *FedAvg* pero añade al entrenamiento local un término proximal, es decir, una penalización que limita cuánto pueden alejarse los pesos locales del modelo global recibido al inicio de la ronda. Con ello reduce la deriva de los clientes cuando sus distribuciones difieren y tolera mejor la heterogeneidad de recursos, al admitir que cada cliente realice una cantidad desigual de trabajo local.

Una segunda línea de mejora actúa sobre el lado del servidor. La familia de optimización federada adaptativa *Federated Optimization* (FedOpt), introducida por Reddi et al. (2021), generaliza *FedAvg* sustituyendo el promedio simple del servidor por un optimizador con momento y tasa de aprendizaje propios. Sus variantes más conocidas, FedAdam y FedAdagrad, emplean en el servidor los optimizadores Adam y Adagrad, respectivamente. Estas estrategias

pueden acelerar la convergencia, pero introducen hiperparámetros adicionales —la tasa de aprendizaje del servidor y un factor de adaptabilidad— cuya configuración resulta delicada, un aspecto que reaparece en los experimentos de flexibilidad de esta memoria (Sección 7.5.3).

Un tercer enfoque corrige de forma explícita la deriva entre clientes. *SCAFFOLD*, propuesto por Karimireddy et al. (2020), introduce variables de control que estiman y compensan la divergencia de cada cliente respecto a la dirección global, lo que mejora la convergencia con datos no IID a costa de transmitir información adicional en cada ronda. Estas variantes extienden *FedAvg*, pero no lo sustituyen como referencia: en este trabajo *FedAvg* se mantiene como algoritmo común a los tres *frameworks* para preservar un escenario controlado, mientras que FedProx, FedOpt, FedAdam y FedAdagrad intervienen únicamente como pruebas puntuales dentro del análisis de flexibilidad (Capítulo 7).

3.3.3 Tipos de aprendizaje federado

El aprendizaje federado no designa un escenario único, sino una familia de configuraciones que conviene distinguir, porque condicionan tanto el diseño del sistema como su evaluación. Una primera clasificación atiende al tipo y número de participantes. Kairouz et al. (2021) diferencian entre escenarios *cross-device* y *cross-silo*. El primero reúne a un número muy elevado de dispositivos de usuario final (teléfonos, tabletas o dispositivos del internet de las cosas), con disponibilidad intermitente, recursos limitados y conexiones inestables; aquí el sistema debe tolerar que solo participe una fracción de los clientes en cada ronda y que algunos abandonen a mitad del proceso. El segundo se da cuando intervienen pocas entidades, más estables y con mayor capacidad de cómputo, como hospitales, bancos o universidades, cada una con una cantidad apreciable de datos y una infraestructura fiable; en este caso la participación suele ser completa y el reto se desplaza hacia la heterogeneidad de las distribuciones y hacia las garantías de seguridad entre organizaciones.

Una segunda clasificación atiende a cómo se reparten los datos en el espacio de características. Yang et al. (2019) distinguen entre aprendizaje federado horizontal, vertical y por transferencia. En el horizontal los clientes comparten el mismo espacio de características pero poseen ejemplos distintos, como dos hospitales que registran las mismas variables sobre pacientes diferentes. En el vertical los clientes comparten parte de los individuos o entidades pero describen atributos distintos, como un banco y una aseguradora que conocen a los mismos clientes desde dimensiones diferentes; este caso exige alinear registros y suele recurrir a técnicas de cómputo seguro. El de transferencia se aplica cuando difieren a la vez los ejemplos y las características, y aprovecha el conocimiento de un dominio para mejorar otro relacionado.

Situar un experimento dentro de esta taxonomía es imprescindible para interpretar sus resultados. El estudio que se desarrolla en esta memoria corresponde a un escenario *cross-silo* con aprendizaje federado horizontal: un número reducido de clientes estables, simulados sobre una misma máquina, que comparten el espacio de características de un mismo conjunto de imágenes y se reparten ejemplos distintos. Esta delimitación permite mantener un escenario controlado y reproducible, en el que las diferencias observadas pueden atribuirse al *framework* y no a la configuración federada.

3.3.4 Retos del aprendizaje federado

El aprendizaje federado hereda los problemas del aprendizaje automático y añade otros propios de su naturaleza distribuida y descentralizada. El más característico es la heterogeneidad de los datos. A diferencia del entrenamiento centralizado, cada cliente solo observa una parte del problema, que puede ser desigual en cantidad y en distribución. Cuando esas distribuciones difieren de un cliente a otro se habla de datos no independientes e idénticamente distribuidos (no IID), una situación habitual en la práctica: dos usuarios de móvil generan datos muy distintos y dos hospitales atienden a poblaciones con características clínicas diferentes (Kairouz et al., 2021). Esta heterogeneidad afecta directamente al entrenamiento, ya que un cliente que observa una porción sesgada del problema produce un modelo local que se aleja del de los demás, lo que ralentiza la convergencia del modelo global o lo vuelve menos estable (Li, Sahu, Talwalkar & Smith, 2020). A ella se suma la heterogeneidad de recursos: no todos los clientes disponen de la misma capacidad de cómputo, memoria o conexión, lo que desequilibra su participación y obliga a diseñar sistemas tolerantes a esas diferencias. Por eso la forma de repartir los datos entre clientes influye tanto en la evaluación de un sistema federado y debe describirse de manera explícita en cualquier experimento.

Un segundo reto es el coste de comunicación. En cada ronda viaja información entre el servidor y los clientes, y ese volumen crece con el tamaño del modelo y con el número de participantes, hasta convertirse en el principal cuello de botella en escenarios con muchos dispositivos o con conexiones limitadas. De hecho, reducir el número de rondas necesarias y la información intercambiada fue una de las motivaciones originales de *FedAvg*, que aumenta el cómputo local de cada cliente para disminuir la frecuencia de las comunicaciones (Li, Sahu, Talwalkar & Smith, 2020; McMahan et al., 2017). Existe así un compromiso entre cómputo local y comunicación que cada sistema resuelve de forma distinta.

El tercer frente, y uno de los más delicados, es la seguridad y la privacidad. Aunque los datos originales no se transfieren, los modelos locales pueden filtrar información sobre los datos que los generaron mediante ataques de inferencia o de reconstrucción, y un cliente malicioso podría enviar modelos manipulados para degradar o sesgar el modelo global. Para mitigar estos riesgos se han desarrollado técnicas complementarias como la agregación segura, la privacidad diferencial o la detección de anomalías (Bonawitz et al., 2019; Kairouz et al., 2021). El aprendizaje federado reduce la necesidad de centralizar datos, pero no resuelve por sí solo todos los problemas de privacidad, lo que explica que algunos *frameworks* incorporen de serie estos mecanismos mientras que otros los dejan a cargo del desarrollador.

4 Estado del arte

En el capítulo anterior se justificó por qué construir un sistema de aprendizaje federado obliga a apoyarse en *frameworks* especializados. Este capítulo revisa el panorama de herramientas disponibles, explica con qué criterios se han seleccionado las tres que se comparan en el trabajo y por qué se han descartado otras, describe cada una con cierto detalle, las contrasta en una tabla comparativa y, por último, sitúa la aportación del proyecto frente a los estudios comparativos ya publicados.

4.1 Panorama de herramientas existentes

El número de *frameworks* de aprendizaje federado ha crecido con rapidez en los últimos años, con propuestas tanto académicas como industriales que responden a prioridades muy distintas (Riedel et al., 2024). Esa diversidad complica la elección, porque dos herramientas pueden resolver el mismo problema con arquitecturas, APIs y modos de ejecución muy diferentes (Bonawitz et al., 2019). A grandes rasgos, las herramientas más conocidas pueden agruparse según su orientación dominante.

Un primer grupo está formado por herramientas orientadas a la investigación y la experimentación. TensorFlow Federated, desarrollado por Google sobre TensorFlow, expone la lógica federada mediante una API funcional y un entorno de simulación pensado para prototipar algoritmos (The TensorFlow Federated Authors, 2019). FedML se presenta como biblioteca de investigación y banco de pruebas, con implementaciones de referencia que facilitan comparaciones reproducibles (C. He et al., 2020). Flower, por su parte, prioriza la facilidad de uso y la independencia respecto de la biblioteca de aprendizaje, lo que lo ha popularizado en entornos académicos (Beutel et al., 2020).

Un segundo grupo se orienta hacia la privacidad y los entornos de producción. PySyft, de la comunidad OpenMined, se centra en técnicas de protección de datos como la privacidad diferencial y el cómputo seguro multiparte (Ryffel et al., 2018). FATE, de WeBank, está orientado a aplicaciones industriales y ofrece soporte para aprendizaje horizontal, vertical y por transferencia con fuertes garantías de protección de datos (Liu et al., 2021). OpenFL, de Intel, se enfoca a escenarios *cross-silo* y ha tenido especial aceptación en el ámbito sanitario (Reina et al., 2021). NVIDIA FLARE se sitúa también en esta franja, con énfasis en el paso de la simulación al despliegue real y en la seguridad integrada (Roth et al., 2022).

Como cada herramienta responde a un conjunto de prioridades, no existe un único *framework* de referencia válido para todos los casos. Un trabajo reciente comparó quince herramientas de código abierto mediante un esquema de puntuación ponderado y situó a Flower en primera posición por características, interoperabilidad y facilidad de uso, por delante de FedML, FATE o TensorFlow Federated (Riedel et al., 2024). Sobre este panorama, el presente trabajo selecciona tres herramientas con enfoques complementarios: Flower, NVIDIA FLARE y FedML.

4.2 Criterios de selección

La selección de los *frameworks* comparados responde a varios criterios. Se ha buscado, en primer lugar, que las herramientas estén en mantenimiento activo, con versiones recientes y documentación al día, para que la comparación tenga validez en el momento de realizarse. En segundo lugar, se ha valorado que sean agnósticas respecto de la biblioteca de aprendizaje, de modo que el mismo modelo en PyTorch pueda ejecutarse en todas y la comparación no quede contaminada por diferencias de implementación del modelo. En tercer lugar, se ha exigido que ofrezcan un modo de simulación capaz de ejecutarse en una única máquina con una sola GPU, dado el entorno experimental disponible. Por último, se ha procurado que los tres representen filosofías de diseño distintas, de manera que la comparación resulte informativa.

Con esos criterios, varias herramientas conocidas quedan fuera del estudio. TensorFlow Federated se ha descartado porque su actividad de desarrollo se ha reducido de forma notable y no publica versiones nuevas desde hace tiempo, lo que lo hace poco recomendable para un trabajo que pretende comparar herramientas vigentes (The TensorFlow Federated Authors, 2019); a ello se añade que su API funcional y su fuerte acoplamiento a TensorFlow encajan mal con un experimento basado en PyTorch. FATE, pese a su madurez industrial, está concebido para despliegues empresariales de gran escala y su instalación y configuración resultan considerablemente más pesadas de lo que requiere un estudio comparativo controlado, con un enfoque orientado sobre todo al aprendizaje vertical entre organizaciones (Liu et al., 2021). OpenFL se ha descartado por su orientación más estrecha a escenarios *cross-silo* concretos y por una comunidad y un ecosistema de ejemplos menores que los de las alternativas elegidas (Reina et al., 2021). PySyft, por último, sitúa el acento en la privacidad y en el cómputo seguro más que en la comparación de rendimiento, y su abstracción difiere lo suficiente como para dificultar una comparación homogénea con las demás (Ryffel et al., 2018).

Frente a estas, las tres herramientas seleccionadas cumplen los criterios y cubren prioridades complementarias: Flower aporta la perspectiva de investigación centrada en la facilidad de uso, NVIDIA FLARE la de la transición hacia producción con seguridad integrada y FedML la del banco de pruebas reproducible. Que las tres sean agnósticas respecto de la biblioteca de aprendizaje y dispongan de un modo de simulación en una sola máquina permite ejecutar en todas un mismo experimento en PyTorch bajo condiciones equivalentes.

4.3 Frameworks seleccionados

Pese a sus distintas orientaciones, los tres *frameworks* comparten una arquitectura conceptual común, que la Figura 4.1 resume en forma de capas. Sobre el *hardware* de cómputo se apoya un *backend* de aprendizaje (en este trabajo, PyTorch), encargado de definir y entrenar el modelo. Por encima, un motor de ejecución y simulación gestiona la creación de los clientes y su ejecución, ya sea en procesos reales o simulados en una misma máquina. Una capa de comunicación traslada los modelos entre el servidor y los clientes mediante mecanismos como gRPC o el paso de mensajes. La estrategia de agregación, situada en el servidor, decide cómo se combinan los modelos locales (con *FedAvg* como opción habitual). En la cima, la lógica del experimento define el modelo, el reparto de los datos y las métricas. Las diferencias entre herramientas no están tanto en la existencia de estas capas como en cómo las exponen al desarrollador y en qué énfasis ponen en cada una.

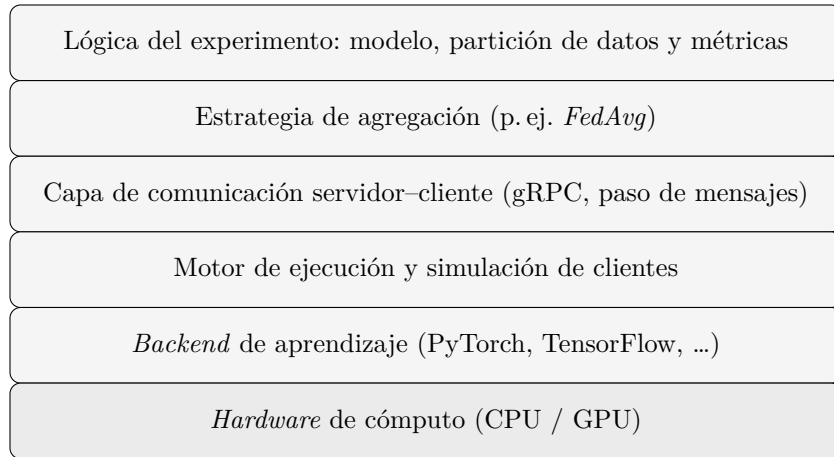


Figura 4.1: Arquitectura por capas común a los *frameworks* de aprendizaje federado comparados. Las herramientas se diferencian en cómo exponen cada capa al desarrollador y en el énfasis que ponen en la simulación, el despliegue o la seguridad. Elaboración propia.

4.3.1 Flower

Flower es un *framework* de aprendizaje federado presentado por Beutel et al. (2020) y surgido de un proyecto de investigación universitario. Su diseño es agnóstico respecto de la biblioteca de aprendizaje automático: el mismo sistema funciona con PyTorch, TensorFlow u otras, y separa con claridad la lógica del cliente (*ClientApp*) de la del servidor (*ServerApp*), de modo que el desarrollador solo implementa las funciones de entrenamiento y evaluación y delega en el *framework* la coordinación de las rondas (Beutel et al., 2020). Está pensado para experimentos a gran escala y para simular escenarios heterogéneos con muchos clientes; su motor de simulación permite instanciar numerosos clientes virtuales sobre los recursos disponibles, lo que lo ha popularizado en investigación. La estrategia de agregación se configura de forma explícita en el servidor, con *FedAvg* como opción por defecto y la posibilidad de sustituirla por otras. En el estudio de Riedel et al. (2024) obtuvo la puntuación más alta entre los quince *frameworks* analizados, en buena medida por su facilidad de uso y su interoperabilidad.

4.3.2 NVIDIA FLARE

NVIDIA FLARE (*NVIDIA Federated Learning Application Runtime Environment*) es un SDK de aprendizaje federado desarrollado por NVIDIA y descrito por Roth et al. (2022). Su lema, «de la simulación al mundo real», resume su orientación: facilitar que un mismo experimento pase de ejecutarse en simulación a desplegarse en una infraestructura real con cambios mínimos. Es igualmente agnóstico respecto de la biblioteca de entrenamiento, pues admite PyTorch, TensorFlow, XGBoost o incluso NumPy, y organiza la ejecución en torno a controladores y *workflows* que definen cómo se coordinan servidor y clientes (Roth et al., 2022). Su rasgo más distintivo es que incorpora de serie mecanismos de seguridad y privacidad, como la agregación con cifrado homomórfico, la privacidad diferencial o el filtrado de mensajes, y un modelo de gestión por sitios pensado para entornos con varias organizaciones. Esta orientación lo acerca a entornos empresariales y de producción, con presencia destacada en el ámbito sanitario, a costa de una configuración algo más estructurada que la de

herramientas centradas solo en la simulación.

4.3.3 FedML

FedML es una biblioteca de investigación y banco de pruebas para aprendizaje federado presentada por C. He et al. (2020) y mantenida posteriormente bajo el proyecto TensorOpera. Su objetivo es facilitar el desarrollo de nuevos algoritmos y permitir comparaciones justas y reproducibles; para ello ofrece una API flexible e implementaciones de referencia de optimizadores, modelos y conjuntos de datos, así como una separación clara entre el entrenador del cliente (*ClientTrainer*) y el agregador del servidor (*ServerAggregator*) (C. He et al., 2020). Admite tres paradigmas de cómputo: entrenamiento en el dispositivo (*edge*), cómputo distribuido y simulación en una sola máquina, de modo que un mismo experimento puede ejecutarse en configuraciones distintas (C. He et al., 2020). En su modo de simulación ofrece, además, dos motores de ejecución (uno secuencial de proceso único y otro basado en paso de mensajes que permite ejecutar clientes en paralelo), lo que resulta especialmente relevante para estudiar el comportamiento del *framework* bajo distintas configuraciones de concurrencia.

4.4 Comparativa de los frameworks seleccionados

La Tabla 4.1 resume las principales diferencias entre los tres *frameworks* a partir de la documentación oficial y de la bibliografía revisada. La comparación se organiza en torno a los aspectos que más influyen en la elección de una herramienta para un estudio como este: su orientación principal, el soporte de bibliotecas de aprendizaje, las capacidades de simulación y de despliegue real, los mecanismos de seguridad integrados, el estado de la documentación, la facilidad de uso percibida y su adecuación al experimento planteado. Conviene leerla como una síntesis cualitativa orientativa; las diferencias cuantitativas de rendimiento y consumo se abordan en los capítulos de resultados.

Tabla 4.1: Comparación cualitativa de los tres *frameworks* seleccionados.

Aspecto	Flower	NVIDIA FLARE	FedML
Orientación principal	Investigación y prototipado	De la simulación a producción	Investigación y <i>benchmarking</i>
Soporte de bibliotecas	Agnóstico (PyTorch, TensorFlow, etc.)	Agnóstico (PyTorch, TensorFlow, XGBoost, NumPy)	Agnóstico, centrado en PyTorch
Simulación	Motor de simulación a gran escala con clientes virtuales	Simulador integrado con paso a despliegue real	Simulación con motor secuencial y motor concurrente
Despliegue real	Soportado mediante despliegue de cliente y servidor	Eje central del diseño, con gestión por sitios	Soportado, con orientación a investigación
Seguridad y privacidad	A cargo del desarrollador o mediante extensiones	Integrada (cifrado homomórfico, privacidad diferencial, filtros)	Disponible según el paradigma de cómputo
Documentación	Extensa y orientada a ejemplos	Amplia, con enfoque empresarial	Amplia pero más fragmentada
Facilidad de uso	Alta, con API sencilla	Media, configuración más estructurada	Media, API flexible pero más densa
Adecuación al experimento	Alta: simulación directa en una GPU	Alta: simulación controlada y registros detallados	Alta: permite comparar configuraciones de concurrencia

4.5 Estudios comparativos previos

Comparar *frameworks* de aprendizaje federado no es una idea nueva, pero la mayoría de los trabajos publicados se centra en aspectos cualitativos o documentales. El estudio de Riedel et al. (2024), ya citado, evalúa quince herramientas según características, interoperabilidad y facilidad de uso mediante un esquema de puntuación ponderado, pero no mide cómo se comportan al ejecutar un mismo entrenamiento ni qué recursos consumen al hacerlo. Otra línea de trabajo analiza el efecto de la heterogeneidad de los datos sobre la precisión del modelo y muestra cómo las distribuciones no IID degradan la convergencia (Li, Sahu, Talwalkar & Smith, 2020), pero lo hace desde una perspectiva algorítmica, sin atender al coste de ejecución de cada herramienta. Por su parte, los trabajos que describen cada *framework* suelen proceder de sus propios autores y presentar las capacidades de la herramienta más que una comparación independiente entre varias (Beutel et al., 2020; C. He et al., 2020; Roth et al., 2022).

Queda así un hueco apreciable. Faltan comparaciones experimentales que, partiendo de un mismo escenario y en condiciones equivalentes, midan a la vez la precisión del modelo y el consumo de recursos de cada *framework*: tiempo de ejecución, memoria, uso de GPU y energía estimada, junto con aspectos prácticos como la reproducibilidad o la facilidad de uso. Cubrir parte de ese hueco es el objetivo de este trabajo.

4.6 Síntesis y posicionamiento

Los tres *frameworks* elegidos cubren prioridades complementarias: Flower apuesta por la facilidad de uso y la investigación, NVIDIA FLARE por la transición hacia producción con

seguridad integrada y FedML por la reproducibilidad y la comparación de algoritmos. Comparten, además, una arquitectura conceptual común y la capacidad de ejecutar en simulación un mismo experimento en PyTorch, lo que los hace idóneos para un estudio controlado. Ejecutarlos sobre un mismo escenario permite observar cómo esas decisiones de diseño se traducen en diferencias concretas de rendimiento, consumo de recursos y experiencia de uso, que es justamente la aportación que persigue este trabajo frente a los estudios previos. El siguiente capítulo describe la metodología experimental de esa comparación: el escenario común, el control de variables y las métricas empleadas.

5 Metodología

Este capítulo describe la metodología seguida para comparar de forma experimental los tres *frameworks* de aprendizaje federado seleccionados: Flower, NVIDIA FLARE y FedML. Se presentan primero el entorno de trabajo y el equipo sobre el que se ejecutaron las pruebas, y a continuación el marco de evaluación, que se articula en tres ejes: una comparativa cuantitativa de rendimiento y dos comparativas cualitativas, de usabilidad y de flexibilidad. Para cada eje se detallan las métricas, las rúbricas y los criterios empleados. El capítulo termina con las medidas adoptadas para garantizar la reproducibilidad y el control de variables, y con el alcance y las limitaciones del estudio. El escenario de entrenamiento común y la batería concreta de experimentos se describen en el Capítulo 6.

5.1 Entorno de trabajo

Todos los experimentos se ejecutaron sobre el mismo equipo y bajo el mismo sistema operativo, de modo que el entorno físico no introdujera diferencias entre las herramientas comparadas. Las ejecuciones se realizaron sobre un equipo portátil con procesador Intel Core i7-13700H de 13.^a generación (20 CPU lógicas), 14 GiB de memoria RAM y una GPU NVIDIA GeForce RTX 3050 Laptop con 6 GB de memoria de vídeo (VRAM), bajo Ubuntu 26.04 LTS. La GPU fue detectada correctamente por PyTorch en los tres entornos.

Cada *framework* se ejecutó, sin embargo, en un entorno Conda independiente, con versiones distintas de Python, PyTorch y CUDA, debido a las dependencias propias de cada herramienta. La Tabla 5.1 recoge las versiones exactas empleadas. Esta diferencia se reconoce como una amenaza menor a la validez de la comparación de rendimiento; se mitiga dejando registradas las versiones de cada caso, lo que además favorece la reproducibilidad.

Tabla 5.1: Versiones de software empleadas en cada entorno de ejecución.

<i>Framework</i>	Entorno	Python	Versión	PyTorch	CUDA
Flower	tfg-flower	3.12.13	1.29.0	2.11.0+cu128	12.8
NVIDIA FLARE	nvflare	3.12.13	2.7.2	2.11.0+cu130	13.0
FedML	fedml-tfg	3.10.20	0.9.6	2.12.0+cu126	12.6

5.2 Comparativa de rendimiento: métricas e instrumentación

La comparativa de rendimiento es la parte cuantitativa del estudio. Distingue entre métricas de aprendizaje, que describen la calidad del modelo obtenido, y métricas de consumo de recursos, que describen el coste de obtenerlo. Las métricas de aprendizaje son la precisión (*accuracy*) y la pérdida (*loss*) del modelo global sobre el conjunto de prueba centralizado,

junto con la pérdida media de entrenamiento. La evaluación sobre el conjunto de prueba se realiza de forma externa al entrenamiento, cargando el modelo global final y midiendo su comportamiento sobre las 10 000 imágenes reservadas, de modo idéntico para los tres *frameworks*.

Las métricas de consumo de recursos se recogen durante toda la ejecución y se resumen en la Tabla 5.2. Para la GPU se muestrea cada segundo la memoria utilizada, la utilización del núcleo, la potencia instantánea y la temperatura mediante `nvidia-smi`. El tiempo total de ejecución y la memoria residente máxima del proceso se obtienen con `/usr/bin/time -v`, y la evolución de la memoria del sistema se registra con `vmstat`.

Tabla 5.2: Métricas de rendimiento recogidas en cada ejecución y herramienta de medida.

Métrica	Unidad	Fuente
Precisión del modelo global	—	Evaluación centralizada
Pérdida del modelo global	—	Evaluación centralizada
Tiempo de ejecución	s	<code>/usr/bin/time -v</code>
Memoria de GPU (máx. y media)	MiB	<code>nvidia-smi</code>
Utilización de GPU	%	<code>nvidia-smi</code>
Potencia	W	<code>nvidia-smi</code>
Energía estimada	Wh	Cálculo (Ec. 5.1)
Temperatura de GPU	°C	<code>nvidia-smi</code>
Memoria residente máxima	KiB	<code>/usr/bin/time -v</code>

La energía no se mide con un vatímetro, sino que se estima a partir de la potencia muestreada. Si se toman N lecturas de potencia P_i a lo largo de una ejecución de duración T segundos, la energía estimada en vatios-hora se obtiene como el producto de la potencia media por la duración, expresado en horas:

$$E_{\text{Wh}} = \frac{1}{3600} \left(\frac{1}{N} \sum_{i=1}^N P_i \right) T. \quad (5.1)$$

Se trata, por tanto, de una estimación basada en el muestreo de potencia, adecuada para comparar herramientas entre sí en igualdad de condiciones, pero que no debe interpretarse como una medida absoluta de consumo eléctrico.

5.3 Comparativa de usabilidad: marco y rúbrica

La comparativa de usabilidad es cualitativa y persigue valorar qué *framework* resulta más sencillo de utilizar desde el punto de vista del desarrollador. No se plantea como un estudio de experiencia de usuario con participantes, sino como una rúbrica de evaluación experta aplicada por el autor del trabajo, apoyada en evidencias registradas durante la implementación: documentación consultada, pasos de instalación, errores encontrados, claridad de los registros, facilidad para configurar los experimentos y capacidad de modificación de cada herramienta.

El marco de valoración se organiza en torno a cuatro dimensiones de la experiencia de uso —*useful*, *usable*, *desirable* y *accessible*— inspiradas en el modelo de experiencia de usuario

de Morville (2004). Dicho modelo distingue siete facetas —*useful*, *usable*, *desirable*, *findable*, *accessible*, *credible* y *valuable*—, concebidas para productos de consumo y, en particular, para sitios web. De ellas se seleccionan y adaptan las cuatro pertinentes para evaluar una herramienta de desarrollo, que además pueden anclarse en estándares reconocidos. Las tres restantes se omiten por solaparse, en este contexto, con las elegidas: *findable* —la facilidad de localizar la documentación, los ejemplos o las opciones de configuración— queda subsumida en *accessible*; *credible* —la confianza en la herramienta por su mantenimiento, su comunidad y su estabilidad— se integra en *desirable*; y *valuable*, que en el modelo de Morville actúa como faceta de síntesis, coincide en lo esencial con *useful*. La dimensión *usable* se apoya además en la norma ISO 9241-11, que define la usabilidad en términos de eficacia, eficiencia y satisfacción en un contexto de uso (International Organization for Standardization, 2018), y en la descomposición de la usabilidad en aprendizaje, eficiencia, errores y satisfacción del Nielsen Norman Group (Nielsen, 1993). Para la dimensión *accessible* se adapta la noción de accesibilidad del W3C al contexto de las herramientas software, entendida como facilidad de acceso a la documentación, los ejemplos, la instalación y los mensajes comprensibles (World Wide Web Consortium, 2018). La Tabla 5.3 concreta los subcriterios de cada dimensión.

Tabla 5.3: Rúbrica de usabilidad. Cada subcriterio se valora con una escala de 1 a 5.

Dimensión	Subcriterios concretos	Cómo se mide
Useful	Permite implementar FedAvg, usar CIFAR-10 y ResNet-18, registrar métricas, ejecutar varios clientes y reproducir experimentos	Escala 1–5 según cobertura funcional
Usable	Instalación, configuración, curva de aprendizaje, claridad de la estructura, facilidad para lanzar experimentos y claridad de errores y registros	Escala 1–5 según esfuerzo y problemas encontrados
Desirable	Sensación práctica como desarrollador: comodidad, limpieza de la API, orden del proyecto, confianza al trabajar y experiencia de depuración	Escala 1–5 con justificación cualitativa
Accessible	Documentación accesible, ejemplos disponibles, guías actualizadas, mensajes comprensibles, requisitos claros y facilidad para empezar sin conocimiento previo profundo	Escala 1–5 según barreras de entrada

El significado de la escala se detalla en la Tabla 5.4. Para mitigar la subjetividad inherente a este tipo de valoración, los mismos subcriterios y la misma escala se aplican por igual a los tres *frameworks*, y cada puntuación se acompaña de la evidencia que la justifica. Aun así, se reconoce que la valoración refleja el juicio del autor como desarrollador, lo que se hace constar como limitación en la Sección 5.6.

Tabla 5.4: Escala de valoración empleada en la rúbrica de usabilidad.

Puntuación	Significado
1	Muy deficiente: difícil de usar o con barreras importantes
2	Deficiente: usable solo con bastante esfuerzo
3	Aceptable: permite trabajar, pero con limitaciones claras
4	Bueno: relativamente claro y práctico
5	Muy bueno: sencillo, claro, completo y bien documentado

5.4 Análisis de flexibilidad: criterios

La flexibilidad se trata como un eje separado de la usabilidad. Mientras que la usabilidad recoge la percepción de uso, la flexibilidad describe una propiedad arquitectónica: hasta qué punto la herramienta permite adaptarse a escenarios o necesidades distintos de los previstos por defecto. Interesa distinguir entre un *framework* cerrado, limitado a las funcionalidades que ya proporciona, y uno que permite integrar soluciones externas o modificar su funcionamiento.

La valoración se realiza mediante una rúbrica de tres niveles (Tabla 5.5) sobre el conjunto de criterios de la Tabla 5.6. Estos criterios definen qué se evalúa en cada caso; la valoración concreta de cada *framework* se presenta en el capítulo de resultados de usabilidad y flexibilidad.

Tabla 5.5: Escala de valoración empleada en el análisis de flexibilidad.

Puntuación	Significado
0	No soportado o no identificado en la documentación ni en los experimentos
1	Posible, pero con limitaciones, código manual o soporte experimental
2	Soporte claro, documentado y relativamente directo

5.5 Reproducibilidad y control de variables

Además del entorno descrito en la Sección 5.1, el estudio adopta varias medidas orientadas a la reproducibilidad y al control de las variables experimentales.

El control de variables descansa en mantener constantes, entre dos ejecuciones cualesquiera, el conjunto de datos, el modelo y los hiperparámetros —y un particionamiento IID equivalente entre herramientas, en el sentido precisado en la Sección 6.1.1—, definidos en el Capítulo 6, de manera que las únicas variables sean el *framework* evaluado y el modo de ejecución. La reproducibilidad se refuerza fijando una semilla aleatoria común (con valor 42) en los generadores de **random**, NumPy y PyTorch, y estableciendo además la semilla de **hash** del intérprete. Cada configuración se ejecutó una única vez con esa semilla; por tanto, las métricas se reportan como valores de una sola ejecución y no como media y desviación típica de varias repeticiones. Esta decisión, motivada por el elevado coste temporal de los experimentos más largos, se asume como una limitación en la Sección 5.6. Por último, antes de cada ejecución se guarda una instantánea del entorno —listado de paquetes, exportación del

Tabla 5.6: Criterios de flexibilidad evaluados. Cada criterio se valora de 0 a 2.

Código	Criterio	Qué se evalúa
F1	Personalización del modelo y el dataset	Posibilidad de usar modelos y conjuntos de datos propios, y librerías como PyTorch o torchvision, sin rehacer toda la arquitectura.
F2	Personalización del entrenamiento local	Facilidad para modificar el bucle de entrenamiento, el optimizador, la función de pérdida, las métricas, las épocas locales, el tamaño de lote y la lógica del cliente.
F3	Estrategias de agregación personalizadas	Capacidad para implementar o sustituir FedAvg por agregadores propios.
F4	Algoritmos federados disponibles	Disponibilidad o facilidad de uso de FedAvg, FedProx, FedOpt, SCAFFOLD u otros algoritmos avanzados.
F5	Soporte de aprendizaje federado asíncrono	Posibilidad de entrenamiento asíncrono de forma nativa, experimental o solo mediante modificaciones profundas. La asincronía reduce las esperas por clientes lentos, pero introduce el problema de las actualizaciones obsoletas.
F6	Modos de ejecución y despliegue	Soporte de simulación local, ejecución multiproceso, ejecución distribuida y escenarios <i>cross-silo</i> , <i>cross-device</i> o despliegue real.
F7	Integración con herramientas externas	Integración con PyTorch, TensorFlow, scikit-learn, Docker, herramientas de registro y seguimiento de métricas (MLflow, W&B) o scripts propios.
F8	Seguridad, privacidad y extensiones avanzadas	Posibilidad de añadir privacidad diferencial, agregación segura, filtros, cifrado o políticas de seguridad.
F9	Modificación de la arquitectura federada	Capacidad de alterar el flujo general: además del esquema servidor-cliente clásico, esquemas cíclicos, <i>swarm</i> , <i>split learning</i> o validación cruzada entre sitios.
F10	Reproducibilidad y configuración	Posibilidad de expresar la configuración mediante ficheros, scripts o recetas, y de repetir experimentos cambiando pocos parámetros.

entorno Conda, salida de `nvidia-smi` y de `lscpu`, y revisión del repositorio— que permite trazar las condiciones exactas de cada corrida. Además, el código de la implementación, los *scripts* de ejecución y los ficheros de configuración de los tres *frameworks* se han publicado en un repositorio público abierto (Luján Domenech, 2026), de modo que el estudio puede reproducirse de forma independiente.

5.6 Alcance y limitaciones del estudio

El estudio compara tres *frameworks* (Flower, NVIDIA FLARE y FedML) sobre un escenario controlado: CIFAR-10 con partición IID, modelo ResNet-18, algoritmo FedAvg, entre cuatro y diez clientes simulados en una única máquina, entre diez y treinta rondas, y los modos de ejecución secuencial y concurrente. La evaluación combina una comparativa cuantitativa de rendimiento con una comparativa cualitativa de usabilidad y flexibilidad. Quedan fuera del alcance los datos reales de entornos federados, los clientes distribuidos geográficamente, los algoritmos federados distintos de FedAvg en la parte cuantitativa y los mecanismos de ataque o defensa de la privacidad.

La principal limitación es de hardware. La GPU de 6 GB y los 14 GiB de memoria RAM condicionaron el modo concurrente en las tres herramientas, aunque de formas distintas. En Flower, la simulación se apoya en Ray y el factor limitante fue la memoria RAM del sistema: al alcanzar ciertos umbrales, Ray terminaba procesos o actores, un comportamiento que se mantuvo con independencia de ajustes como reducir el número de procesos de carga de datos o liberar la caché. En NVIDIA FLARE, aumentar la concurrencia mediante el número de hilos implica más procesos y clientes activos a la vez, lo que resultó limitado por la memoria y el número de procesos en la simulación concurrente; no se afirma que se produjera un agotamiento de memoria de GPU, al no disponer de un registro que lo demuestre. En FedML, el backend MPI sí proporcionó concurrencia real entre procesos de cliente, pero se mostró más sensible al consumo de memoria y podía fallar por falta de recursos en las configuraciones más grandes. Ningún *framework* quedó, por tanto, libre de impacto en el modo concurrente, si bien FedML con MPI fue el único que representó una concurrencia real entre procesos.

A esta limitación se añaden otras que conviene tener presentes al interpretar los resultados. Primero, cada herramienta se ejecutó en un entorno software distinto, con versiones diferentes de Python, PyTorch y CUDA, lo que constituye una amenaza menor a la validez de la comparación de rendimiento, mitigada al documentar las versiones exactas. Segundo, los experimentos se limitan a una distribución IID, por lo que no se evalúa el efecto de la heterogeneidad no IID. Tercero, los hiperparámetros se fijaron a priori, sin optimización individual por *framework*, de modo que los resultados corresponden a esa configuración concreta. Cuarto, cada configuración se ejecutó una sola vez con una semilla fija, de modo que no se dispone de medidas de variabilidad entre repeticiones y los valores deben interpretarse como una única observación por configuración. Quinto, toda la experimentación se realiza en simulación sobre una sola máquina, por lo que no se miden latencias de red reales ni efectos propios de un despliegue distribuido. Por último, las métricas de consumo de recursos dependen del hardware empleado y solo deben extrapolarse con cautela a otros equipos; las valoraciones de usabilidad y flexibilidad son más generales, pero incorporan el juicio del autor.

6 Diseño experimental e implementación

Este capítulo concreta la batería de experimentos ejecutada sobre los tres *frameworks* con la metodología descrita en el Capítulo 5. Se define primero el escenario de entrenamiento común a las tres herramientas —conjunto de datos, modelo y configuración de aprendizaje—, que se mantiene constante para que las diferencias observadas se atribuyan al *framework* y no a otros factores. A continuación se presentan la matriz de experimentos y los dos modos de ejecución, secuencial y concurrente, en los que se ejecuta cada configuración.

6.1 Diseño del experimento común

La comparación parte de un principio básico de control experimental: todo aquello que no es el propio *framework* debe permanecer constante. Por ese motivo, el conjunto de datos, el modelo, el algoritmo de aprendizaje federado, los hiperparámetros y el hardware son idénticos en las tres herramientas, y el particionamiento de los datos es equivalente entre ellas: misma naturaleza IID y mismos tamaños por cliente, aunque cada *framework* lo materialice con su propia tubería de carga de datos (Sección 6.1.1). La única variable que cambia entre ejecuciones es la herramienta evaluada y, dentro de cada una, el modo de ejecución descrito en la Sección 6.3.

La implementación del experimento en cada *framework* se apoyó en su documentación oficial y en los proyectos de ejemplo que proporciona cada herramienta, tomados como punto de partida y adaptados al escenario común descrito en este capítulo. En Flower se siguieron su guía y ejemplos oficiales (Beutel et al., 2020); en NVIDIA FLARE, la documentación del SDK y sus tutoriales (Roth et al., 2022); y en FedML, la documentación de la plataforma y sus implementaciones de referencia (C. He et al., 2020).

La implementación se reparte de forma deliberada en dos planos. Este capítulo recoge el plano común y operativo —el escenario idéntico para las tres herramientas y las órdenes concretas con que se ejecuta cada modo (Sección 6.3)—, mientras que las adaptaciones de código propias de cada *framework* —modelo, carga de datos, bucle de entrenamiento, estrategia de agregación y automatización de la ejecución— se documentan junto a la evaluación de flexibilidad de cada herramienta, en las Secciones 7.2.3, 7.3.3 y 7.4.3. Se ha optado por esta organización para mantener cada implementación al lado del análisis que la interpreta y evitar duplicar el código entre capítulos.

6.1.1 Conjunto de datos y particionamiento

Como banco de pruebas se emplea CIFAR-10, un conjunto de 60 000 imágenes en color de 32×32 píxeles distribuidas en diez clases. Se respeta la partición oficial de 50 000 imágenes de entrenamiento y 10 000 de prueba. Las 50 000 imágenes de entrenamiento se dividen, a su vez, en 40 000 para el entrenamiento federado y 10 000 para validación local en cada cliente,

mientras que las 10 000 imágenes de prueba se reservan para la evaluación centralizada del modelo global, ajena al entrenamiento.

El reparto entre clientes es independiente e idénticamente distribuido (IID). Para ello se barajan los índices con una semilla fija y se asignan en bloques de igual tamaño a cada cliente, de modo que todos reciben una muestra estadísticamente equivalente del conjunto global. Conviene matizar que cada *framework* construye ese reparto con su propia tubería de carga de datos —Flower mediante el particionador IID de `flwr-datasets` y NVIDIA FLARE y FedML mediante un barajado global con `torchvision`—, por lo que las particiones no contienen necesariamente las mismas imágenes en cada cliente, pero sí comparten la naturaleza IID y los mismos tamaños por cliente; las diferencias observadas no son, por tanto, atribuibles al reparto. Se ha optado por una distribución IID de forma deliberada: el objetivo es aislar el comportamiento del *framework* del efecto añadido de la heterogeneidad de datos, que introduciría una fuente de variación difícil de separar. El estudio de escenarios no IID queda planteado como ampliación futura y se recoge en la Sección 5.6.

Sobre las imágenes se aplican las transformaciones habituales para CIFAR-10. En entrenamiento se usa aumento de datos mediante recorte aleatorio con relleno de cuatro píxeles y volteo horizontal aleatorio; en validación y prueba no se aplica aumento. En todos los casos las imágenes se normalizan con las medias y desviaciones típicas por canal propias del conjunto.

6.1.2 Modelo

El modelo común es una red residual ResNet-18 adaptada a las dimensiones de CIFAR-10. La arquitectura original está pensada para imágenes de 224×224 píxeles, por lo que se sustituye la primera convolución por una de núcleo 3×3 con paso 1 y se elimina la capa de *maxpooling* inicial, de manera que la red no reduce en exceso la resolución de unas imágenes ya pequeñas. La capa totalmente conectada final se ajusta a diez salidas, una por clase. Se eligió ResNet-18 por tratarse de una arquitectura estándar, suficientemente expresiva para esta tarea y con un coste de cómputo asumible en una GPU de 6 GB. El modelo se implementa de forma idéntica en los tres *frameworks*.

6.1.3 Algoritmo y configuración de entrenamiento

El algoritmo de aprendizaje federado es Federated Averaging (FedAvg), descrito en el capítulo de fundamentos teóricos. En cada ronda, los clientes entrenan el modelo global con sus datos locales y el servidor promedia los modelos locales resultantes, ponderándolos por el número de ejemplos de cada cliente. El entrenamiento local emplea descenso de gradiente estocástico (SGD) con tasa de aprendizaje 0,02, momento 0,9 y *weight decay* de 5×10^{-4} , con un tamaño de lote de 32 y entropía cruzada como función de pérdida. Estos valores son comunes a las tres herramientas.

Los hiperparámetros no se determinaron mediante una búsqueda exhaustiva, sino que se fijaron a priori para disponer de una configuración común, controlada y comparable entre los tres *frameworks*. El objetivo del trabajo no es maximizar el rendimiento del modelo, sino comparar Flower, NVIDIA FLARE y FedML bajo idénticas condiciones; por ese motivo se escogieron valores moderados y estables, atendiendo a configuraciones habituales de FedAvg,

al hardware disponible y a una serie de pruebas preliminares que confirmaron que los experimentos se ejecutaban correctamente. Esta decisión se asume como una limitación del estudio (Sección 5.6): los resultados corresponden a esta configuración concreta y no a una optimización individual de cada herramienta.

6.2 Matriz de experimentos

A partir del escenario común se define una matriz de experimentos que varía de forma controlada el número de clientes, el número de rondas, las épocas locales y la fracción de clientes que participa en cada ronda. La Tabla 6.1 resume las configuraciones. El experimento e0 actúa como referencia; e1 constituye la configuración principal del estudio, con más épocas locales por ronda; e2 explora la escalabilidad variando el número de clientes; y e3 analiza la participación parcial, en la que solo una fracción de los clientes interviene en cada ronda.

Tabla 6.1: Matriz de experimentos del estudio. Cada configuración se ejecuta en los tres *frameworks* y en los dos modos de ejecución.

ID	Propósito	Clientes	Clientes/ronda	Rondas	Épocas locales
e0	Línea base de referencia	10	10	30	2
e1	Configuración principal	5	5	20	10
e2-c4	Escalabilidad (4 clientes)	4	4	10	2
e2-c5	Escalabilidad (5 clientes)	5	5	10	2
e2-c8	Escalabilidad (8 clientes)	8	8	10	2
e3	Participación parcial	8	4	20	10

El experimento e3 exige una aclaración terminológica. Mide la *participación parcial* de clientes, es decir, el efecto de que solo una fracción del total intervenga en cada ronda. No debe confundirse con la *flexibilidad* de cada herramienta, entendida como propiedad arquitectónica, que se analiza de forma cualitativa en la Sección 5.4. A lo largo de la memoria, el término flexibilidad se reserva para ese eje cualitativo.

6.3 Modos de ejecución: secuencial y concurrente

Cada configuración de la matriz se ejecuta en dos modos distintos. En el modo *secuencial* se mantiene una sola tarea de entrenamiento activa cada vez, de manera que los clientes de una ronda se procesan uno tras otro. En el modo *concurrente* se intenta mantener varios clientes o procesos activos a la vez, con el fin de observar cómo gestiona cada *framework* el paralelismo y qué impacto tiene en el consumo de recursos y en la estabilidad de la ejecución.

La forma de activar cada modo depende de la arquitectura de la herramienta. En NVIDIA FLARE, el simulador admite un parámetro que fija el número de hilos de entrenamiento activos: con un único hilo la ejecución es secuencial y, al aumentarlo hasta el número de clientes por ronda, se vuelve concurrente. El Código 6.1 muestra ambas invocaciones.

Modos de ejecución en NVIDIA FLARE

```
# Modo secuencial: un único hilo de entrenamiento activo
nvflare simulator -w workspace -n 5 -t 1 --gpu 0 job_dir

# Modo concurrente: tantos hilos como clientes por ronda
nvflare simulator -w workspace -n 5 -t 5 --gpu 0 job_dir
```

En FedML, el modo secuencial corresponde al backend de proceso único (**sp**), que entrena a los clientes de forma sucesiva, mientras que el modo concurrente se apoya en el backend MPI, lanzado con **mpirun**, que ejecuta varios procesos de cliente en paralelo (Código 6.2).

Modos de ejecución en FedML

```
# Modo secuencial: backend de proceso único
python main.py --cf config_sp.yaml

# Modo concurrente: backend MPI con varios procesos de cliente
mpirun --oversubscribe -np 6 python main.py --cf config_mpi.yaml
```

En Flower, la simulación se gestiona internamente mediante Ray y la ejecución se lanza con **flwr run**, indicando la configuración de la federación. A diferencia de NVIDIA FLARE y FedML, la concurrencia no se controla con un parámetro de hilos o de *backend*, sino con los recursos de GPU que se reservan para cada cliente mediante **client-resources-num-gpus**. Si cada cliente reserva la GPU completa (1,0), solo puede entrenar un cliente a la vez y la ejecución es secuencial; si reserva una fracción (0,5), varios clientes caben simultáneamente en la GPU y la ejecución se vuelve concurrente. El número total de clientes simulados se fija con **num-supernodes**. El Código 6.3 muestra ambas invocaciones.

Modos de ejecución en Flower

```
# Modo secuencial: cada cliente reserva la GPU completa,
# por lo que solo se entrena un cliente a la vez
flwr run . --federation-config="num-supernodes=5
↪ client-resources-num-gpus=1.0"

# Modo concurrente: cada cliente reserva una fracción de la GPU,
# permitiendo varios clientes simultáneos
flwr run . --federation-config="num-supernodes=5
↪ client-resources-num-gpus=0.5"
```

7 Resultados

Este capítulo reúne los resultados experimentales de la comparación entre Flower, NVIDIA FLARE y FedML sobre el escenario común descrito en el capítulo de metodología, integrando los tres ejes de evaluación del trabajo: el rendimiento cuantitativo del modelo, la usabilidad entendida como experiencia de desarrollo y la flexibilidad como propiedad arquitectónica. En lugar de tratar cada eje en un capítulo independiente, los resultados se organizan por *framework*: para cada herramienta se presentan de forma consecutiva su rendimiento, su usabilidad y su flexibilidad, de modo que el lector obtenga una visión completa de cada una antes de pasar a la siguiente. Tras el análisis individualizado, una sección de comparativa contrasta los tres *frameworks* en cada eje y una síntesis final recoge las conclusiones transversales.

Los tres ejes son complementarios y responden a preguntas distintas. El rendimiento mide el comportamiento cuantitativo del modelo —precisión, tiempo y consumo de recursos— bajo condiciones idénticas de datos, modelo e hiperparámetros. La usabilidad valora el coste y la experiencia de poner en marcha cada *framework*: el esfuerzo de instalación, las fricciones que aparecen durante la implementación y la depuración, y la claridad con la que cada herramienta permite trabajar. La flexibilidad describe hasta qué punto cada herramienta permite adaptarse a escenarios distintos de los previstos por defecto, modificando, sustituyendo o ampliando su comportamiento. Como se justifica más adelante, los resultados de aprendizaje comparables proceden del modo de ejecución secuencial; el modo concurrente se incorpora solo donde los datos lo permiten y, en los casos en que no fue viable, se documenta como una incidencia experimental.

7.1 Consideraciones previas

Antes de presentar los resultados conviene fijar algunos criterios de lectura comunes a los tres ejes. Cada eje aporta una evidencia de naturaleza distinta —cuantitativa en el rendimiento, cualitativa y experta en la usabilidad, y arquitectónica en la flexibilidad—, por lo que las cifras y valoraciones que siguen deben interpretarse dentro del marco metodológico de cada uno.

En lo que respecta al rendimiento, cada configuración corresponde a una ejecución con semilla fija. La precisión y la pérdida finales se refieren al modelo global evaluado sobre el conjunto de prueba; cada *framework* realiza esa evaluación con su propio mecanismo, por lo que pequeñas diferencias deben interpretarse con prudencia. Se descartaron las ejecuciones marcadas como correctas pero sin aprendizaje real (precisión estancada en torno a 0,10, equivalente al azar en diez clases) y las interrumpidas antes de completar las rondas.

Las primeras ejecuciones de FedML se vieron afectadas por un problema de instrumentación asociado a la librería NVML, que dejó sin métricas de GPU algunas corridas preliminares; las configuraciones afectadas se repitieron con la monitorización corregida, de modo que en la comparación final se emplean esas ejecuciones, ya con métricas de GPU, utilización, energía y memoria residente válidas. Hecha esta salvedad, dos limitaciones de medida afectan a la

lectura de los resultados de rendimiento. El consumo de memoria del sistema se obtiene con `/usr/bin/time` sobre el proceso principal. En el modo secuencial, esta cifra es completa para NVIDIA FLARE y para FedML, que se ejecutan en un único proceso (FedML emplea aquí el *backend* de proceso único `sp`), pero infravalora a Flower, cuyo motor de simulación Ray reparte el trabajo en procesos hijo que esta métrica no contabiliza; la salvedad equivalente para FedML solo aparece en el modo concurrente, donde su *backend* MPI sí lanza procesos de cliente separados. Por ello la memoria residente no es directamente comparable entre herramientas, y en particular la cifra de Flower debe leerse como una cota inferior. Por último, el seguimiento por rondas de algunas ejecuciones quedó incompleto, lo que se indica donde corresponde.

La valoración de usabilidad aplica a los tres *frameworks* la rúbrica definida en el Capítulo 5, organizada en torno a las cuatro dimensiones de la experiencia de uso *useful*, *usable*, *desirable* y *accessible*. A diferencia de la comparativa de rendimiento, que mide el comportamiento cuantitativo del modelo, y del análisis de flexibilidad, que describe la capacidad arquitectónica de cada herramienta para adaptarse a escenarios distintos del previsto, la usabilidad valora el coste y la experiencia de desarrollo: qué esfuerzo exigió poner en marcha cada *framework*, qué fricciones aparecieron durante la implementación y la depuración, y con qué claridad permitió trabajar. La descripción técnica de cada solución —arquitectura de archivos, adaptación del experimento y automatización— se recoge, en lo esencial, dentro del análisis de flexibilidad de cada *framework*, junto con la documentación oficial en la que se basó; las valoraciones de usabilidad no la repiten, sino que la interpretan desde el punto de vista del desarrollador.

Esa valoración es una evaluación experta cualitativa aplicada por el autor, no un estudio de experiencia de usuario con participantes. Cada dimensión se puntúa en la escala de uno a cinco descrita en el Capítulo 5 y se acompaña de la evidencia que la justifica: pasos de instalación reales, errores encontrados y resueltos, claridad de los registros y de los mensajes de error, y barreras de entrada de la documentación. Para mitigar la subjetividad, los mismos subcriterios y la misma escala se aplican por igual a los tres *frameworks*, recogidos sobre un único equipo y un mismo flujo experimental.

El eje de flexibilidad, por su parte, describe una propiedad arquitectónica: hasta qué punto la herramienta permite adaptarse a escenarios distintos de los previstos por defecto. Conviene recordar que este eje no debe confundirse con la participación parcial de clientes medida en el experimento e3, que es una variación cuantitativa del escenario; aquí el término flexibilidad se reserva para la capacidad de modificar, sustituir o ampliar el comportamiento del *framework*. La valoración se apoya en los diez criterios F1–F10 definidos en la Sección 5.4 y se puntúa con la escala de tres niveles (0, 1, 2) de la Tabla 5.5.

Para cada *framework* se distinguen dos planos de evidencia en el análisis de flexibilidad. El primero es la evidencia experimental: cambios efectivamente realizados en el código, la configuración y los *scripts*, y ejecuciones reales sobre el escenario común. El segundo es la evidencia documental: funcionalidades avanzadas descritas en la documentación oficial pero no implementadas en los experimentos del trabajo. Esta separación es deliberada y evita presentar como probado aquello que solo se ha revisado en la documentación, y preserva la comparabilidad del diseño experimental.

7.2 Flower

Esta sección reúne los resultados de Flower en los tres ejes de evaluación, comenzando por el rendimiento cuantitativo y siguiendo con la usabilidad y la flexibilidad.

7.2.1 Rendimiento de Flower

La Tabla 7.1 recoge los resultados secuenciales de Flower. El experimento de referencia (e0), con diez clientes y treinta rondas, alcanza un 87,84 %, mientras que la configuración principal (e1) obtiene la precisión más alta de todo el estudio, con un 92,03 %, y mantiene una utilización media de GPU elevada. Los experimentos de escalabilidad muestran el patrón esperable: al repartir el mismo conjunto entre más clientes, cada uno entrena con menos datos y la precisión final desciende. El experimento de participación parcial (e3) alcanza un 89,93 % con una duración coherente con esa configuración; su seguimiento por rondas, no obstante, quedó incompleto —se registraron 16 de las 20 rondas—, por lo que solo el valor final se emplea en la comparación.

Tabla 7.1: Resultados de Flower en modo secuencial. La RAM corresponde al proceso principal; en Flower, el motor de simulación Ray ejecuta procesos hijo, por lo que esta cifra infravalora el consumo real de memoria del sistema.

Exp.	Cli.	Ron.	Acc.	Loss	Tiempo (s)	GPU (MiB)	Util. (%)	Energía (Wh)	Temp. (°C)	RAM (GiB)
e0	10	30	0,8784	0,3608	3615	757	82,91	46,31	79	0,66
e1	5	20	0,9203	0,2650	11864	758	85,40	137,18	83	0,58
e2-c4	4	10	0,8451	0,4580	1196	760	83,57	14,92	72	0,68
e2-c5	5	10	0,8277	0,5071	1216	758	82,67	15,57	83	0,57
e2-c8	8	10	0,7839	0,6314	1240	762	80,82	16,02	85	0,58
e3	8	20	0,8993	0,3301	5202	761	82,21	53,52	87	0,69

La Figura 7.1 muestra la evolución de la precisión global del modelo por ronda en el experimento principal, según la evaluación del servidor. El registro por rondas conserva la serie completa de la ejecución hasta la ronda 20: la precisión parte de un valor próximo al azar, supera el 83 % en la ronda 3 y se estabiliza por encima del 90 % a partir de la ronda nueve, hasta finalizar con la precisión del 92,03 % recogida en la Tabla 7.1.

En modo concurrente, Flower no produjo resultados comparables en el equipo utilizado. La simulación concurrente lanza varios clientes en paralelo mediante Ray y, en repetidas ejecuciones, el nodo superó el umbral de memoria configurado por Ray, que terminó procesos de trabajo con el error *OutOfMemoryError*. Como consecuencia, las rondas recibían menos resultados de los esperados o la ejecución quedaba incompleta, con precisiones que no superaban el comportamiento inicial. Estos resultados se tratan, por tanto, como una incidencia del entorno y se retoman en el análisis del comportamiento concurrente (apartado 7.5.1.5).

7.2.2 Usabilidad de Flower

La usabilidad de Flower se valora a continuación según las cuatro dimensiones de la rúbrica de experiencia de uso.

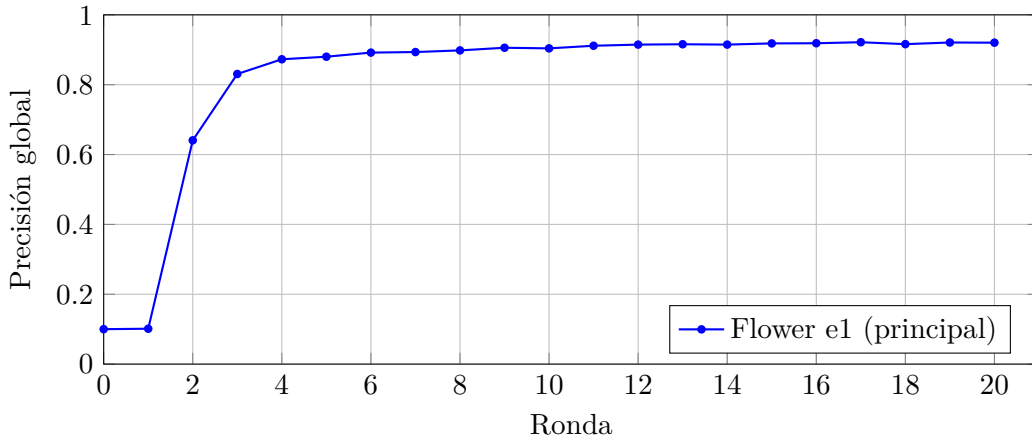


Figura 7.1: Convergencia de la precisión global de Flower en el experimento principal.

7.2.2.1 Utilidad funcional (*useful*)

Flower cubre por completo el caso de uso del estudio. Permitió implementar *Federated Averaging* sobre CIFAR-10 con una ResNet-18 adaptada, ejecutar varios clientes mediante simulación, recoger métricas de entrenamiento y evaluación, y reproducir los experimentos a partir de una configuración declarativa. La separación entre la aplicación de cliente, la aplicación de servidor y la lógica de tarea encaja de forma natural con el patrón cliente-servidor del aprendizaje federado, de modo que cada responsabilidad del experimento tiene un lugar evidente en el proyecto. La única limitación funcional relevante desde la óptica del uso es que la evaluación federada permanece desactivada cuando la fracción de evaluación es nula, lo que obliga a apoyar la medición de la precisión en una evaluación centralizada en el servidor; no es un fallo, sino una decisión de diseño que conviene conocer de antemano para no interpretar como anómalo el aviso correspondiente.

7.2.2.2 Facilidad de uso (*usable*)

La puesta en marcha de Flower fue de las más directas del estudio. La instalación se resolvió con un gestor de paquetes estándar y un conjunto reducido de dependencias, sin necesidad de recurrir a entornos especiales ni a versiones concretas del intérprete. La estructura del proyecto es compacta y predecible, y la configuración de cada experimento se concentra en un único fichero declarativo, lo que reduce el número de lugares donde intervenir para cambiar el número de rondas, la fracción de clientes o los hiperparámetros.

Durante las primeras ejecuciones aparecieron incidencias de configuración, como la ausencia de una clave esperada por el experimento, que se manifestaban como errores claros y localizables. Estas incidencias eran atribuibles a la propia parametrización inicial y no al framework, y se corrigieron en una sola sesión de trabajo propagando la configuración donde correspondía. La fricción de uso más significativa apareció en el modo concurrente, basado en un planificador de procesos que mantiene actores vivos entre rondas y acumula presión de memoria; en el equipo de pruebas, con memoria limitada, esa acumulación condujo a un agotamiento de memoria que impidió completar las ejecuciones concurrentes y obligó a tratarlas

como limitación experimental. En el modo secuencial, en cambio, el framework se comportó de forma estable y predecible.

7.2.2.3 Experiencia de desarrollo (*desirable*)

Como entorno de trabajo, Flower transmite una sensación de orden y limpieza. Su interfaz de programación se integra de forma idiomática con el ecosistema de PyTorch, de manera que el bucle de entrenamiento, la definición del modelo y la carga de datos se escriben de forma casi idéntica a un entrenamiento no federado, y solo la frontera de comunicación introduce conceptos específicos del framework. Esa cercanía reduce la carga cognitiva y aumenta la confianza al modificar el experimento. La contrapartida en la experiencia es que la capa de ejecución concurrente añade una complejidad que rompe esa sensación de control: cuando el planificador de procesos interviene, el comportamiento se vuelve menos transparente y la depuración de los problemas de memoria resulta menos inmediata que en el resto del flujo.

7.2.2.4 Accesibilidad (*accessible*)

Flower presenta una barrera de entrada baja. Su documentación es abundante y está orientada a casos prácticos, con ejemplos que sirven de punto de partida directo, y sus mensajes de registro durante la ejecución resultan legibles y suficientes para seguir la evolución de las rondas sin instrumentación adicional. Para una persona con conocimientos previos de PyTorch, el tiempo desde la instalación hasta la primera ejecución útil es corto. Las guías cubren con claridad la configuración básica, de modo que la mayor parte del esfuerzo inicial se dedica al experimento en sí y no a entender la herramienta.

7.2.2.5 Valoración de la usabilidad de Flower

Flower es el framework más accesible y agradable de usar de los tres para el caso de estudio, con un coste de aprendizaje bajo y un flujo secuencial muy estable; su punto débil de usabilidad se concentra en el modo concurrente. La Tabla 7.2 resume la valoración por dimensiones.

Tabla 7.2: Valoración de la usabilidad de Flower según las cuatro dimensiones de la rúbrica (escala 1–5).

Dimensión	Síntesis de la evidencia	Puntuación
<i>Useful</i>	Cobertura funcional completa; evaluación federada opcional	5
<i>Usable</i>	Instalación directa; estable en secuencial; concurrencia frágil	4
<i>Desirable</i>	API limpia e idiomática con PyTorch; capa concurrente menos clara	4
<i>Accessible</i>	Documentación práctica y abundante; logs legibles	5
Media		4,5

7.2.3 Flexibilidad de Flower

Flower es un framework agnóstico respecto a la biblioteca de aprendizaje automático y con una separación clara entre la lógica del cliente y la del servidor (Beutel et al., 2020), rasgos que condicionan directamente su flexibilidad. La evaluación experimental se centró en cinco capacidades: modificar la configuración federada, sustituir la estrategia de agregación, integrar un modelo y un conjunto de datos propios en PyTorch, registrar métricas personalizadas y alternar el modo de ejecución. Todas las ejecuciones se realizaron con **flwr** 1.29.0 sobre el backend de simulación Ray (Moritz et al., 2018), con Python 3.12 (entorno **tfg-flower**), una GPU NVIDIA GeForce RTX 3050 Laptop de 6 GB y el modelo ResNet-18 adaptado sobre CIFAR-10 descrito en el capítulo de metodología.

7.2.3.1 Adaptaciones para habilitar la evaluación

La primera evidencia de flexibilidad es la propia parametrización del escenario. Para que Flower recibiera la configuración de federación en cada ejecución, se ajustó el script de lanzamiento de modo que toda la configuración viajara en una única cadena pasada a **flwr run**, como muestra el Código 7.1. Con ello fue posible variar el número de supernodos o clientes y la fracción de GPU asignada a cada uno en función del modo de ejecución, sin tocar el código de la aplicación. El parámetro **init-args-num-cpus=2** no es cosmético: se incorporó para resolver un agotamiento de memoria RAM del anfitrión, una limitación que se detalla más adelante (apartado 7.2.3.7).

run_flower_experiments.sh

```
1 --federation-config="num-supernodes=$clients \  
2   init-args-num-cpus=2 \  
3   client-resources-num-cpus=1 \  
4   client-resources-num-gpus=$gpu_share"
```

Una segunda adaptación afectó a la declaración de la estrategia. Flower exige que las claves que se sobrescriben mediante **--run-config** estén previamente declaradas en la sección **[tool.flwr.app.config]** del fichero **pyproject.toml**; de lo contrario, la ejecución aborta con el mensaje **Key 'strategy' is not present in the main dictionary**. Por ese motivo se añadió la clave **strategy** a la configuración de la aplicación, junto con el resto de hiperparámetros del escenario (Código 7.2). Asimismo, el valor de **learning-rate** se fijó en 0,02 para que coincidiera con el que el script aplica de forma efectiva, eliminando una posible fuente de confusión entre el valor declarado y el realmente utilizado.

pyproject.toml

```
1 [tool.flwr.app.config]  
2 num-server-rounds = 3  
3 fraction-train = 0.5  
4 fraction-evaluate = 0.0  
5 local-epochs = 1
```

```

6 learning-rate = 0.02
7 batch-size = 32
8 strategy = "fedavg"

```

7.2.3.2 Selección dinámica de la estrategia de agregación

El núcleo de la evaluación experimental de flexibilidad fue la sustitución de la estrategia de agregación sin alterar el resto del sistema. Para ello se modificó el servidor de Flower de manera que la estrategia se seleccionara dinámicamente a partir de la configuración de ejecución, leyendo la clave `strategy` de `context.run_config`. El servidor instancia entonces la clase correspondiente y, en el caso de las estrategias adaptativas, fija los hiperparámetros del optimizador del servidor antes de iniciar el entrenamiento, como recoge el Código 7.3. Gracias a este diseño, el mismo código de cliente, modelo y conjunto de datos puede ejecutarse con distintas estrategias modificando únicamente la lógica del servidor.

server_app.py

```

1 from flwr.serverapp.strategy import FedAvg, FedAdagrad, FedAdam
2
3 strategy_name = str(context.run_config.get("strategy", "fedavg")).lower()
4
5 if strategy_name == "fedadam":
6     strategy_class = FedAdam
7     adaptive_kwargs = dict(
8         eta=float(context.run_config.get("server-lr", 0.01)),
9         tau=float(context.run_config.get("tau", 0.01)),
10    )
11 elif strategy_name == "fedadagrad":
12     strategy_class = FedAdagrad
13     adaptive_kwargs = dict(
14         eta=float(context.run_config.get("server-lr", 0.01)),
15         tau=float(context.run_config.get("tau", 0.01)),
16    )
17 else:
18     strategy_class = FedAvg
19     adaptive_kwargs = {}
20
21 strategy = strategy_class(**common_kwargs, **adaptive_kwargs)

```

Las estrategias FedAdam y FedAdagrad pertenecen a la familia de optimización federada adaptativa propuesta por Reddi et al. (2021), que generaliza FedAvg sustituyendo el promedio del servidor por un optimizador con momento y tasa de aprendizaje propios. Con los valores por defecto de Flower (`eta` = 0,1 y `tau` = 0,001), ambas estrategias divergieron numéricamente en este escenario: la pérdida del modelo global creció hasta el orden de 10^9 mientras las pérdidas locales de entrenamiento descendían con normalidad. El problema se localiza en el paso de agregación del servidor, donde una tasa de aprendizaje alta combinada con un factor de adaptabilidad pequeño produce primeros pasos desproporcionados antes de que el segundo momento se estabilice. Reducir la tasa del servidor a `eta` = 0,01 y elevar el

factor a $\tau = 0,01$ estabilizó el entrenamiento. Lejos de restar valor a la evaluación, este ajuste evidencia precisamente la flexibilidad buscada: Flower expone esos hiperparámetros de agregación y permite configurarlos desde el servidor.

7.2.3.3 Mini-experimento de flexibilidad

Para comprobar la sustitución de estrategias de forma controlada se definió una variante reducida del experimento e3, denominada **e3_flex_mini**, cuyos parámetros recoge la Tabla 7.3. La finalidad de esta prueba no era comparar el rendimiento de aprendizaje, sino demostrar que Flower permite cambiar la estrategia federada y la configuración de participación sin rehacer la implementación, reduciendo a la vez el coste temporal de las pruebas. Se ejecutó con una única semilla (42) y una repetición, por lo que las cifras de aprendizaje son auxiliares y no llevan estimación de variabilidad.

Tabla 7.3: Configuración del mini-experimento de flexibilidad de Flower.

Parámetro	Valor
Clientes totales	8
Clientes por ronda	4
Fracción de entrenamiento	0,5
Rondas federadas	3
Épocas locales	1
Tamaño de lote	32
Tasa de aprendizaje (cliente)	0,02
η servidor (FedAdam/FedAdagrad)	0,01
τ (FedAdam/FedAdagrad)	0,01
Modo de ejecución	Secuencial
Repeticiones / semilla	1 / 42

Los tres mini-experimentos terminaron correctamente, lo que confirma que el mismo flujo se ejecutó con FedAvg, FedAdam y FedAdagrad manteniendo constantes el conjunto de datos, el modelo, el número de clientes, los clientes por ronda, las rondas, las épocas locales, el tamaño de lote y la tasa de aprendizaje del cliente. La Tabla 7.4 resume las métricas obtenidas.

Tabla 7.4: Resultados del mini-experimento de flexibilidad de Flower con tres estrategias de agregación. Las cifras son auxiliares y no deben interpretarse como una comparación de rendimiento. Se reportan dos medidas de memoria de GPU: la registrada por **nvidia-smi** a nivel de sistema (*GPU sist.*), afectada por otras aplicaciones del equipo, y el pico reservado por PyTorch al proceso del cliente (*GPU proc.*), métrica comparable entre estrategias.

Estrategia	Estado	Acc.	Loss	Dur. (s)	GPU sist. (MiB)	GPU proc. (MiB)	Energía (Wh)
FedAvg	success	0,4257	1,5265	132	757	≈ 289	1,272
FedAdam	success	0,1410	2,3401	124	757	≈ 289	1,252
FedAdagrad	success	0,1000	18,7832	144	1623	≈ 289	1,123

Estos resultados no permiten concluir que una estrategia sea superior a otra, por dos

motivos. En primer lugar, se trata de un mini-experimento de solo tres rondas y una época local, insuficiente para que las estrategias adaptativas estabilicen su segundo momento y converjan; con los hiperparámetros ajustados ya no divergen, pero todavía no alcanzan el comportamiento de FedAvg. En segundo lugar, se ejecutó con una única semilla, sin medida de variabilidad. Conviene además una advertencia de medición: la memoria de GPU reportada por `nvidia-smi` a nivel de sistema mostró un valor anómalo de 1623 MiB en la ejecución de FedAdagrad frente a 757 MiB en las otras dos, atribuible a memoria ocupada por otras aplicaciones del equipo durante esa corrida; el consumo real por proceso fue prácticamente idéntico (≈ 289 MiB) en las tres estrategias, por lo que para cualquier comparación debe emplearse la métrica por proceso. La conclusión experimental válida es que Flower permitió sustituir la estrategia federada entre FedAvg, FedAdam y FedAdagrad sin modificar el cliente, el modelo ni el conjunto de datos, lo que confirma su flexibilidad para alterar la lógica de agregación desde el servidor y la configuración de ejecución.

7.2.3.4 Integración de modelo, datos y métricas

Más allá de las estrategias, la evaluación confirmó que Flower integra sin fricción un modelo y un conjunto de datos propios. El modelo es la ResNet-18 adaptada a CIFAR-10, construida en PyTorch sustituyendo la primera convolución por una de núcleo 3×3 y eliminando el *maxpool* inicial, mientras que el conjunto de datos se carga mediante `FederatedDataset` con un particionador IID. El bucle de entrenamiento local es código propio del cliente: emplea descenso de gradiente estocástico con momento y *weight decay*, entropía cruzada como función de pérdida y un número de épocas, tasa y tamaño de lote configurables. Además, el cliente registra métricas personalizadas, entre ellas la pérdida de entrenamiento, el número de ejemplos, la duración y el pico de memoria de GPU asignada por PyTorch, que el script de ejecución vuelca junto con los registros completos en ficheros de resumen reutilizables. Esta combinación de modelo propio, bucle de entrenamiento editable y métricas a medida sustenta una valoración alta en los criterios de personalización, integración y reproducibilidad.

7.2.3.5 Capacidades avanzadas según la documentación

Aparte de lo ejecutado, la documentación oficial de Flower —consultada en su versión vigente (1.31), compatible con la serie 1.29 empleada en los experimentos— describe un conjunto amplio de técnicas de flexibilidad que se valoran como evidencia documental. En el plano de la configuración y el control del servidor, se detallan cuatro vías de personalización de las estrategias: ajustar los parámetros de una estrategia existente (`fraction_train`, `fraction_evaluate`, `min_available_nodes`); inyectar funciones de devolución como `evaluate_fn` que el servidor ejecuta en momentos concretos; sobrescribir métodos internos como `configure_train`, `aggregate_fit` o `aggregate_evaluate`; e implementar una estrategia nueva heredando de `BaseStrategy` (The Flower Authors, 2026). La misma documentación muestra cómo enviar configuraciones por ronda mediante `ConfigRecord` —por ejemplo, reducir progresivamente la tasa de aprendizaje leyendo `server_round`— y cómo agregar métricas con criterios propios a través de `evaluate_metrics_aggr_fn`, en lugar del promedio ponderado por número de ejemplos. Los clientes, sin estado por defecto, pueden conservar información entre rondas mediante `Context.state`, lo que habilita técnicas como el *checkpointing* en el cliente.

Una pieza distintiva son los *Mods*, funciones que interceptan los mensajes entre servidor y cliente para transformarlos sin alterar la lógica principal; se encadenan en un orden definido y permiten, por ejemplo, recortar gradientes (`fixedclipping_mod`, `adaptiveclipping_mod`) o limitar el tamaño de los mensajes (`message_size_mod`) (The Flower Authors, 2026). Sobre esta base se construye la privacidad diferencial, documentada en modo preliminar en tres variantes —central del lado del servidor, central del lado del cliente y local mediante `LocalDpMod`—, todas con recorte y ruido gaussiano configurables, a las que se suma la agregación segura. La documentación también ejemplifica esquemas no estándar, como FedBN —que excluye los parámetros de normalización por lotes del intercambio para preservar las estadísticas locales de cada cliente—, el aprendizaje federado vertical y la integración con XGBoost.

La documentación recoge asimismo soporte para el entrenamiento asíncrono, si bien de carácter experimental y fuera de las guías principales. Las notas de versión describen una *Driver API* experimental (versión 1.2) que ejecuta un servidor programable, asíncrono y multi-inquilino, y la *API Flower Next* (versión 1.8), un conjunto de primitivas de bajo nivel que permite personalizar el bucle de entrenamiento e incluye, de forma explícita, soporte para aprendizaje federado asíncrono y esquemas cíclicos, con un `ServerApp` capaz de permanecer activo de forma indefinida (The Flower Authors, 2026). Por su naturaleza en desarrollo y de bajo nivel, esta capacidad se valora como soporte parcial y no como una funcionalidad directa y estable. Todas estas capacidades se reconocen como existentes, pero no se contabilizan como demostradas empíricamente en este trabajo.

7.2.3.6 Ejecución, despliegue y operación según la documentación

La flexibilidad de Flower se extiende a su modelo de ejecución y despliegue. En local, los comandos `flwr run` y `flwr list` levantan automáticamente un *SuperLink* que gestiona las ejecuciones, mientras que las simulaciones se lanzan sobre el *Simulation Runtime*, cuyo número de *SuperNodes* y recursos por cliente se fijan de forma persistente o se sobrescriben por ejecución; la documentación contempla incluso simulaciones sobre un clúster Ray multi-anfitrión (The Flower Authors, 2026). Para despliegues distribuidos, la federación se compone de un *SuperLink* que coordina y varios *SuperNodes* que ejecutan los clientes, modelo que la documentación lleva hasta entornos de producción en la nube —Google Cloud sobre Kubernetes, Microsoft Azure y Red Hat OpenShift— e incluso a la interconexión de varios clústeres mediante Skupper.

Este recorrido se acompaña de mecanismos de seguridad operativa que también cuentan para la flexibilidad: cifrado de las comunicaciones mediante TLS con certificados propios, autenticación de *SuperNodes* basada en firmas y lista blanca de claves públicas, autenticación de cuentas de usuario mediante OpenID Connect con autorización fina y un registro de auditoría en formato JSON que traza qué actor ejecuta cada acción (The Flower Authors, 2026). Por último, la interfaz de línea de comandos puede emitir su salida en formato JSON mediante la opción `--format json` para integrarse en flujos automatizados, y las herramientas de gestión de federaciones permiten inspeccionar nodos, ejecuciones y estado. Ninguna de estas capacidades se ejercitó en los experimentos, pero refuerzan la valoración de Flower en los modos de ejecución y despliegue y en la integración con herramientas externas.

7.2.3.7 Limitaciones observadas

El hallazgo más relevante de la evaluación afecta a la memoria RAM del anfitrión y no a la GPU. El motor de simulación de Flower delega en Ray, que al iniciarse detecta los procesadores del equipo (veinte en este caso) y prelanza un proceso *worker* ocioso por cada uno; cada worker carga el intérprete y las bibliotecas (en torno a 0,26 GiB), de modo que el conjunto de workers, sumado al proceso de simulación y al resto del sistema, agotaba los aproximadamente 14 GiB de RAM del equipo. Ray respondía terminando procesos con un error de memoria agotada, lo que ocurría incluso en modo secuencial, donde a priori solo hay un cliente entrenando a la vez. La solución fue limitar el número de procesadores que ve la *Simulation Runtime* mediante `init-args-num-cpus=2`, lo que reduce drásticamente la huella de RAM. Se trata de un resultado comparativo de interés: el backend basado en Ray presenta una huella de memoria de anfitrión notablemente mayor que el simulador de NVIDIA FLARE o el backend de proceso único de FedML, que se ejecutan en un único proceso, por lo que en equipos con poca RAM la simulación de Flower obliga a acotar los recursos manualmente.

La ejecución concurrente, probada también mediante Ray con una fracción de GPU por cliente de 0,5, quedó condicionada por esta misma limitación de memoria, en línea con lo descrito en la valoración de rendimiento de este framework (Subsección 7.2.1) y en el análisis del comportamiento concurrente (apartado 7.5.1.5). Por su parte, la divergencia de las estrategias adaptativas con los valores por defecto, ya comentada, es una limitación de uso y no del framework: las estrategias de la familia FedOpt son sensibles a la tasa de aprendizaje del servidor y requieren un ajuste deliberado, especialmente con pocas rondas (Reddi et al., 2021). Ninguna de estas limitaciones impidió completar la evaluación, pero todas matizan la valoración final.

7.2.3.8 Valoración de la flexibilidad de Flower

La Tabla 7.5 resume la valoración de Flower sobre los diez criterios F1–F10, distinguiendo el tipo de evidencia que sustenta cada puntuación. Los criterios de personalización del modelo y los datos (F1), del entrenamiento local (F2), de la estrategia de agregación (F3) y de los algoritmos federados disponibles (F4) se valoran con la máxima puntuación a partir de evidencia experimental directa, lo mismo que la reproducibilidad (F10). Los modos de ejecución y despliegue (F6) y la integración con herramientas externas (F7) alcanzan también la máxima puntuación, en su caso combinando evidencia experimental y documental. El soporte de aprendizaje asíncrono (F5), la seguridad y privacidad (F8) y la modificación de la arquitectura federada (F9) reciben puntuación parcial por estar respaldados solo por documentación; en el caso de F5, mediante interfaces de bajo nivel todavía en desarrollo (la *Driver API* y *Flower Next*), por lo que se reconoce el soporte sin elevarlo a la máxima puntuación.

Flower obtiene una valoración alta en flexibilidad (17/20). La evidencia experimental muestra una herramienta cómoda de adaptar: permitió integrar un modelo y un conjunto de datos propios en PyTorch, editar el bucle de entrenamiento local, sustituir la estrategia de agregación entre FedAvg, FedAdam y FedAdagrad desde el servidor —incluyendo el ajuste de sus hiperparámetros— y automatizar la recogida de métricas. La valoración no alcanza el máximo porque varias capacidades avanzadas —el aprendizaje asíncrono, la privacidad diferencial, la agregación segura o los esquemas no horizontales— solo se verificaron documentalmente,

Tabla 7.5: Valoración de la flexibilidad de Flower según los criterios F1–F10. El tipo de evidencia distingue las capacidades ejecutadas en los experimentos (experimental) de las descritas en la documentación oficial (documental).

Criterio	Descripción	Punt.	Evidencia
F1	Personalización del modelo y el dataset	2/2	Experimental
F2	Personalización del entrenamiento local	2/2	Experimental
F3	Estrategias de agregación personalizadas	2/2	Experimental
F4	Algoritmos federados disponibles	2/2	Experimental
F5	Aprendizaje federado asíncrono	1/2	Documental
F6	Modos de ejecución y despliegue	2/2	Exp. + documental
F7	Integración con herramientas externas	2/2	Exp. + documental
F8	Seguridad, privacidad y extensiones	1/2	Documental
F9	Modificación de la arquitectura federada	1/2	Documental
F10	Reproducibilidad y configuración	2/2	Experimental
Total		17/20	

y porque el soporte asíncrono se apoya en interfaces todavía experimentales; a ello se añade que la viabilidad práctica del modo concurrente quedó condicionada por la elevada huella de memoria del backend Ray en el equipo empleado. Estas conclusiones se contrastan con las de NVIDIA FLARE y FedML en la comparativa de flexibilidad (Subsección 7.5.3).

7.3 NVIDIA FLARE

Esta sección presenta los resultados de NVIDIA FLARE en los tres ejes de evaluación, siguiendo el mismo orden empleado con Flower: rendimiento, usabilidad y flexibilidad.

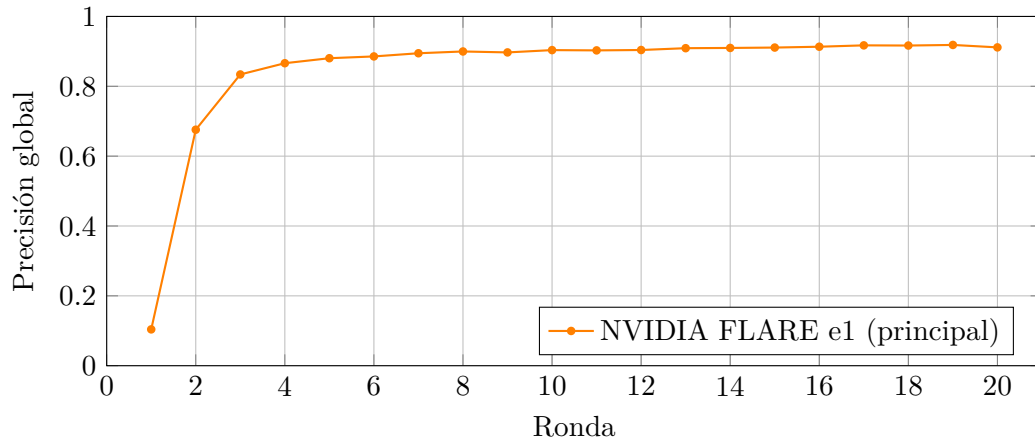
7.3.1 Rendimiento de NVIDIA FLARE

La Tabla 7.6 resume los resultados secuenciales de NVIDIA FLARE. La configuración principal alcanza un 91,14 % de precisión, muy cerca de Flower, pero con el tiempo de ejecución más alto del estudio (algo más de tres horas) y una utilización media de GPU elevada. El comportamiento en escalabilidad reproduce de nuevo la caída de precisión al aumentar el número de clientes, y el experimento de participación parcial obtiene un buen resultado (90,86 %) completando las veinte rondas.

En el flujo estándar, NVIDIA FLARE no evalúa el modelo global de forma centralizada en cada ronda, sino solo al final de la ejecución. Para obtener una curva de convergencia equivalente a la de Flower y FedML, el experimento principal se ejecutó guardando el modelo global agregado de cada ronda; esos veinte modelos se evaluaron después de forma centralizada sobre las 10 000 imágenes del conjunto de prueba completo. La Figura 7.2 muestra esa precisión centralizada por ronda: la convergencia es rápida en las primeras rondas y se estabiliza por encima del 90 % a partir de la ronda diez, hasta una precisión final del 91,14 %. Esta es la ejecución del experimento principal cuyas métricas se recogen en la Tabla 7.6.

Tabla 7.6: Resultados de NVIDIA FLARE en modo secuencial. La RAM corresponde al proceso principal.

Exp.	Cli.	Ron.	Acc.	Loss	Tiempo (s)	GPU (MiB)	Util. (%)	Energía (Wh)	Temp. (°C)	RAM (GiB)
e0	10	30	0,8260	0,5564	5758	466	56,90	51,76	83	2,2
e1	5	20	0,9114	0,3058	12304	466	83,47	138,51	87	2,0
e2-c4	4	10	0,8145	0,5796	1320	468	75,97	14,84	66	1,8
e2-c5	5	10	0,8167	0,5510	1404	466	71,06	15,08	66	1,8
e2-c8	8	10	0,7713	0,6605	1617	470	61,80	16,01	68	2,0
e3	8	20	0,9086	0,3277	5593	470	87,59	70,61	87	2,0

**Figura 7.2:** Convergencia de la precisión centralizada del modelo global de NVIDIA FLARE en el experimento principal, evaluando sobre el conjunto de prueba completo el modelo agregado de cada ronda.

En modo concurrente, NVIDIA FLARE tampoco proporcionó resultados utilizables. Al aumentar el número de hilos de entrenamiento del simulador, crece el número de procesos y clientes activos de forma simultánea, y en el equipo utilizado esto provocó cuelgues del sistema que obligaron a reiniciar la máquina. No se afirma que se produjera un agotamiento de memoria de GPU, ya que no se dispone de un registro que lo demuestre; lo observado es una saturación de memoria y procesos del sistema. Estos casos se documentan más adelante, en el apartado 7.5.1.5.

7.3.2 Usabilidad de NVIDIA FLARE

La usabilidad de NVIDIA FLARE se valora a continuación según las cuatro dimensiones de la rúbrica de experiencia de uso.

7.3.2.1 Utilidad funcional (*useful*)

NVIDIA FLARE cubre el caso de estudio y excede ampliamente sus necesidades. A través de una plantilla de trabajo que reproduce el patrón de dispersión y agregación equivalente a *Federated Averaging*, permitió ejecutar el mismo experimento con CIFAR-10, la ResNet-18 adaptada y varios clientes simulados, registrar métricas por ronda y realizar una evaluación centralizada final sobre el conjunto de prueba. Su orientación hacia escenarios de producción aporta funcionalidades muy por encima de lo requerido, lo que en términos de utilidad es una ventaja, pero también explica que una parte del esfuerzo inicial se dedique a discernir qué subconjunto de sus capacidades es pertinente para una comparativa académica controlada.

7.3.2.2 Facilidad de uso (*usable*)

La facilidad de uso es la dimensión más débil de NVIDIA FLARE y la que más diferencia su experiencia respecto a Flower. La instalación no fue inmediata: el primer intento sobre un entorno virtual con una versión muy reciente del intérprete falló durante la compilación de dependencias nativas, y solo se resolvió migrando a un entorno gestionado por Conda con una versión del intérprete contrastada, sobre la que se instalaron además los extras necesarios. Esta dependencia de una combinación concreta de versiones constituye ya una primera barrera.

Una vez instalado, el framework exige asimilar un número considerable de conceptos propios —trabajo, espacio de trabajo, sitio, ejecutor, flujo de trabajo, persistor y agregador, entre otros— y una estructura de directorios extensa, de modo que la distancia entre instalar la herramienta y lanzar el experimento previsto es notable. La adaptación requirió resolver una sucesión de incidencias de naturaleza diversa: dependencias ausentes para la carga de datos, un agotamiento de memoria del sistema al preparar una prueba con muchos clientes que llegó a derribar la sesión gráfica del equipo, la selección incorrecta de dispositivo al invocar rutinas de la GPU cuando el entrenamiento debía realizarse en otro dispositivo, el cierre prematuro del agente de comunicación cuando el cliente esperaba mensajes de un trabajo ya finalizado, y errores derivados del parcheo automático de la configuración que llegaban a alterar el número de rondas o de clientes efectivos respecto al experimento planificado. Cada una de estas incidencias era resoluble, pero su acumulación sitúa la curva de aprendizaje claramente por encima de la del resto de herramientas evaluadas.

7.3.2.3 Experiencia de desarrollo (*desirable*)

La experiencia de desarrollo con NVIDIA FLARE es ambivalente. En su contra juega una depuración exigente: los fallos de un cliente pueden manifestarse de forma indirecta como errores de comunicación —pérdida del par, ausencia de latido o aborto de la tarea— que ocultan la causa real y obligan a inspeccionar varios artefactos antes de localizar el origen del problema. A su favor, el framework genera un conjunto muy completo de registros y artefactos por ejecución, con resúmenes, trazas, monitorización del sistema y registros de evaluación separados, lo que proporciona una trazabilidad experimental superior a la del resto de herramientas y transmite confianza una vez superada la fase de configuración. El resultado es una experiencia que recompensa la inversión de aprendizaje con rigor y reproducibilidad, pero que penaliza con dureza al desarrollador en las primeras fases.

7.3.2.4 Accesibilidad (*accessible*)

La accesibilidad de NVIDIA FLARE se ve condicionada por la profundidad del framework. La documentación es extensa y existe abundante material de referencia, pero la barrera de entrada es alta: empezar a producir resultados requiere un conocimiento previo no trivial de su modelo de ejecución, y algunos comandos o vías esperadas no coincidían con las efectivamente disponibles en la versión instalada, lo que obliga a contrastar la documentación con la herramienta real. Los mensajes de error, especialmente los relacionados con la comunicación entre procesos, no siempre son autoexplicativos para quien se inicia. Para un perfil con experiencia en sistemas distribuidos la documentación es valiosa, pero para una primera aproximación la facilidad de acceso es limitada.

7.3.2.5 Valoración de la usabilidad de NVIDIA FLARE

NVIDIA FLARE es el framework más potente y trazable, pero también el más costoso de usar: combina una utilidad funcional sobresaliente con una facilidad de uso y una accesibilidad penalizadas por su complejidad y su curva de aprendizaje. La Tabla 7.7 resume la valoración.

Tabla 7.7: Valoración de la usabilidad de NVIDIA FLARE según las cuatro dimensiones de la rúbrica (escala 1–5).

Dimensión	Síntesis de la evidencia	Puntuación
<i>Useful</i>	Cobertura completa y capacidades muy por encima de lo requerido	5
<i>Usable</i>	Instalación sensible a versiones; muchos conceptos; depuración costosa	2
<i>Desirable</i>	Trazabilidad excelente, pero depuración indirecta y exigente	3
<i>Accessible</i>	Documentación amplia con barrera de entrada alta; errores opacos	3
Media		3,25

7.3.3 Flexibilidad de NVIDIA FLARE

NVIDIA FLARE se presenta como un SDK de aprendizaje federado agnóstico al dominio y compatible con flujos de trabajo de aprendizaje automático ya existentes, con una orientación

que abarca desde la simulación local hasta el despliegue en producción y en el borde (NVIDIA, 2026l; Roth et al., 2022). La evaluación experimental de su flexibilidad se apoya en un paquete de cinco ejecuciones de la variante reducida del experimento e3, la misma `e3_flex_mini` empleada con Flower (apartado 7.2.3.3), con ocho clientes totales, cuatro por ronda, tres rondas, una época local, tamaño de lote 32, tasa de aprendizaje 0,02 y semilla 42, en modo secuencial con un único hilo del simulador. Conviene precisar el alcance: el paquete inspeccionado contiene una selección de suites concretas y no la totalidad del histórico de ejecuciones, por lo que las conclusiones experimentales se refieren únicamente a los archivos incluidos en él.

7.3.3.1 Diseño modular: jobs, componentes y workflows

La flexibilidad de NVIDIA FLARE descansa en su modelo de configuración modular. La unidad de ejecución es el *job*, que describe por completo un experimento mediante ficheros de configuración de servidor y de cliente (`config_fed_server.conf` y `config_fed_client.conf`), en los que cada elemento del sistema se declara como un componente con su ruta de clase y sus argumentos de construcción (NVIDIA, 2026h). Al desplegar el job, el servidor y los clientes analizan esos ficheros e instancian dinámicamente los componentes declarados, registrándolos en el bucle de eventos del framework; cada componente se identifica con un `component_id` que permite referenciarlo desde otros componentes o workflows (NVIDIA, 2026h). Este diseño es el que permite sustituir un agregador, un persistor, un *executor*, un generador de objetos compartibles (*Shareable*) o un filtro modificando la configuración, sin reescribir la aplicación completa.

Sobre esa base de bajo nivel, la documentación ofrece además una capa de *recipes* que empaqueta algoritmos y workflows habituales —FedAvg, FedProx, FedOpt, SCAFFOLD, aprendizaje cíclico, XGBoost, Sklearn, estadísticas federadas, evaluación cruzada, PSI, integración con Flower y swarm learning, entre otros (NVIDIA, 2026a). Esta dualidad, configuración granular por componentes y configuración de alto nivel por recetas, sustenta una valoración elevada en personalización y reproducibilidad, y constituye el punto de partida de las adaptaciones realizadas para probar estrategias alternativas.

7.3.3.2 Adaptaciones para soportar las estrategias

El objetivo de la prueba fue comprobar si un mismo job podía adaptarse para ejecutar FedAvg, FedProx y variantes FedOpt del servidor (FedAdagrad y FedAdam), manteniendo constante la estructura general del experimento. Para ello se introdujeron tres cambios de naturaleza distinta. En primer lugar, la selección de estrategia se trasladó al cliente, que admite los valores `fedavg`, `fedprox`, `fedadam` y `fedadagrad` mediante un argumento de línea de órdenes que, por defecto, toma su valor de la variable de entorno `TFG_FL_STRATEGY`. En segundo lugar, FedProx se implementó en el entrenamiento local: cuando la estrategia es `fedprox` y el coeficiente proximal es positivo, el entrenamiento añade un término que penaliza la desviación de los pesos locales respecto al modelo global recibido al inicio de la tarea, manteniendo el descenso de gradiente estocástico como optimizador local (Li, Sahu, Zaheer et al., 2020; NVIDIA, 2026f).

El tercer cambio afecta al servidor y es el más relevante para la flexibilidad de agregación. Para las variantes FedOpt, el script de ejecución parchea el fichero de configuración del

servidor sustituyendo el generador de compartibles por defecto, `FullModelShareableGenerator`, por `PTFedOptModelShareableGenerator`, y añade un componente de modelo fuente requerido por este generador, como ilustra el Código 7.4. Este generador actualiza el modelo global mediante un optimizador de PyTorch —y, opcionalmente, un planificador de tasa de aprendizaje— cuyos argumentos incluyen `optimizer_args`, `lr_scheduler_args`, `source_model` y `device` (NVIDIA, 2026i). En consecuencia, FedAdam y FedAdagrad no son recetas independientes, sino configuraciones de FedOpt que se obtienen seleccionando `torch.optim.Adam` o `torch.optim.Adagrad` como optimizador del servidor.

Fragmento representativo de `config_fed_server.conf` (FedOpt)

```

1 "shareable_generator": {
2   "path": "nvflare.app_opt.pt.fedopt.PTFedOptModelShareableGenerator",
3   "args": {
4     "source_model": "model",
5     "optimizer_args": { "path": "torch.optim.Adagrad" }
6   }
7 },
8 "components": [
9   { "id": "model", "path": "<ruta de la ResNet-18 adaptada>" }
10 ]

```

Una precaución metodológica relevante surgió con el dispositivo de ejecución. En todos los ficheros de configuración del cliente aparece `--device cuda`, pero en la ejecución de FedAdam en CPU el propio cliente seleccionó CPU porque `torch.cuda.is_available()` devolvió falso al lanzarse con `CUDA_VISIBLE_DEVICES` vacío. Por ello, el dispositivo realmente empleado debe leerse del registro de ejecución y no del fichero de configuración, criterio que se ha seguido al elaborar la Tabla 7.8.

7.3.3.3 Mini-experimento de flexibilidad

La Tabla 7.8 resume las cinco ejecuciones. FedAvg sirvió de referencia y se ejecutó correctamente con el workflow *Scatter and Gather*, en el que el servidor distribuye los pesos iniciales, los clientes entrenan localmente y devuelven sus pesos, y el sistema los agrega por promedio ponderado (NVIDIA, 2026f). A partir de esa base, FedProx finalizó las tres rondas y los registros confirmaron `strategy=fedprox` con un coeficiente proximal `fedprox_mu=0,001` activo durante el entrenamiento. FedAdagrad también completó las tres rondas como FedOpt real —y no como mera etiqueta— porque la configuración del servidor incorporaba el generador `PTFedOptModelShareableGenerator` y el registro mostraba las actualizaciones del servidor con `torch.optim.Adagrad`.

El caso de FedAdam exige una lectura cuidadosa. La estrategia llegó a configurarse realmente mediante FedOpt con `torch.optim.Adam`, pero su ejecución en GPU abortó durante el paso de actualización del servidor (`torch.optim.Adam.step()`) con un error de inicialización de CUDA, registrado como `FATAL_SYSTEM_ERROR`. Para aislar el problema, la prueba se repitió en CPU, donde FedAdam completó las tres rondas; esta ejecución valida FedAdam como estrategia configurable, pero su duración (1280 s) es muy superior precisamente por entrenarse en CPU, de modo que no debe mezclarse con la comparativa de rendimiento en

Tabla 7.8: Resultados del mini-experimento de flexibilidad de NVIDIA FLARE. Las cifras de aprendizaje son auxiliares (tres rondas, una época, una semilla) y no constituyen una comparación de rendimiento. El dispositivo real procede del registro de ejecución, no del fichero de configuración del cliente.

Estrategia	Estado	Disp.	FedOpt	Rondas	Dur. (s)	Acc.	Loss
FedAvg	success	cuda:0	No	3	205	0,4178	1,5601
FedProx	success	cuda:0	No	3	225	0,4045	1,5876
FedAdagrad	success	cuda:0	Sí	3	199	0,1402	2,3785
FedAdam	failed	cuda:0	Sí	0	77	—	—
FedAdam	success	cpu	Sí	3	1280	0,2586	3,4439

GPU (Subsección 7.5.1). En coherencia con ello, la fila de la ejecución fallida no reporta precisión ni pérdida, ya que no completó ninguna ronda.

Estos resultados no permiten ordenar las estrategias por calidad, por los mismos motivos señalados para Flower: tres rondas, una época local y una sola semilla son insuficientes para que las variantes adaptativas converjan. Su valor es demostrar que NVIDIA FLARE permite adaptar tanto el entrenamiento local como la lógica de actualización del servidor mediante configuración y componentes. Importa además una cautela de interpretación: la validez de FedAdagrad y FedAdam no se sustenta en el nombre de la ejecución, sino en la presencia del generador FedOpt y del optimizador del servidor en los registros. Del mismo modo, el valor `fedprox_mu=0,001` que aparece en la configuración no implica que FedProx se aplique a todas las estrategias: en FedAdagrad y FedAdam el entrenamiento imprime `fedprox_mu=0,0`, porque el término proximal solo se activa cuando la estrategia es FedProx.

De esta experiencia se desprende un contraste claro entre ambos frameworks. Donde Flower sustituía la estrategia cambiando una única clase en el servidor, NVIDIA FLARE exigió modificar componentes del servidor, añadir un modelo fuente para FedOpt y comprobar en los registros que la estrategia configurada era realmente la que se ejecutaba. La flexibilidad de NVIDIA FLARE es, por tanto, amplia, pero su ejercicio práctico conlleva una complejidad de configuración mayor.

7.3.3.4 Capacidades avanzadas según la documentación

Como en el caso de Flower, varias capacidades relevantes no se ejecutaron en los experimentos y se valoran únicamente como evidencia documental. La documentación oficial las articula en torno a las *Job Recipes*, una API de alto nivel que encapsula un trabajo federado exponiendo solo los parámetros clave —`min_clients`, `num_rounds`, modelo y *script* de entrenamiento— y ocultando la complejidad de controladores y *workflows* (NVIDIA, 2026a). Estas recetas se ejecutan sobre tres entornos intercambiables: **SimEnv** simula los clientes en procesos locales, **PocEnv** los lanza como procesos separados en la misma máquina y **ProdEnv** despliega el trabajo en una instalación de producción; un mismo trabajo puede así trasladarse de la simulación al despliegue sin cambiar el código (NVIDIA, 2026a). El catálogo de recetas cubre FedAvg, FedProx, FedOpt —donde el optimizador del servidor se fija con un argumento que indica su ruta de clase e hiperparámetros, por ejemplo `torch.optim.SGD` con tasa y momento—, SCAFFOLD y el aprendizaje cíclico, además de XGBoost federado en sus

variantes horizontal, *bagging* y vertical (NVIDIA, 2026f, 2026i).

En cuanto al aprendizaje asíncrono, la receta **EdgeFedBuffRecipe**, orientada a dispositivos de borde, admite entrenamiento síncrono y asíncrono: en modo síncrono el servidor espera todas las actualizaciones antes de generar un nuevo modelo, mientras que en modo asíncrono lo genera en cuanto recibe la primera, lo que reduce el tiempo total cuando los dispositivos presentan latencias heterogéneas (NVIDIA, 2026j, 2026l). La receta expone parámetros como el número máximo de versiones de modelo activas o el tamaño de selección de dispositivos por ronda. No obstante, la propia documentación advierte de que la asincronía general aún no está disponible para todas las recetas y se concentra en el escenario de borde, por lo que su valoración se mantiene como soporte documental parcial.

En materia de seguridad y privacidad, la documentación es notablemente extensa. NVIDIA FLARE implementa la privacidad mediante un mecanismo general de filtros que procesan los objetos compartibles antes o después de la comunicación, configurables en la receta como **task_data_filters** o **task_result_filters**. Soporta privacidad diferencial a nivel de muestra mediante la integración de DP-SGD con Opacus —controlada por los parámetros **epsilon** y **delta**— y filtros de DP local como **PercentilePrivacy**, que comparte solo las diferencias de pesos por encima de un percentil, **SVTPPrivacy**, basado en la técnica del vector disperso con ruido de Laplace, y **StatisticsPrivacyFilter** para estadísticas federadas (NVIDIA, 2026d). A un nivel superior, la solución de computación confidencial ejecuta la agregación dentro de un entorno de ejecución confiable (*Trusted Execution Environment*) sobre hardware como AMD SEV-SNP o Intel TDX, protege el modelo y el código del cliente en máquinas virtuales confidenciales y permite combinar privacidad diferencial con cifrado homomórfico o agregación segura (NVIDIA, 2026g, 2026k).

Por último, NVIDIA FLARE documenta esquemas descentralizados mediante los *workflows* controlados por clientes, contruidos sobre las clases base **ServerSideController** y **ClientSideController** para escenarios donde el servidor no es plenamente confiable (NVIDIA, 2026b). Entre ellos destacan el aprendizaje cíclico, en el que cada cliente entrena con el modelo recibido del anterior según un orden fijo o aleatorio (NVIDIA, 2026c); el aprendizaje *swarm*, que designa un cliente agregador distinto por ronda y mejora la resiliencia frente a fallos del servidor (NVIDIA, 2026b); y la evaluación cruzada entre sitios, que permite a los clientes validar modelos de otros mediante comunicación entre pares sin exponerlos al servidor. A esto se suman el aprendizaje vertical con XGBoost y las estadísticas federadas, así como las utilidades para integrar el seguimiento de experimentos con TensorBoard, MLflow o Weights & Biases sobre cualquier receta (NVIDIA, 2026a, 2026e). La documentación reconoce, además, dos cautelas: la API de recetas se encuentra en estado de *technical preview* y no cubre todavía todos los algoritmos, y la configuración de la computación confidencial exige infraestructura especializada.

7.3.3.5 Limitaciones observadas

La limitación más concreta de la evaluación fue el fallo de FedAdam en GPU por un error de inicialización de CUDA durante la actualización FedOpt del servidor. Debe interpretarse como una incidencia técnica del entorno, no como una carencia del framework, puesto que la misma configuración completó las tres rondas en CPU; aun así, impide presentar FedAdam como ejecución correcta en GPU. La segunda limitación es la complejidad práctica ya señalada: a diferencia de Flower, alterar la lógica de agregación obligó a intervenir en componentes

del servidor y a validar cuidadosamente los registros, lo que incrementa la probabilidad de errores de configuración. Por último, el modo concurrente quedó condicionado por los recursos del equipo, en línea con lo expuesto en el análisis del comportamiento concurrente (apartado 7.5.1.5); aunque la documentación describe un sistema de ejecución multi-job con gestión de recursos, esa capacidad no pudo validarse de forma estable en el hardware empleado.

7.3.3.6 Valoración de la flexibilidad de NVIDIA FLARE

La Tabla 7.9 recoge la valoración sobre los criterios F1–F10, con el mismo convenio aplicado a Flower: puntuación máxima cuando existe evidencia experimental directa, puntuación parcial cuando la capacidad solo consta en la documentación y puntuación nula cuando no se identifica soporte. NVIDIA FLARE obtiene la máxima puntuación en la personalización del entrenamiento local (F2) y en la estrategia de agregación (F3) a partir de evidencia experimental directa, y en la personalización del modelo y los datos (F1), los algoritmos federados disponibles (F4), los modos de ejecución y despliegue (F6), la integración con herramientas externas (F7) y la reproducibilidad (F10) combinando evidencia experimental y documental. El aprendizaje asíncrono (F5) recibe puntuación parcial, al igual que en Flower: NVIDIA FLARE lo documenta de forma explícita mediante FedBuff, orientado al escenario de borde, con un nivel de soporte equivalente al de la *Driver API* y *Flower Next* de Flower, en ambos casos respaldado solo por la documentación y no validado en los experimentos; la diferencia de mecanismo no altera la puntuación. La seguridad y privacidad (F8) y la modificación de la arquitectura federada (F9) reciben también puntuación parcial por estar respaldadas solo documentalmente.

Tabla 7.9: Valoración de la flexibilidad de NVIDIA FLARE según los criterios F1–F10. El tipo de evidencia distingue las capacidades ejecutadas en los experimentos de las descritas en la documentación oficial.

Criterio	Descripción	Punt.	Evidencia
F1	Personalización del modelo y el dataset	2/2	Exp. + documental
F2	Personalización del entrenamiento local	2/2	Experimental
F3	Estrategias de agregación personalizadas	2/2	Experimental
F4	Algoritmos federados disponibles	2/2	Exp. + documental
F5	Aprendizaje federado asíncrono	1/2	Documental
F6	Modos de ejecución y despliegue	2/2	Exp. + documental
F7	Integración con herramientas externas	2/2	Exp. + documental
F8	Seguridad, privacidad y extensiones	1/2	Documental
F9	Modificación de la arquitectura federada	1/2	Documental
F10	Reproducibilidad y configuración	2/2	Exp. + documental
Total		17/20	

NVIDIA FLARE alcanza también 17/20. La evidencia experimental demuestra que permite adaptar el entrenamiento local —incluido el término proximal de FedProx— y, sobre todo, la lógica de agregación del servidor mediante FedOpt, con FedAvg, FedProx y FedAdagrad ejecutados correctamente y FedAdam validado en CPU. A ello se suma una de las superficies

de capacidades documentadas más amplias del estudio, especialmente sólida en seguridad y privacidad, que abarca aprendizaje asíncrono, workflows cíclicos y *swarm*, privacidad diferencial, cifrado homomórfico y computación confidencial. El contrapunto es una complejidad de configuración superior a la de Flower y el hecho de que esas capacidades avanzadas solo se verificaron documentalmente; además, la ejecución de FedAdam en GPU falló por un error de CUDA y el modo concurrente quedó limitado por el hardware. Esta lectura se contrasta con la de los otros frameworks en la comparativa de flexibilidad (Subsección 7.5.3).

7.4 FedML

Esta sección presenta los resultados de FedML en los tres ejes de evaluación, manteniendo el mismo orden que en los frameworks anteriores: rendimiento, usabilidad y flexibilidad.

7.4.1 Rendimiento de FedML

La Tabla 7.10 recoge los resultados secuenciales de FedML tras repetir las configuraciones afectadas por los problemas iniciales de instrumentación. Las ejecuciones finales de e0, e1, e2-c4, e2-c5, e2-c8 y e3 disponen de métricas de aprendizaje, tiempo, GPU, energía estimada y memoria residente. FedML reproduce la tendencia de escalabilidad de los otros dos *frameworks*, con una caída de precisión más acusada al aumentar el número de clientes: con ocho clientes alcanza un 71,15 %, valor inferior al de Flower y NVIDIA FLARE en la misma configuración.

Tabla 7.10: Resultados de FedML en modo secuencial. La RAM corresponde al proceso principal.

Exp.	Cli.	Ron.	Acc.	Loss	Tiempo (s)	GPU/Util./Energía	RAM (GiB)
e0	10	30	0,8320	0,4981	3447	414 MiB / 95,23 % / 48,10 Wh	2,60
e1	5	20	0,9071	0,4034	11304	416 MiB / 86,29 % / 140,40 Wh	2,29
e2-c4	4	10	0,7893	0,6054	1419	416 MiB / 86,33 % / 13,12 Wh	2,19
e2-c5	5	10	0,7697	0,6668	1243	416 MiB / 86,62 % / 15,80 Wh	2,28
e2-c8	8	10	0,7115	0,8108	1267	416 MiB / 85,15 % / 16,87 Wh	2,47
e3	8	20	0,8895	0,4321	5857	416 MiB / 87,09 % / 70,97 Wh	2,23

FedML registra la precisión de prueba centralizada del modelo global en cada ronda de comunicación. La Figura 7.3 muestra esa curva para el experimento principal (e1), la misma configuración empleada en las curvas de Flower y NVIDIA FLARE. Con ello, las tres curvas de convergencia (Figuras 7.1, 7.2 y 7.3) corresponden ya al mismo experimento (e1) y representan la misma magnitud —la precisión del modelo global agregado evaluado de forma centralizada sobre el conjunto de prueba completo al final de cada ronda—, por lo que son directamente comparables entre sí. En las tres se reproduce el mismo patrón: un ascenso muy rápido en las primeras rondas y una estabilización posterior por encima del 90 %, con diferencias finales reducidas entre los tres *frameworks*.

A diferencia de los otros dos *frameworks*, el modo concurrente de FedML sí se ejecutó. El backend MPI, lanzado con `mpirun`, mantiene varios procesos de cliente en paralelo y completó las ejecuciones registrando métricas de tiempo y de consumo de recursos (Tabla 7.11). La matriz concurrente reúne las configuraciones e1, e2 y e3; el experimento de referencia (e0) no

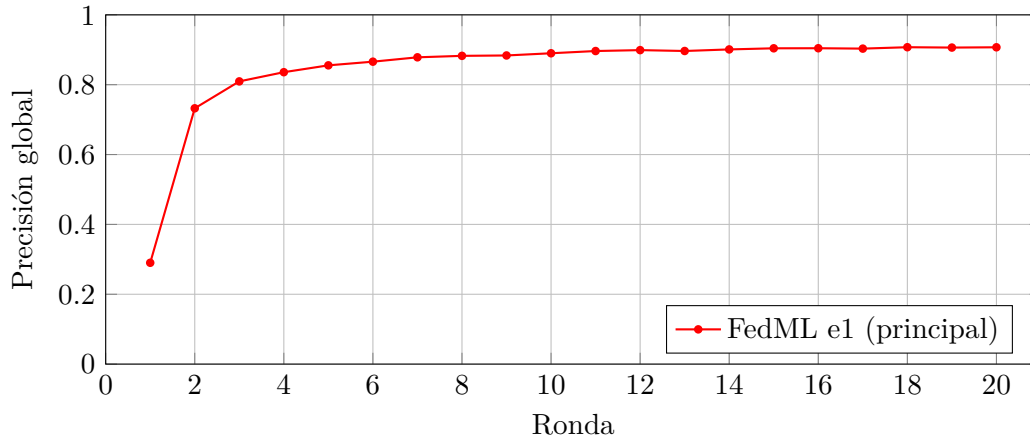


Figura 7.3: Convergencia de la precisión centralizada del modelo global de FedML en el experimento principal, evaluada sobre el conjunto de prueba completo al final de cada ronda.

figura entre ellas, al no disponerse de una corrida concurrente con métricas completas. Sin embargo, estas ejecuciones no guardaron una precisión final extraíble en los *logs* analizados, por lo que se utilizan únicamente para describir el comportamiento computacional del modo concurrente, no para comparar la calidad del modelo. Se observa una utilización de GPU muy alta (cercana al 100 %) y un uso de memoria de GPU sensiblemente mayor que en secuencial, que crece con el número de clientes.

Tabla 7.11: Resultados de FedML en modo concurrente (backend MPI). No se dispone de precisión final extraída; solo se reportan tiempo y consumo de recursos.

Exp.	Cli.	Ron.	Tiempo (s)	GPU (MiB)	Util. (%)	Pot. media (W)	Temp. (°C)	RAM (GiB)
e1	5	20	11380	2358	99,80	43,15	80	2,0
e2-c4	4	10	1102	1907	98,91	46,01	80	2,1
e2-c5	5	10	1109	2358	98,81	49,60	87	2,0
e2-c8	8	10	1114	3727	97,84	50,61	85	1,8
e3	8	20	5457	1915	99,61	49,45	83	2,1

7.4.2 Usabilidad de FedML

La usabilidad de FedML se valora a continuación según las cuatro dimensiones de la rúbrica de experiencia de uso.

7.4.2.1 Utilidad funcional (*useful*)

FedML cubre el caso de estudio y aporta una característica útil desde el punto de vista del uso: la disponibilidad de distintos backends de ejecución, uno para el modo secuencial y otro basado en paso de mensajes para el modo concurrente, bajo una misma configuración. Permite ejecutar *Federated Averaging* sobre CIFAR-10 con la ResNet-18 adaptada y los mismos hiperparámetros, y facilita repetir una misma configuración con distintas semillas, si bien las

métricas reportadas en este trabajo corresponden a una única semilla, en coherencia con el control de variables descrito en la Sección 5.5. La principal limitación funcional en términos de uso es que el modo concurrente no produjo una métrica de precisión final aprovechable en sus registros, lo que restringe su utilidad a la medición de tiempo y recursos y obliga a tenerlo en cuenta al planificar qué resultados puede sustentar cada modo.

7.4.2.2 Facilidad de uso (*usable*)

La facilidad de uso de FedML se sitúa en un punto intermedio. La instalación y la puesta en marcha no presentaron fricciones reseñables, y la configuración se gestiona mediante ficheros declarativos que centralizan los parámetros del experimento. La fricción se concentró en dos aspectos. El primero es la fiabilidad de la instrumentación: en varias ejecuciones la inicialización de la biblioteca de monitorización de la GPU falló por un desajuste entre la versión del controlador y la de la biblioteca, de modo que, aunque las métricas de aprendizaje seguían siendo válidas, las métricas de GPU de esas corridas no eran utilizables y debían marcarse como no disponibles en lugar de mezclarse con ceros. El segundo es la extracción de resultados: en el modo concurrente, los registros no siempre contenían la información necesaria de forma directa, y algunas ejecuciones terminaron por agotamiento de tiempo o sin recoger por completo los resultados, lo que exigió un trabajo de filtrado más cuidadoso que en Flower.

7.4.2.3 Experiencia de desarrollo (*desirable*)

Como entorno de desarrollo, FedML resulta funcional y razonablemente cómodo. El mecanismo de extensión por herencia, que separa el entrenador del cliente y el agregador del servidor, ofrece puntos de intervención claros y una sensación de orden al adaptar el experimento. La experiencia se ve algo mermada por la menor pulcritud de los registros en determinados modos y por la necesidad de validar con cuidado qué corridas son aprovechables, lo que introduce una capa de verificación adicional sobre la salida del framework. El modo concurrente, basado en paso de mensajes, se mostró en algunas pruebas más estable que el de Flower, lo que mejora la percepción de robustez en ese escenario.

7.4.2.4 Accesibilidad (*accessible*)

La accesibilidad de FedML es intermedia. Dispone de documentación y de ejemplos que permiten arrancar, pero parte de la configuración y, sobre todo, la interpretación y extracción de las métricas requieren un esfuerzo de lectura mayor que en Flower, y algunos comportamientos solo se comprenden tras inspeccionar los registros con detenimiento. No impone barreras de entrada tan altas como NVIDIA FLARE, pero tampoco alcanza la inmediatez de Flower para pasar de la instalación a un resultado limpio y bien documentado.

7.4.2.5 Valoración de la usabilidad de FedML

FedML ofrece una usabilidad correcta y equilibrada, sin la fricción de instalación de NVIDIA FLARE pero sin la inmediatez de Flower; sus puntos débiles son la fiabilidad de la instrumentación de GPU y la claridad de los registros en el modo concurrente. La Tabla 7.12 resume la valoración.

Tabla 7.12: Valoración de la usabilidad de FedML según las cuatro dimensiones de la rúbrica (escala 1–5).

Dimensión	Síntesis de la evidencia	Puntuación
<i>Useful</i>	Cobertura completa; doble backend; concurrente sin precisión final	4
<i>Usable</i>	Instalación sin fricción; instrumentación de GPU y logs irregulares	3
<i>Desirable</i>	Extensión por herencia clara; verificación extra de resultados	3
<i>Accessible</i>	Documentación suficiente; extracción de métricas exigente	3
Media		3,25

7.4.3 Flexibilidad de FedML

FedML es una biblioteca de investigación y banco de pruebas para aprendizaje federado (C. He et al., 2020), documentada actualmente dentro de la plataforma TensorOpera aunque la librería y los ejemplos siguen empleando el paquete `fedml`. Su documentación organiza el framework en varios escenarios —simulación, cross-silo y cross-device— y ofrece un catálogo amplio de algoritmos de referencia. La evaluación de su flexibilidad combina la implementación propia realizada para la comparativa, que extiende las abstracciones de FedML, con una prueba reducida de sustitución de estrategias. Un apunte sobre el alcance: por el coste computacional, la prueba reducida de FedML se ejecutó con cuatro clientes totales, dos por ronda y dos rondas, una configuración algo menor que la empleada con Flower y NVIDIA FLARE; las métricas de aprendizaje son, por tanto, una comprobación funcional y no una comparación de rendimiento.

7.4.3.1 Configuración y componentes personalizados

La configuración de FedML se apoya en un fichero `fedml_config.yaml` estructurado en secciones (`common_args`, `data_args`, `model_args`, `train_args`, `device_args`, `comm_args` y `tracking_args`), donde se declaran los parámetros federados como `federated_optimizer`, `client_num_in_total`, `client_num_per_round`, `comm_round`, `epochs`, `batch_size`, `client_optimizer` y `learning_rate` (TensorOpera, 2026e). Esta organización permitió variar clientes, rondas, épocas e hiperparámetros entre ejecuciones sin reescribir la aplicación.

Más allá del fichero de configuración, la flexibilidad de FedML en este trabajo se sustenta en evidencia experimental directa: la implementación de la comparativa define un entrenador y un agregador propios que extienden las abstracciones `ClientTrainer` y `ServerAggregator` del framework. El entrenador `FedMLCifar10Trainer` encapsula el bucle de entrenamiento local sobre la ResNet-18 adaptada —descenso de gradiente estocástico, entropía cruzada y métricas propias—, mientras que el agregador `FedMLCifar10Aggregator` gobierna la evaluación del modelo global; ambos se conectan al flujo mediante `FedMLRunner`. La documentación respalda esta vía, al indicar que el usuario puede definir modelo, datos, optimizadores, funciones de pérdida y lógica de entrenamiento arbitrarios, así como dataloaders propios que respeten la estructura esperada por el simulador (TensorOpera, 2026b, 2026c). Esta capacidad de sustituir componentes centrales mediante herencia sostiene una valoración alta en personalización del entrenamiento local, de la agregación y de la integración.

7.4.3.2 Sustitución de la estrategia federada

La prueba de flexibilidad consistió en ejecutar cuatro estrategias modificando únicamente la configuración, manteniendo constantes el conjunto de datos, el modelo y el resto de hiperparámetros. FedAvg y FedProx se seleccionan asignando `federated_optimizer` a "FedAvg" o "FedProx", mientras que FedAdam y FedAdagrad se obtienen mediante `federated_optimizer: "FedOpt"` combinado con un optimizador de servidor Adam o Adagrad, tal como ilustra el Código 7.5. Durante las primeras pruebas, las variantes FedOpt fallaban porque FedML esperaba el atributo `ci` en los argumentos de configuración; tras añadir `ci: 0` a la configuración común, ambas pudieron completar la ejecución. La selección del algoritmo desde la configuración está respaldada por la documentación, que controla el optimizador federado a través del mismo campo (TensorOpera, 2026e).

Fragmento de `fedml_config.yaml` (FedAdam vía FedOpt)

```
1 train_args:
2   federated_optimizer: "FedOpt"
3   server_optimizer: "Adam"
4   ci: 0
5   client_num_in_total: 4
6   client_num_per_round: 2
7   comm_round: 2
8   epochs: 1
9   batch_size: 32
10  learning_rate: 0.02
```

7.4.3.3 Mini-experimento de flexibilidad

La Tabla 7.13 resume las cuatro ejecuciones, todas finalizadas con estado `success`. FedAvg y FedProx concluyeron con métricas coherentes para una prueba de dos rondas, con precisiones del 43,01 % y el 42,60 % respectivamente, lo que confirma el soporte experimental de una estrategia alternativa a FedAvg con comportamiento correcto. FedAdam y FedAdagrad también completaron la ejecución, lo que demuestra que fue posible activar FedOpt con optimizadores de servidor Adam y Adagrad; sin embargo, sus métricas finales fueron claramente deficientes, con una precisión próxima al 10 % —equivalente al azar en un problema de diez clases— y pérdidas muy elevadas.

Tabla 7.13: Resultados del mini-experimento de flexibilidad de FedML. Las cifras de aprendizaje son auxiliares (dos rondas, una época, una semilla) y constituyen una comprobación funcional, no una comparación de rendimiento entre estrategias.

Estrategia	Estado	Dur. (s)	Acc.	Loss	GPU (MiB)	Util. (%)
FedAvg	success	118	0,4301	1,5248	416	75,29
FedProx	success	123	0,4260	1,5137	484	79,01
FedAdam	success	117	0,0998	809,7697	416	76,09
FedAdagrad	success	116	0,0998	191,2948	416	76,85

La lectura correcta de estos datos es que FedML permite cambiar la estrategia federada mediante configuración, pero que la mera disponibilidad de una estrategia no garantiza su estabilidad ni un buen rendimiento. Las variantes FedOpt ejecutaron de principio a fin, de modo que la flexibilidad queda demostrada, pero su falta de aprendizaje en esta prueba indica que requieren un ajuste específico de hiperparámetros —como la tasa de aprendizaje del servidor o un mayor número de rondas— para resultar competitivas. Este patrón es coherente con lo observado en los otros frameworks: en Flower las estrategias adaptativas divergían con los valores por defecto y exigieron ajustar **eta** y **tau**, y en NVIDIA FLARE FedAdam no llegó a completarse en GPU. La fragilidad de las variantes FedOpt con poca configuración es, por tanto, un comportamiento transversal y no una carencia particular de FedML.

7.4.3.4 Capacidades avanzadas según la documentación

La documentación de FedML respalda una superficie de capacidades más amplia que la probada experimentalmente, organizada en varias capas. En el plano de uso, ofrece una interfaz de línea de comandos —integrada en la plataforma TensorOpera— con órdenes para autenticación y gestión de dispositivos (`fedml login`, `fedml device bind`), lanzamiento y gestión de trabajos y clústeres (`fedml launch`, `fedml cluster`, `fedml run`), y gestión y despliegue de modelos como servicios de inferencia (`fedml model`) (TensorOpera, 2026g). En el plano de programación, expone una API de Python con una llamada de una línea (`fedml.run_simulation()`) y una API más explícita de varias líneas que inicializa dispositivo, datos y modelo y ejecuta la federación mediante `SimulatorMPI` o las clases `Client/Server` envueltas por `FedMLRunner` (TensorOpera, 2026e).

En cuanto al catálogo algorítmico, el simulador Parrot enumera implementaciones de referencia como FedOpt, FedNova, FedGKT, aprendizaje federado descentralizado, vertical y jerárquico, FedNAS y *Split Learning*, seleccionables cambiando el campo `federated_optimizer` (TensorOpera, 2026e, 2026f), y el módulo de seguridad declara compatibilidad con optimizadores como FedAvg, FedSGD, FedOpt, FedProx, FedGKT y FedNova (TensorOpera, 2026h). A este catálogo se añaden la analítica federada, que calcula estadísticas distribuidas como promedios o percentiles mediante un ejecutor específico sin compartir datos crudos, y un proyecto independiente orientado al ajuste federado de grandes modelos de lenguaje, que reutiliza las mismas API y la plataforma de operaciones (TensorOpera, 2026g). En cuanto al aprendizaje asíncrono, FedML Octopus documenta un escenario jerárquico heterogéneo en el que cada silo puede ejecutar entrenamiento distribuido con varias GPU y orquestarse mediante optimización federada asíncrona o síncrona (TensorOpera, 2026a); se trata, no obstante, de un soporte menos específico que el de NVIDIA FLARE, ya que no se documenta una receta directa equivalente a FedBuff, aprendizaje cíclico o *swarm*.

En privacidad y seguridad, el perfil de FedML difiere del de NVIDIA FLARE. Su fortaleza documental está en la simulación de ataques y defensas mediante `FedMLAttacker` y `FedMLDefender`, con ataques de tipo bizantino, *label flipping* y reemplazo de modelo, y defensas como recorte de norma, Krum, mediana geométrica o FoolsGold, activables desde el fichero de configuración con `enable_attack` y `enable_defense` (TensorOpera, 2026h), además de un ejemplo de agregación segura basada en computación multiparte (TensorOpera, 2026g). En cambio, en las páginas revisadas no figura una integración de privacidad diferencial o cifrado homomórfico comparable a la de NVIDIA FLARE, por lo que su valo-

ración en este criterio se sostiene en el *benchmarking* de robustez más que en la privacidad criptográfica. Por último, para integración y trazabilidad, FedML documenta el registro de métricas, artefactos y modelos mediante `fedml.log()`, la captura automática de métricas de hardware y GPU, los backends de comunicación MPI, gRPC, PyTorch RPC y MQTT+S3, y la integración con la plataforma de operaciones de TensorOpera para gestionar agentes, experimentos y despliegues (TensorOpera, 2026d, 2026g).

7.4.3.5 Limitaciones observadas

La limitación más visible es que las variantes FedOpt finalizaron sin aprender bajo la configuración empleada, lo que confirma que su soporte es real pero condicionado a un ajuste de hiperparámetros que queda fuera del alcance de esta prueba de flexibilidad. A ello se añade el detalle de configuración del atributo `ci`, que tuvo que fijarse manualmente para que FedOpt funcionara, y el hecho de que la prueba reducida de FedML emplea una configuración menor que la de los otros dos frameworks, por lo que sus métricas auxiliares no son directamente comparables. En cuanto a la ejecución, el modo concurrente de FedML mediante el backend MPI fue el único que llegó a completarse entre los tres frameworks, según se documenta en el análisis del comportamiento concurrente (apartado 7.5.1.5), si bien se mostró sensible al consumo de memoria en las configuraciones más grandes.

7.4.3.6 Valoración de la flexibilidad de FedML

La Tabla 7.14 recoge la valoración de FedML sobre los criterios F1–F10, con el convenio ya aplicado a los otros frameworks. FedML obtiene la máxima puntuación en personalización del modelo y los datos (F1), del entrenamiento local (F2) y de la agregación (F3), respaldadas por la implementación de entrenador y agregador propios; en los algoritmos federados disponibles (F4), por haber ejecutado cuatro estrategias mediante configuración; en los modos de ejecución (F6), por ser el único framework cuyo modo concurrente se completó; y en integración (F7) y reproducibilidad (F10). El aprendizaje asíncrono (F5), la seguridad y privacidad (F8) y la modificación de la arquitectura federada (F9) reciben puntuación parcial por estar respaldados únicamente por la documentación.

FedML cierra la terna con la misma valoración de flexibilidad, 17/20. Su mayor fortaleza experimental reside en la facilidad para sustituir componentes centrales del sistema —entrenador y agregador— mediante herencia, y en haber ejecutado el abanico más amplio de estrategias del estudio a través de la configuración. Su perfil documental es asimismo notable, con un catálogo extenso de algoritmos y escenarios y un soporte de seguridad orientado al benchmarking de ataques y defensas. Las cautelas son dos: las variantes FedOpt completaron la ejecución pero no aprendieron sin un ajuste adicional, y algunas capacidades avanzadas —asincronía, privacidad criptográfica y esquemas alternativos— solo se verificaron documentalmente.

7.5 Comparativa entre frameworks

Tras presentar cada framework por separado, esta sección los contrasta de forma transversal en los tres ejes de evaluación: primero el rendimiento cuantitativo, después la usabilidad y, por último, la flexibilidad.

Tabla 7.14: Valoración de la flexibilidad de FedML según los criterios F1–F10. El tipo de evidencia distingue las capacidades ejecutadas en los experimentos de las descritas en la documentación oficial.

Criterio	Descripción	Punt.	Evidencia
F1	Personalización del modelo y el dataset	2/2	Exp. + documental
F2	Personalización del entrenamiento local	2/2	Exp. + documental
F3	Estrategias de agregación personalizadas	2/2	Experimental
F4	Algoritmos federados disponibles	2/2	Exp. + documental
F5	Aprendizaje federado asíncrono	1/2	Documental
F6	Modos de ejecución y despliegue	2/2	Exp. + documental
F7	Integración con herramientas externas	2/2	Exp. + documental
F8	Seguridad, privacidad y extensiones	1/2	Documental
F9	Modificación de la arquitectura federada	1/2	Documental
F10	Reproducibilidad y configuración	2/2	Exp. + documental
Total		17/20	

7.5.1 Comparativa de rendimiento

Esta comparativa contrasta los tres *frameworks* sobre las métricas comunes del modo secuencial, que es el que ofrece resultados de aprendizaje equiparables.

7.5.1.1 Precisión final

La Figura 7.4 compara la precisión final por experimento. En el experimento principal (e1) los tres *frameworks* quedan muy próximos, encabezados por Flower (92,03 %), seguido de NVIDIA FLARE (91,14 %) y FedML (90,71 %). Las diferencias son mayores en los experimentos de menor duración: en escalabilidad, FedML obtiene precisiones algo inferiores, especialmente con ocho clientes. En participación parcial (e3) las diferencias entre *frameworks* son reducidas: NVIDIA FLARE alcanza el 90,86 %, Flower el 89,93 % y FedML el 88,95 %; las tres herramientas quedan, por tanto, dentro de un margen estrecho, con Flower en una posición intermedia entre NVIDIA FLARE y FedML en esta configuración.

7.5.1.2 Tiempo de ejecución

La Figura 7.5 compara la duración. Los experimentos con muchas épocas locales (e1 y e3) dominan el tiempo total, como era de esperar. En el experimento principal, los tres *frameworks* quedan en el mismo orden de magnitud temporal: FedML completa la ejecución en 11304 s, Flower en 11864 s y NVIDIA FLARE en 12304 s, de modo que las tres herramientas resultan muy próximas, con FedML ligeramente por delante y NVIDIA FLARE como la más lenta en esta configuración. En los experimentos cortos de escalabilidad los tres *frameworks* quedan en el mismo orden de magnitud, en torno a veinte minutos. Estas diferencias de tiempo deben leerse junto con la Tabla 5.1: como cada *framework* se ejecutó con versiones distintas de Python, PyTorch y CUDA, parte de la distancia observada podría atribuirse a esas diferencias de entorno y no solo a la arquitectura de cada herramienta, una amenaza menor a la validez

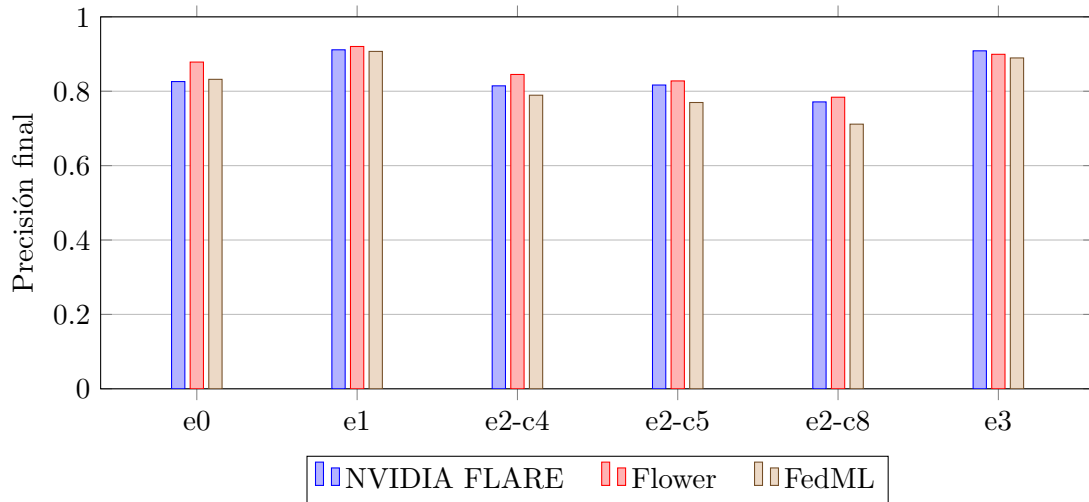


Figura 7.4: Precisión final por experimento y *framework* (modo secuencial).

que ya se recoge en la Sección 5.6.

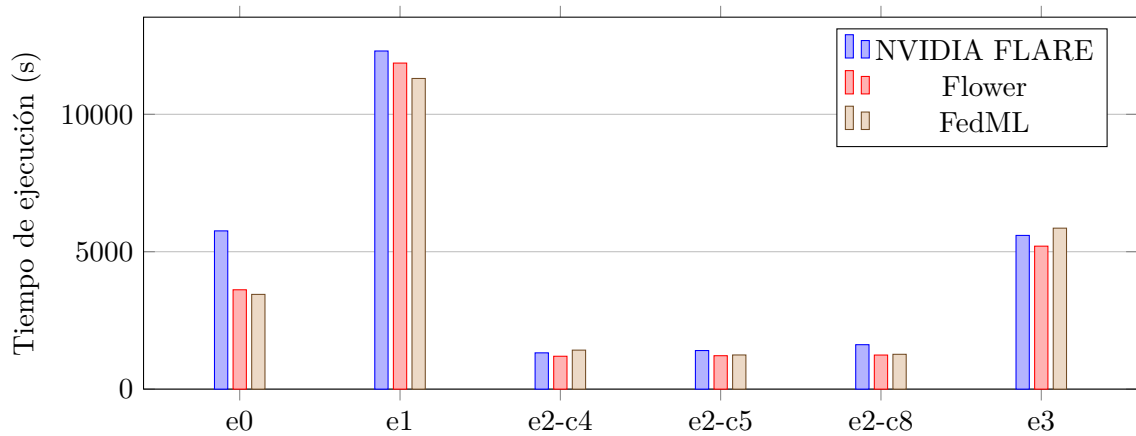


Figura 7.5: Tiempo de ejecución por experimento y *framework* (modo secuencial).

7.5.1.3 Escalabilidad

La Figura 7.6 muestra la precisión final frente al número de clientes en los experimentos de escalabilidad (cuatro, cinco y ocho clientes, con dos épocas locales y diez rondas). Los tres *frameworks* tienden a perder precisión a medida que aumenta el número de clientes, lo que es coherente con el reparto del conjunto: con más clientes, cada uno entrena con menos datos y el mismo número de rondas resulta insuficiente para alcanzar la misma calidad. La caída es monótona en Flower y FedML, mientras que NVIDIA FLARE muestra una ligera subida entre cuatro y cinco clientes (del 81,45 % al 81,67 %) antes de descender con ocho. La pendiente es más pronunciada en FedML.

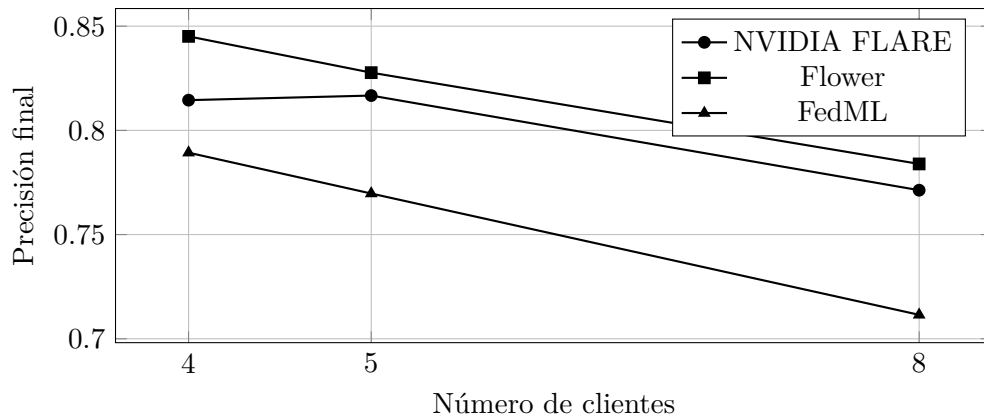


Figura 7.6: Precisión final frente al número de clientes en los experimentos de escalabilidad (modo secuencial).

7.5.1.4 Consumo de recursos

La Figura 7.7 compara la energía estimada de los tres *frameworks*. Tras repetir las ejecuciones afectadas por la instrumentación, FedML dispone de métricas energéticas válidas en todos los experimentos secuenciales principales. En e0, su energía estimada (48,10 Wh) queda próxima a la de Flower (46,31 Wh) y NVIDIA FLARE (51,76 Wh). El consumo energético sigue de cerca al tiempo de ejecución y a la utilización de GPU: los experimentos largos con muchas épocas locales (e1 y e3) concentran el mayor gasto. En el experimento principal (e1), FedML presenta el valor más alto (140,40 Wh), seguido muy de cerca por NVIDIA FLARE (138,51 Wh) y Flower (137,18 Wh), con los tres prácticamente empatados. En participación parcial (e3), Flower obtiene el consumo más bajo (53,52 Wh), mientras que NVIDIA FLARE y FedML quedan prácticamente empatados, con 70,61 Wh y 70,97 Wh respectivamente.

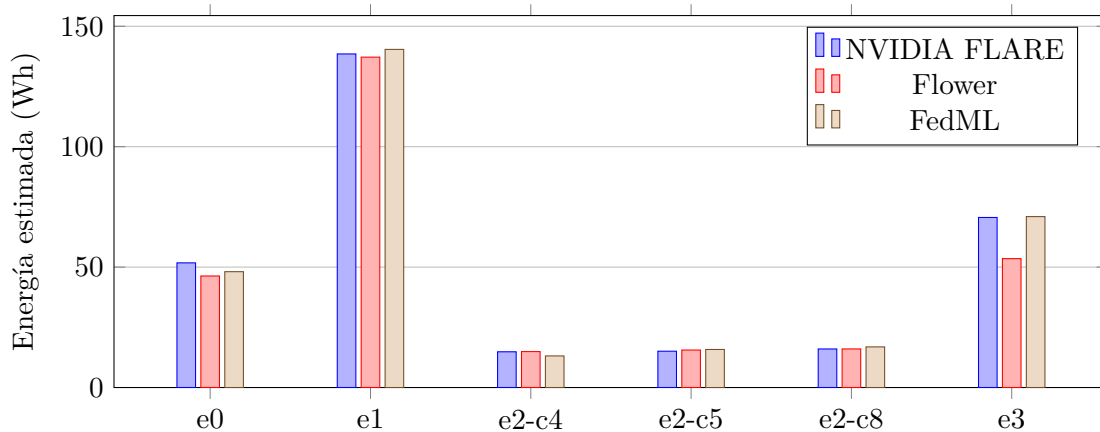


Figura 7.7: Energía estimada por experimento (modo secuencial).

En cuanto a la memoria de GPU, los tres *frameworks* se mantienen muy por debajo de los 6 GB disponibles en el modo secuencial: NVIDIA FLARE en torno a 466 MiB, Flower

alrededor de 760 MiB y FedML en torno a 414–416 MiB. La diferencia más relevante aparece en el modo concurrente de FedML, donde la memoria de GPU crece de forma marcada con el número de clientes (hasta 3727 MiB con ocho clientes), lo que ilustra el coste del paralelismo real entre procesos.

7.5.1.5 Comportamiento en modo concurrente

El modo concurrente reveló las diferencias de arquitectura entre los tres *frameworks* y los límites del equipo utilizado. Cada herramienta implementa la concurrencia de una forma distinta. Flower simula varios clientes en paralelo mediante Ray, que lanza un actor por cliente dentro del mismo nodo. NVIDIA FLARE controla la concurrencia con el número de hilos de entrenamiento del simulador, lo que se traduce en más procesos y clientes activos a la vez. FedML, en su backend concurrente, recurre a MPI: `mpirun` arranca un proceso independiente por cliente más uno de coordinación, que se comunican mediante paso de mensajes.

El resultado en el equipo de pruebas, con 6 GB de GPU y 14 GiB de RAM, fue desigual. En Flower, Ray terminó procesos de trabajo al superar el umbral de memoria del nodo, con errores explícitos de *OutOfMemoryError*; los ajustes probados, como reducir el número de procesos de carga de datos o liberar la caché, no evitaron el fallo. En NVIDIA FLARE, el aumento de hilos provocó una saturación de memoria y procesos que llegó a colgar el sistema. FedML con MPI fue el único que completó la ejecución concurrente, a costa de una utilización de GPU cercana al 100 % y de un mayor consumo de memoria de GPU, aunque sin proporcionar una precisión final extraíble en los *logs*: en el *backend* MPI las ejecuciones finalizaron sin registrar una evaluación final comparable, de modo que solo permiten analizar el comportamiento computacional. Esa precisión podría recuperarse evaluando a posteriori el modelo final guardado con el mismo procedimiento de evaluación centralizada usado en el modo secuencial, lo que queda planteado como trabajo futuro. En conjunto, ningún *framework* resultó inmune a las limitaciones de recursos en modo concurrente, si bien solo FedML representó una concurrencia real entre procesos.

7.5.2 Comparativa de usabilidad

La Tabla 7.15 reúne las doce puntuaciones de las cuatro dimensiones para los tres *frameworks*, lo que permite una lectura transversal de la usabilidad.

Tabla 7.15: Comparativa de la valoración de usabilidad de los tres frameworks según las cuatro dimensiones de la rúbrica (escala 1–5).

Dimensión	Flower	NVIDIA FLARE	FedML
<i>Useful</i>	5	5	4
<i>Usable</i>	4	2	3
<i>Desirable</i>	4	3	3
<i>Accessible</i>	5	3	3
Media	4,5	3,25	3,25

La lectura por dimensiones revela un patrón coherente. En utilidad funcional los tres fra-

meworks obtienen valoraciones altas, ya que todos permiten cubrir el caso de estudio; las diferencias proceden de limitaciones puntuales, como la ausencia de precisión final en el modo concurrente de FedML. Es en la facilidad de uso donde la distancia es máxima: Flower ofrece una puesta en marcha directa y un flujo secuencial estable, FedML se sitúa en un punto intermedio condicionado por la fiabilidad de su instrumentación, y NVIDIA FLARE queda por detrás por su instalación sensible a versiones, su gran número de conceptos y su depuración costosa. En accesibilidad se reproduce la misma ordenación, con Flower claramente por delante gracias a una barrera de entrada baja y unos registros legibles. La dimensión de experiencia de desarrollo es la más matizada, porque la excelente trazabilidad de NVIDIA FLARE compensa parcialmente su dificultad y eleva su valoración por encima de lo que sugeriría su facilidad de uso aislada.

De esta lectura se desprende una recomendación de uso dependiente del perfil del proyecto. Para prototipado rápido, docencia o trabajos en los que el tiempo desde la instalación hasta el primer resultado es crítico, Flower es la opción más usable. Para proyectos que prioricen la trazabilidad, la reproducibilidad rigurosa y la cercanía a un despliegue real, y que puedan asumir una curva de aprendizaje alta, NVIDIA FLARE compensa su coste de uso con la calidad de sus artefactos. FedML ocupa una posición intermedia adecuada cuando interesa disponer de varios backends de ejecución bajo una misma configuración, siempre que se asuma un trabajo adicional de verificación y extracción de métricas.

7.5.3 Comparativa de flexibilidad

Una vez evaluados los tres frameworks por separado, esta comparativa los contrasta sobre el mismo conjunto de criterios. La Tabla 7.16 reúne las puntuaciones F1–F10 de Flower, NVIDIA FLARE y FedML. Los tres obtienen la misma valoración, 17/20, lo que confirma que las tres herramientas son, en lo esencial, comparablemente flexibles para el tipo de adaptaciones que exige este trabajo y que la diferenciación entre ellas debe buscarse, más que en la cifra global, en el perfil cualitativo de cada una.

Tabla 7.16: Comparativa de la valoración de flexibilidad de los tres frameworks según los criterios F1–F10 definidos en la metodología.

Criterio	Descripción	Flower	NVIDIA FLARE	FedML
F1	Modelo y dataset	2/2	2/2	2/2
F2	Entrenamiento local	2/2	2/2	2/2
F3	Agregación personalizada	2/2	2/2	2/2
F4	Algoritmos disponibles	2/2	2/2	2/2
F5	Aprendizaje asíncrono	1/2	1/2	1/2
F6	Modos de ejecución y despliegue	2/2	2/2	2/2
F7	Integración externa	2/2	2/2	2/2
F8	Seguridad y privacidad	1/2	1/2	1/2
F9	Arquitectura federada	1/2	1/2	1/2
F10	Reproducibilidad y configuración	2/2	2/2	2/2
Total		17/20	17/20	17/20

La coincidencia en los criterios experimentales (F1–F4, F6, F7 y F10) refleja un hecho de fondo: bajo un mismo escenario de modelo, datos y algoritmo base, los tres frameworks permitieron integrar la ResNet-18 sobre CIFAR-10, editar el entrenamiento local, sustituir la estrategia de agregación, registrar métricas y reproducir los experimentos mediante ficheros y scripts. La igualdad también alcanza a los tres criterios documentales: ninguno validó experimentalmente el aprendizaje asíncrono (F5), la seguridad avanzada (F8) ni esquemas alternativos a la arquitectura clásica (F9), pero los tres los documentan, por lo que reciben idéntica puntuación parcial. En F5, en concreto, los tres declaran soporte —Flower mediante la *Driver API* y *Flower Next*, NVIDIA FLARE mediante FedBuff en el borde y FedML en su escenario jerárquico cross-silo—, si bien en todos los casos se trata de capacidades experimentales, limitadas a entornos concretos o de bajo nivel. Dentro de ese nivel parcial compartido, el soporte de Flower es el más inmaduro de los tres: se apoya en interfaces de bajo nivel todavía en desarrollo y no en una receta o función dedicada, a diferencia de la **EdgeFedBuffRecipe** de NVIDIA FLARE o del escenario jerárquico de FedML. La puntuación de 1/2 marca, por tanto, un umbral mínimo que las tres herramientas apenas alcanzan, más que un soporte equivalente y consolidado.

Más allá de las cifras, el estudio revela un comportamiento transversal en las estrategias adaptativas de la familia FedOpt. En los tres frameworks fue posible configurar FedAdam y FedAdagrad, pero su ejecución resultó frágil con poca configuración: en Flower divergían con los valores por defecto y necesitaron ajustar la tasa del servidor y el factor de adaptabilidad; en NVIDIA FLARE, FedAdam no llegó a completarse en GPU por un error de CUDA; y en FedML, FedAdam y FedAdagrad finalizaron pero sin aprender. La conclusión es clara y común: la disponibilidad de una estrategia no equivale a su estabilidad, que depende de un ajuste deliberado de hiperparámetros, especialmente con pocas rondas.

Por último, cada framework muestra un perfil de flexibilidad propio que la puntuación, por su granularidad, no captura del todo. Flower es el más directo para sustituir la estrategia de agregación —basta con cambiar una clase en el servidor— y su documentación revela una superficie amplia que abarca *Mods*, privacidad diferencial, FedBN y, sobre todo, un modelo de despliegue muy desarrollado hacia la nube (Google Cloud, Azure, OpenShift e interconexión de clústeres); su contrapartida es que el soporte asíncrono sigue siendo experimental y que el backend Ray arrastra una huella de memoria elevada. NVIDIA FLARE ofrece la mayor profundidad en seguridad y privacidad —filtros de privacidad diferencial, cifrado homomórfico y computación confidencial sobre entornos de ejecución confiables— junto con un catálogo de recetas y entornos (**SimEnv**, **PocEnv**, **ProdEnv**) orientado a producción, a costa de una complejidad de configuración mayor, pues alterar la agregación exige intervenir en componentes del servidor. FedML destaca por la sustitución de entrenador y agregador mediante herencia, por haber ejecutado el mayor número de estrategias, por ser el único cuyo modo concurrente se completó, y por una integración estrecha con su plataforma de operaciones y la analítica federada, con un soporte de seguridad orientado al *benchmarking* de ataques y defensas. La elección entre ellos, en términos de flexibilidad, depende por tanto de la prioridad del proyecto: la simplicidad y el despliegue en la nube de Flower, la profundidad en seguridad y la orientación a producción de NVIDIA FLARE, o la extensibilidad por componentes y la experimentación algorítmica de FedML.

7.6 Desglose gráfico por métrica

Esta sección complementa las tablas y las comparativas anteriores con una representación gráfica detallada del modo secuencial. Primero, para cada *framework* se muestra el valor de cada métrica medida a lo largo de todos los experimentos (e0 a e3), lo que permite observar de un vistazo cómo varía cada indicador con la configuración. Después, tres figuras finales contrastan, métrica a métrica, el valor máximo y mínimo de cada *framework* y señalan el experimento en el que se alcanzó. Todas las cifras proceden de las Tablas 7.1, 7.6 y 7.10; las barras que comienzan en cero permiten comparar magnitudes, por lo que en métricas de poca variación (como la memoria de GPU) las diferencias entre experimentos aparecen, correctamente, como pequeñas.

7.6.1 Desglose por métrica: Flower

Las Figuras 7.8 a 7.11 recogen, métrica a métrica, el comportamiento de Flower a lo largo de los experimentos secuenciales. La precisión reproduce el patrón ya descrito —máxima en la configuración principal (e1) y decreciente al repartir el conjunto entre más clientes en los experimentos de escalabilidad—, mientras que el tiempo de ejecución y la energía estimada se concentran en las configuraciones con más épocas locales (e1 y e3). La memoria de GPU se mantiene estable y muy por debajo de los 6 GB del equipo, y la utilización de GPU permanece alta en todas las configuraciones.

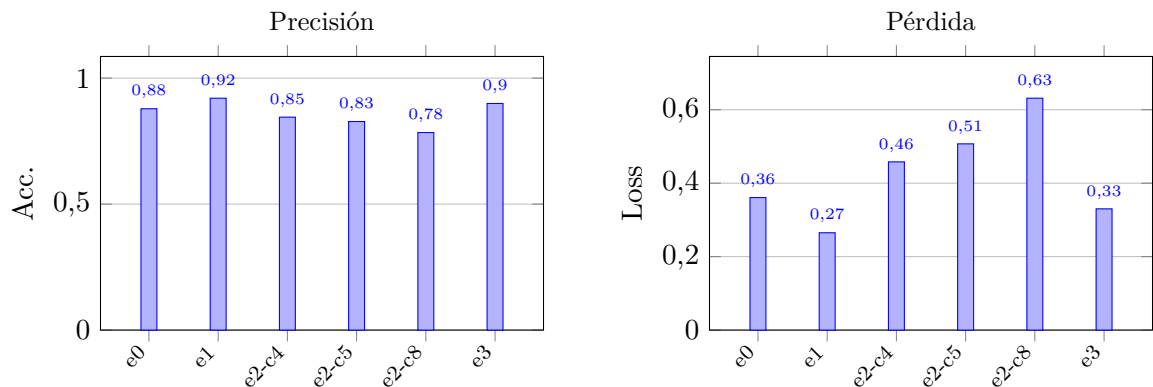


Figura 7.8: Flower: precisión y pérdida final por experimento (modo secuencial).

7.6.2 Desglose por métrica: NVIDIA FLARE

Las Figuras 7.12 a 7.15 muestran las mismas métricas para NVIDIA FLARE. El perfil de precisión y pérdida es muy similar al de Flower, con el mejor resultado en la configuración principal (e1) y la caída esperable al aumentar el número de clientes. Su rasgo diferencial aparece en la memoria de GPU, sensiblemente más baja (en torno a 466 MiB), y en una utilización de GPU que crece con la carga de trabajo, mientras que el tiempo de ejecución y la energía vuelven a concentrarse en los experimentos largos e1 y e3.

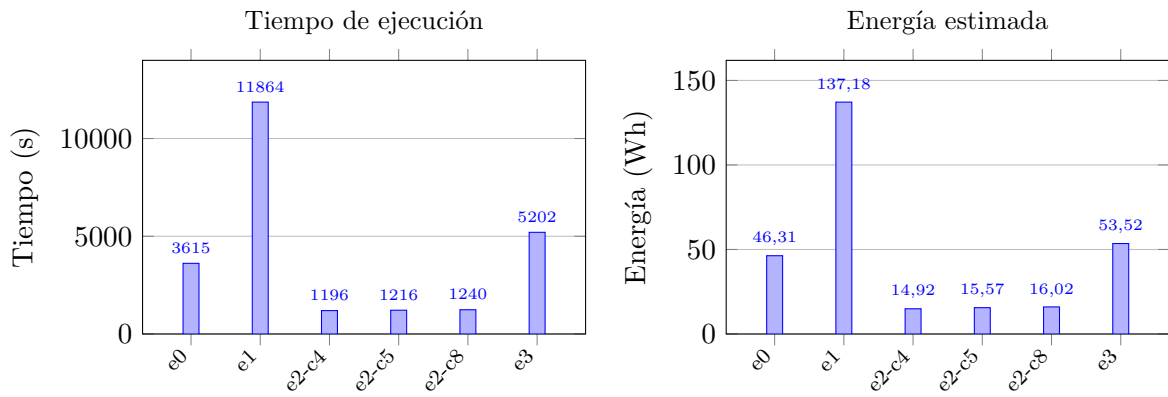


Figura 7.9: Flower: tiempo de ejecución y energía estimada por experimento (modo secuencial).

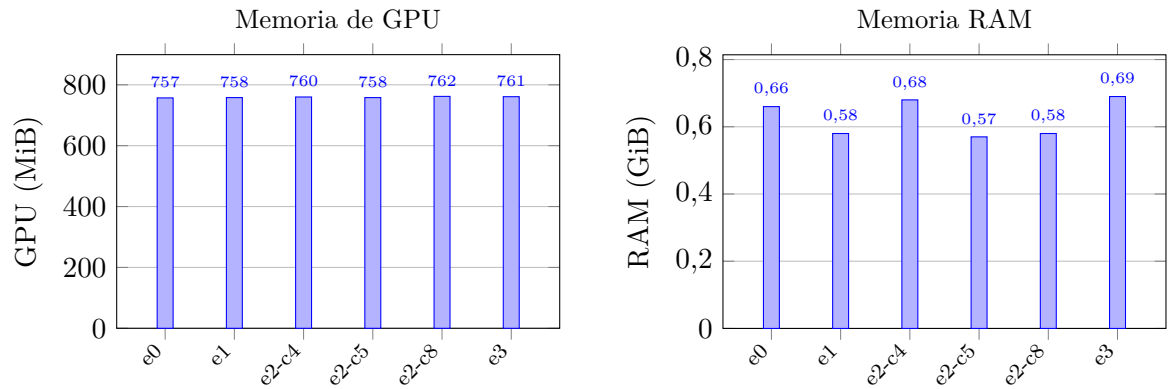


Figura 7.10: Flower: memoria de GPU y memoria RAM por experimento (modo secuencial). La RAM corresponde al proceso principal; el motor de simulación Ray reparte el trabajo en procesos hijo que esta métrica no contabiliza, por lo que infravalora el consumo real.

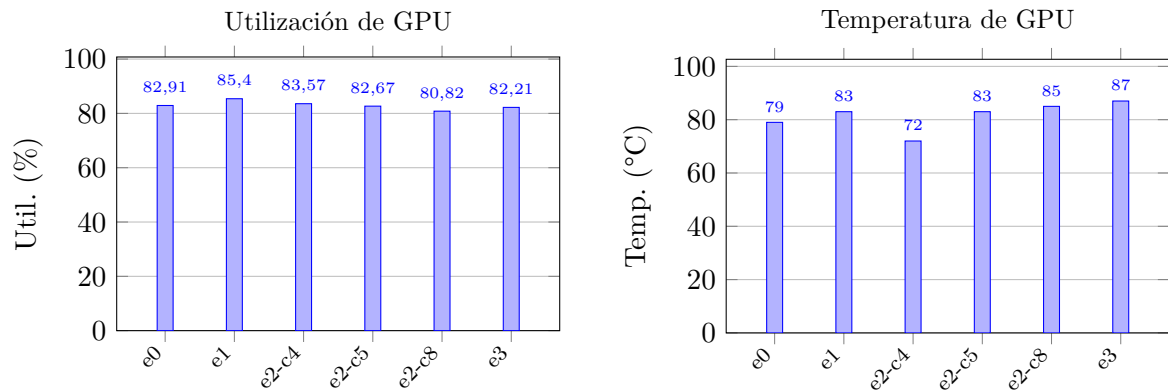


Figura 7.11: Flower: utilización de GPU y temperatura por experimento (modo secuencial).

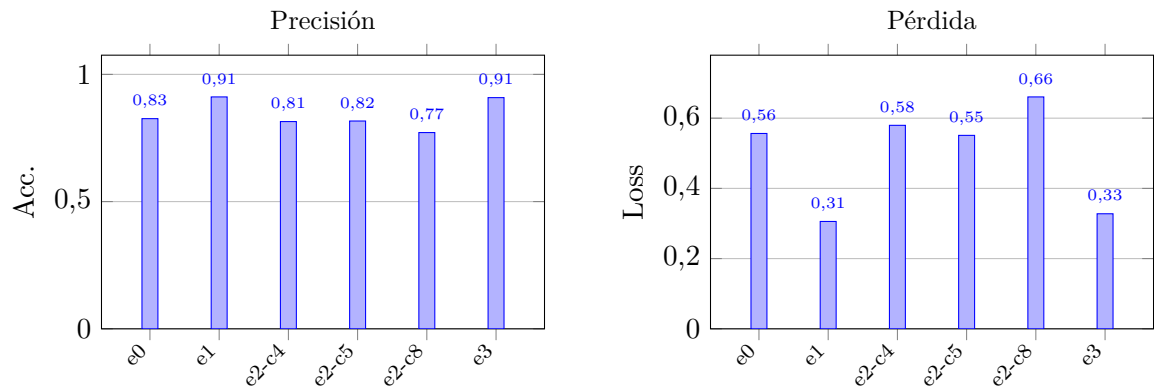


Figura 7.12: NVIDIA FLARE: precisión y pérdida final por experimento (modo secuencial).

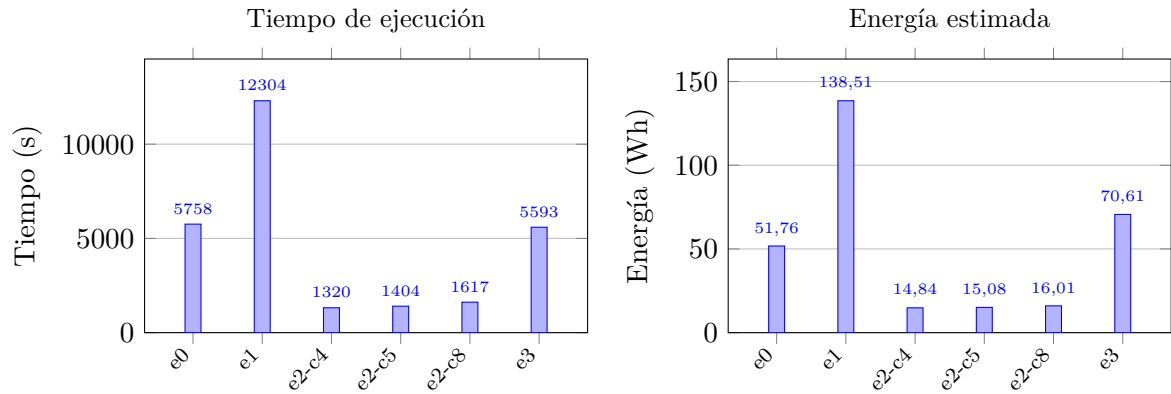


Figura 7.13: NVIDIA FLARE: tiempo de ejecución y energía estimada por experimento (modo secuencial).

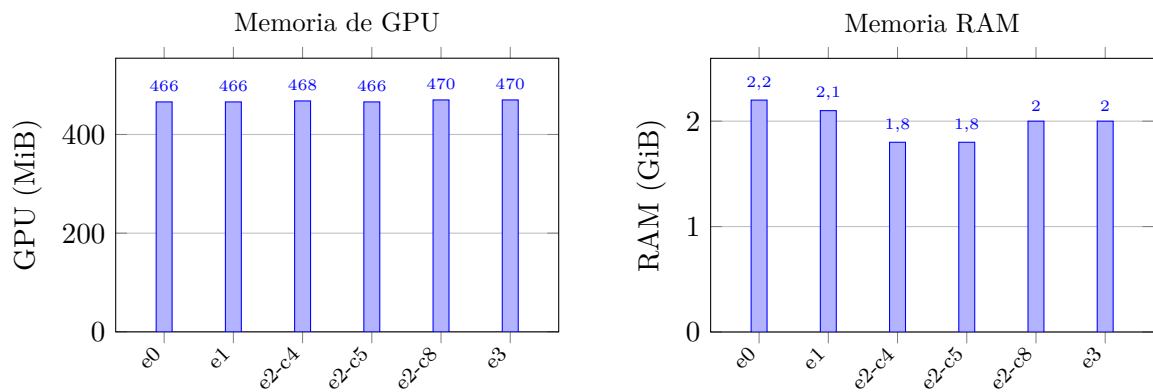


Figura 7.14: NVIDIA FLARE: memoria de GPU y memoria RAM por experimento (modo secuencial). La RAM corresponde al proceso principal.

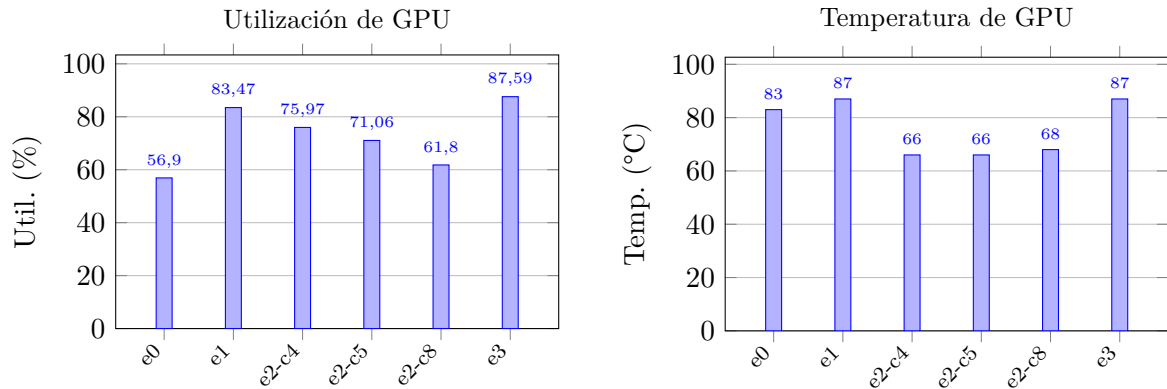


Figura 7.15: NVIDIA FLARE: utilización de GPU y temperatura por experimento (modo secuencial).

7.6.3 Desglose por métrica: FedML

Las Figuras 7.16 a 7.19 completan el desglose con FedML. La precisión sigue la tendencia común, aunque con la caída más pronunciada del estudio en los experimentos de escalabilidad, especialmente con ocho clientes. El tiempo de ejecución y la energía se mantienen en el mismo orden de magnitud que los otros dos *frameworks* en el modo secuencial, con la utilización de GPU más alta de la terna; la memoria de GPU es la menor de las tres herramientas (en torno a 414–416 MiB) en este modo.

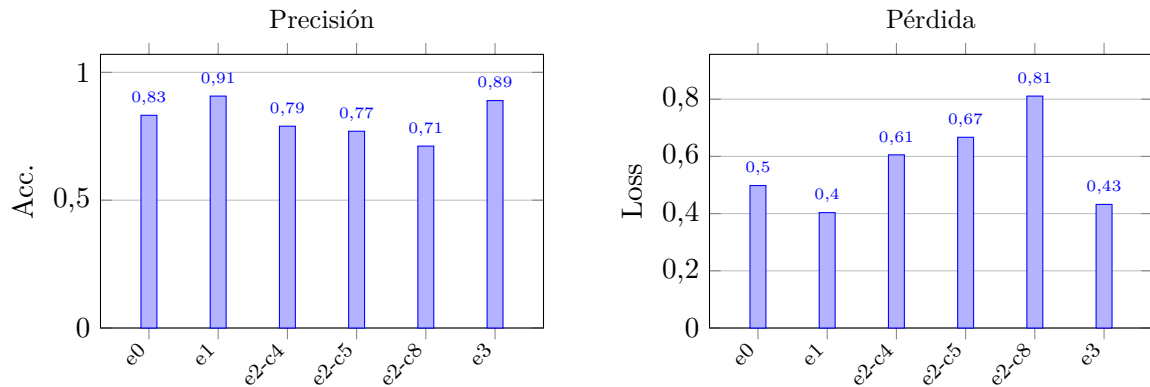


Figura 7.16: FedML: precisión y pérdida final por experimento (modo secuencial).

7.6.4 Valores máximos y mínimos entre frameworks

Las Figuras 7.20, 7.21 y 7.22 resumen, para tres métricas representativas, el valor máximo y mínimo que alcanzó cada *framework* a lo largo de todos los experimentos secuenciales, indicando bajo cada barra el experimento en el que se registró. En estas figuras, NVFLARE abrevia NVIDIA FLARE. Cuando el extremo se repitió en varios experimentos, se enumeran todos.

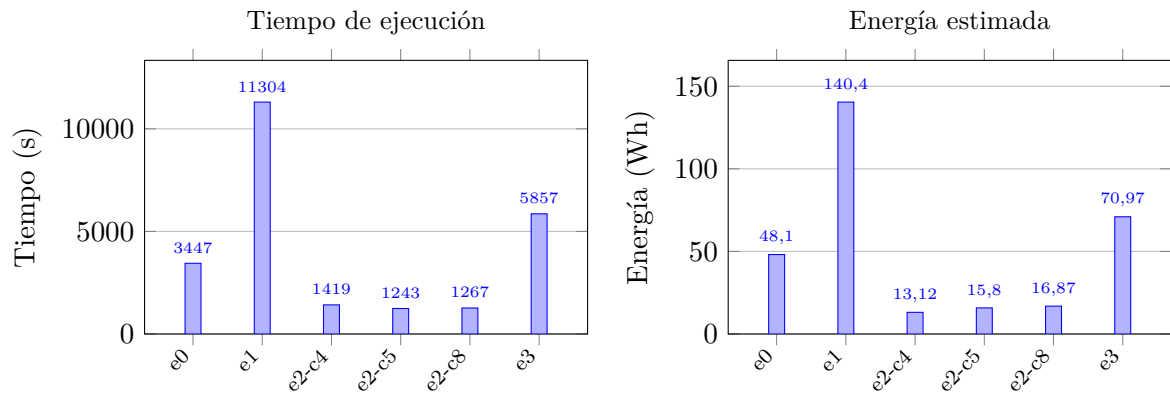


Figura 7.17: FedML: tiempo de ejecución y energía estimada por experimento (modo secuencial).

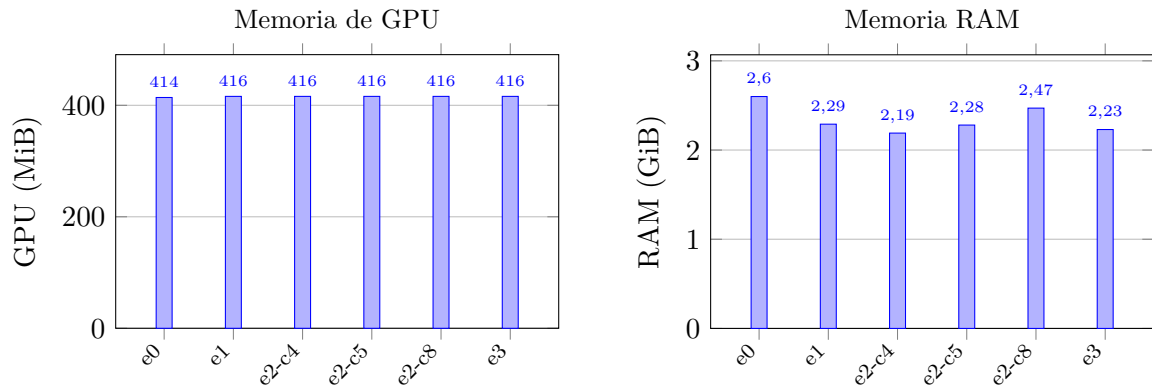


Figura 7.18: FedML: memoria de GPU y memoria RAM por experimento (modo secuencial). En este modo FedML se ejecuta en un único proceso (*backend sp*), por lo que la RAM del proceso principal refleja el consumo real del *framework*.

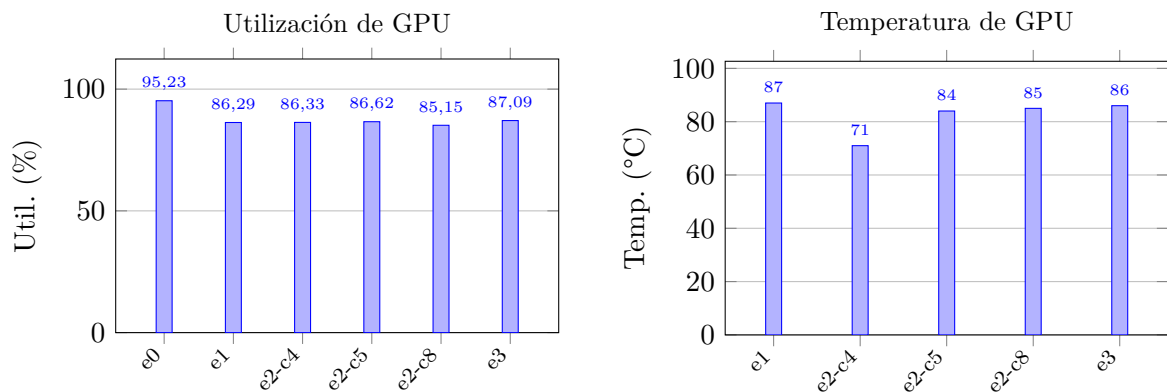


Figura 7.19: FedML: utilización de GPU y temperatura por experimento (modo secuencial). No se registró la temperatura de FedML en e0.

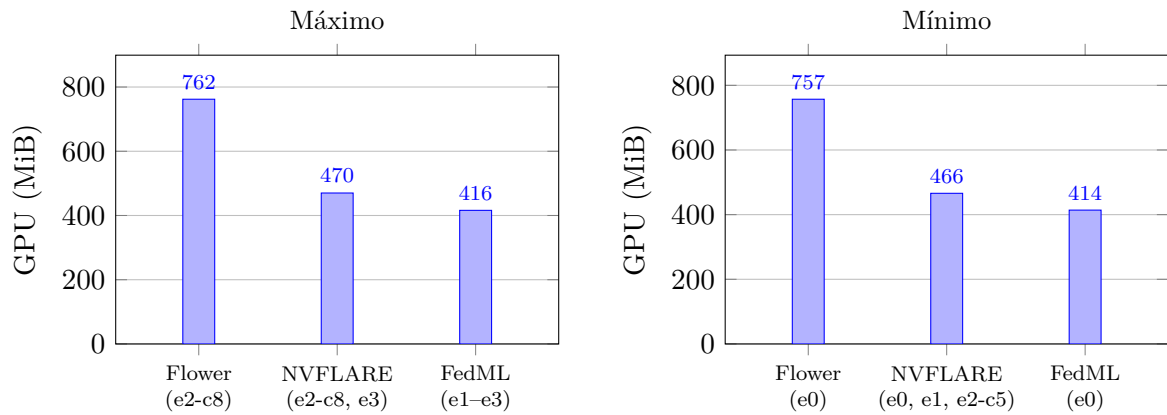


Figura 7.20: Memoria de GPU máxima y mínima por *framework* (modo secuencial), con el experimento en el que se alcanzó. En NVIDIA FLARE y FedML el extremo se repite en varios experimentos.

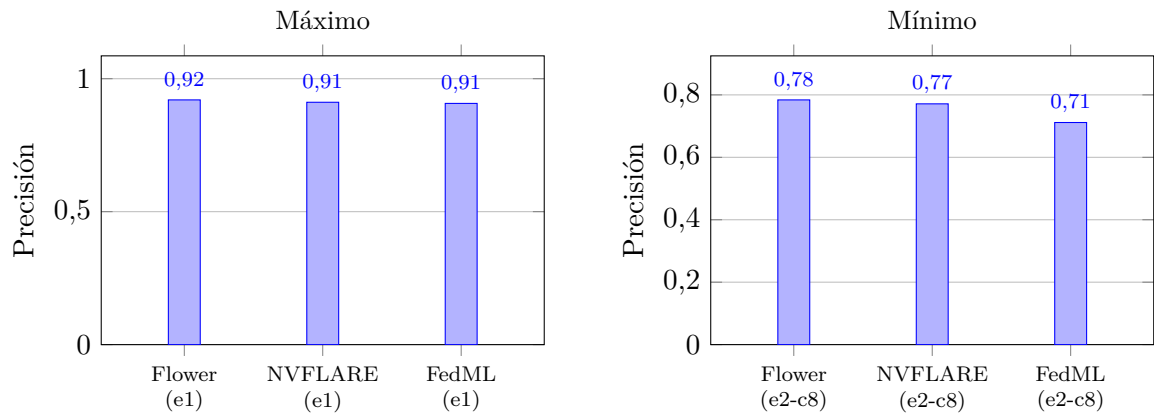


Figura 7.21: Precisión máxima y mínima por *framework* (modo secuencial). El máximo de los tres se dio en e1 (configuración principal) y el mínimo en e2-c8 (ocho clientes).

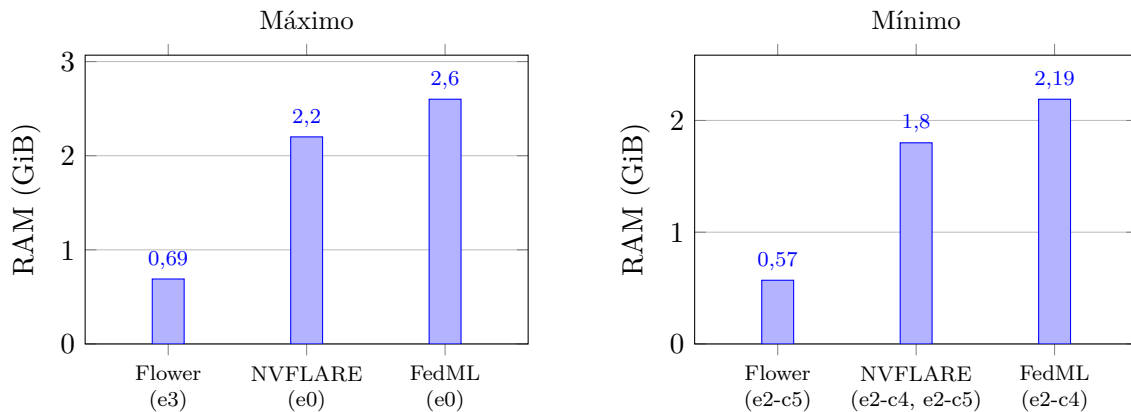


Figura 7.22: Memoria RAM máxima y mínima por *framework* (modo secuencial). La RAM corresponde al proceso principal medido con `/usr/bin/time`. En este modo, NVIDIA FLARE y FedML (*backend sp*) se ejecutan en un único proceso, mientras que Flower delega en Ray, que reparte el trabajo en procesos hijo no contabilizados por esta métrica; por ello la cifra de Flower aparece artificialmente baja y la comparación entre *frameworks* debe interpretarse con cautela.

7.7 Síntesis global de resultados

Este capítulo ha evaluado Flower, NVIDIA FLARE y FedML sobre los tres ejes planteados —rendimiento, usabilidad y flexibilidad—, organizando los resultados por *framework* para ofrecer una visión completa de cada herramienta antes de contrastarlas. La síntesis que sigue reúne las conclusiones de los tres ejes y permite una lectura conjunta del comportamiento de cada herramienta.

En el plano del rendimiento, en modo secuencial los tres *frameworks* alcanzan precisiones muy similares en el experimento principal (en torno al 90,7–92,0 %: Flower 92,03 %, NVIDIA FLARE 91,14 % y FedML 90,71 %), lo que confirma que, bajo idénticas condiciones de datos, modelo e hiperparámetros, la elección de la herramienta influye poco en la calidad del modelo y mucho en el coste de obtenerlo. Las diferencias más claras aparecen en el tiempo de ejecución, en la utilización de GPU y, sobre todo, en la estabilidad del modo concurrente, donde el equipo utilizado marcó el límite: en el experimento principal NVIDIA FLARE y FedML registraron tiempos prácticamente equivalentes, ligeramente inferiores al de Flower, y FedML fue el único *framework* cuyo modo concurrente llegó a completarse, aunque sin producir una precisión final comparable en los registros analizados.

En el plano de la usabilidad, los tres *frameworks* resultaron funcionalmente aptos para el caso de estudio, pero ofrecieron experiencias de desarrollo marcadamente distintas: Flower destacó por su accesibilidad y su bajo coste de aprendizaje (media de 4,5), NVIDIA FLARE por su trazabilidad a costa de una notable complejidad (3,25), y FedML por un equilibrio razonable con limitaciones concretas en la instrumentación y la extracción de resultados (3,25). Conviene recordar que estas valoraciones reflejan el juicio del autor como desarrollador sobre un único entorno de pruebas, por lo que deben interpretarse como una guía cualitativa y no como una medida absoluta; esta condición se recoge entre las amenazas a la validez del estudio.

En el plano de la flexibilidad, los tres *frameworks* obtuvieron la misma valoración global (17/20), lo que indica que todos permiten el tipo de adaptaciones que exige este trabajo —integrar el modelo y los datos, editar el entrenamiento local, sustituir la estrategia de agregación, registrar métricas y reproducir los experimentos—, y que su diferenciación reside en el perfil cualitativo: la simplicidad y el despliegue en la nube de Flower, la profundidad en seguridad y la orientación a producción de NVIDIA FLARE, y la extensibilidad por componentes y la experimentación algorítmica de FedML. Un hallazgo transversal a los tres ejes es la fragilidad de las estrategias adaptativas de la familia FedOpt: su disponibilidad quedó demostrada en todos los frameworks, pero su estabilidad exige un ajuste deliberado de hiperparámetros que va más allá de su mera activación.

En conjunto, los resultados confirman que la elección de un *framework* de aprendizaje federado no puede basarse únicamente en su rendimiento, pues bajo idénticas condiciones experimentales las tres herramientas convergen en calidad de modelo y se diferencian sobre todo en el coste de obtenerlo, en la experiencia de uso y en el perfil de flexibilidad. Estas observaciones, integradas a partir de los tres ejes de evaluación, sirven de base para las conclusiones del trabajo.

8 Conclusiones

Este capítulo cierra el trabajo recapitulando lo realizado, comprobando en qué medida se han alcanzado los objetivos planteados y sintetizando las conclusiones que se desprenden de la comparación experimental. A diferencia del capítulo de resultados, que presenta y discute las cifras en detalle, aquí se adopta una lectura de conjunto: qué enseña el estudio sobre la elección de un *framework* de aprendizaje federado, qué aporta respecto al estado del arte, con qué limitaciones debe interpretarse y qué líneas quedan abiertas. No se introducen datos ni resultados nuevos; las conclusiones se apoyan en la evidencia ya expuesta en los capítulos anteriores.

8.1 Consecución de los objetivos

El objetivo general del trabajo era comparar de forma experimental varios *frameworks* de aprendizaje federado ejecutando un mismo escenario de entrenamiento en todos ellos y observando tanto el rendimiento del modelo como el consumo de recursos y la experiencia de uso. Este objetivo se ha cumplido: Flower, NVIDIA FLARE y FedML se han implementado y evaluado sobre un escenario común —clasificación de CIFAR-10 con una ResNet-18 adaptada, el algoritmo FedAvg y una configuración cross-silo horizontal— y se han contrastado a lo largo de tres ejes complementarios. Los objetivos específicos también se han satisfecho de la manera que se resume a continuación.

El primer objetivo específico, relativo al estudio de los fundamentos del aprendizaje federado, se abordó en el capítulo de fundamentos teóricos, que delimitó el funcionamiento de la ronda federada, el algoritmo FedAvg, los tipos de aprendizaje federado y los retos derivados de la heterogeneidad de datos y recursos. El segundo, centrado en la necesidad de *frameworks* especializados y en los aspectos que influyen en su elección, se cubrió en el estado del arte, donde se revisó el panorama de herramientas, se fijaron los criterios de selección y se justificó la terna escogida.

El tercer y el cuarto objetivo, de diseño e implementación, se materializaron en el capítulo de metodología y en el de diseño experimental e implementación: se definió un escenario común con el conjunto de datos, el modelo, el algoritmo y los hiperparámetros constantes, y se adaptó ese escenario a la arquitectura propia de cada herramienta. El quinto objetivo, relativo a la recogida de métricas de aprendizaje, rendimiento y consumo de recursos, y el sexto, sobre la evaluación de la experiencia práctica de uso, se desarrollaron en el capítulo de resultados mediante una comparativa cuantitativa de rendimiento y dos valoraciones cualitativas, de usabilidad y de flexibilidad. Por último, el séptimo objetivo, de comparación crítica y señalamiento de ventajas, limitaciones y escenarios de uso, se atendió en la comparativa transversal entre *frameworks* y se recoge, condensado, en las conclusiones que siguen.

8.2 Conclusiones principales

La conclusión central del trabajo es que, bajo idénticas condiciones de datos, modelo e hiperparámetros, la elección del *framework* influye poco en la calidad del modelo y mucho en el coste de obtenerlo, en la experiencia de desarrollo y en la estabilidad del modo concurrente. Esta idea, que articula los tres ejes, se concreta a continuación.

8.2.1 Rendimiento

En modo secuencial, los tres *frameworks* alcanzaron precisiones muy próximas en el experimento principal, en torno al 90,7–92,0%, con una separación máxima de poco más de un punto porcentual entre el mejor y el peor resultado. Esa convergencia confirma que, cuando se mantienen constantes el conjunto de datos, la arquitectura del modelo y la configuración de entrenamiento, la herramienta apenas condiciona la precisión final del modelo global. Las diferencias relevantes aparecieron en otros indicadores: el tiempo de ejecución, la utilización de la GPU y, sobre todo, el comportamiento en modo concurrente. En el experimento principal, NVIDIA FLARE y FedML registraron tiempos prácticamente equivalentes, ligeramente inferiores al de Flower, y FedML fue el único cuyo modo concurrente llegó a completarse en el entorno utilizado, aunque esas ejecuciones no proporcionaron una precisión final comparable en los registros analizados. El modo concurrente marcó además el límite del equipo utilizado: ninguna de las tres herramientas resultó inmune a las restricciones de recursos, si bien solo el backend MPI de FedML representó una concurrencia real entre procesos.

8.2.2 Usabilidad

En el eje de usabilidad, los tres *frameworks* resultaron funcionalmente aptos para el caso de estudio, pero ofrecieron experiencias de desarrollo claramente distintas. Flower fue la herramienta más accesible y de menor coste de aprendizaje, con una puesta en marcha directa y un flujo secuencial estable. NVIDIA FLARE destacó por la trazabilidad y la riqueza de sus registros, a costa de una instalación sensible a las versiones, un número elevado de conceptos propios y una depuración exigente. FedML ocupó una posición intermedia, equilibrada pero condicionada por la fiabilidad de su instrumentación y por el trabajo adicional de extracción de métricas. De esta lectura se desprende una recomendación dependiente del perfil del proyecto: Flower para prototipado rápido, docencia o trabajos en los que el tiempo hasta el primer resultado es crítico; NVIDIA FLARE cuando se priorizan la trazabilidad, la reproducibilidad rigurosa y la cercanía a un despliegue real; y FedML cuando interesa disponer de varios backends de ejecución bajo una misma configuración.

8.2.3 Flexibilidad

En flexibilidad, las tres herramientas obtuvieron una valoración global equivalente, lo que indica que todas permiten el tipo de adaptaciones que exige este trabajo: integrar un modelo y un conjunto de datos propios, editar el entrenamiento local, sustituir la estrategia de agregación, registrar métricas a medida y reproducir los experimentos mediante ficheros y scripts. La diferenciación entre ellas no está, por tanto, en la cifra global, sino en el perfil cualitativo: la simplicidad para sustituir la estrategia de agregación y la orientación al despliegue en la

nube de Flower; la profundidad en seguridad y privacidad y la vocación de producción de NVIDIA FLARE; y la extensibilidad por componentes y la experimentación algorítmica de FedML.

Un hallazgo transversal a los tres ejes es la fragilidad de las estrategias adaptativas de la familia FedOpt. En los tres *frameworks* fue posible configurar FedAdam y FedAdagrad, pero su ejecución resultó inestable con poca configuración —divergían, fallaban o no llegaban a aprender según el caso—, lo que evidencia que la mera disponibilidad de una estrategia no equivale a su estabilidad: esta depende de un ajuste deliberado de hiperparámetros, especialmente cuando el número de rondas es reducido.

La lectura conjunta de los tres ejes conduce, así, a una conclusión única: seleccionar un *framework* de aprendizaje federado atendiendo solo a su rendimiento resulta insuficiente, porque en condiciones equivalentes las tres herramientas rinden de forma prácticamente indistinguible y la diferencia real reside en cuánto cuesta llegar a ese modelo, en lo cómodo que resulta el desarrollo y en el margen de adaptación que ofrece cada una.

8.3 Aportaciones del trabajo

La aportación principal de este trabajo es una comparación experimental, razonada y reproducible de tres *frameworks* de aprendizaje federado, que ayuda a cubrir un hueco detectado en el estado del arte: la mayoría de los estudios previos se centra en aspectos cualitativos o documentales y rara vez mide, sobre un mismo escenario y en condiciones equivalentes, tanto la precisión del modelo como el consumo de recursos de cada herramienta. Sobre esa idea general, el trabajo aporta varios elementos concretos.

En primer lugar, el diseño de un escenario experimental común y controlado, con el conjunto de datos, el modelo, el algoritmo, los hiperparámetros y el hardware fijados, que permite atribuir las diferencias observadas al *framework* y no a otros factores. En segundo lugar, una evaluación articulada en tres ejes complementarios —rendimiento, usabilidad y flexibilidad— que ofrece una visión más completa que la habitual centrada solo en la precisión o en la velocidad. En tercer lugar, dos instrumentos de valoración reutilizables: una rúbrica de usabilidad basada en las dimensiones de experiencia de uso y una rúbrica de flexibilidad con diez criterios, que separan de forma explícita la evidencia experimental de la documental. Por último, la documentación rigurosa del comportamiento en modo concurrente, incluidas las incidencias de cada herramienta, que constituye una evidencia práctica de interés para quien deba afrontar este tipo de ejecuciones en hardware limitado.

8.4 Limitaciones del estudio

Las conclusiones anteriores deben interpretarse a la luz de las limitaciones ya señaladas en la metodología. La principal es de hardware: la GPU de 6 GB y los 14 GiB de memoria RAM del equipo condicionaron el modo concurrente en las tres herramientas, de modo que el comportamiento concurrente refleja tanto el diseño de cada *framework* como el límite del entorno de pruebas. Además, cada herramienta se ejecutó en un entorno de software distinto, con versiones diferentes de Python, PyTorch y CUDA, lo que supone una amenaza menor a la validez de la comparación de rendimiento, mitigada al documentar las versiones exactas.

A ello se añaden otras restricciones de alcance. El estudio se limita a una distribución de datos IID, por lo que no evalúa el efecto de la heterogeneidad no IID. Los hiperparámetros se fijaron a priori, sin optimización individual por *framework*, de modo que los resultados corresponden a esa configuración concreta. Cada configuración se ejecutó una sola vez con una semilla fija, por lo que no se dispone de medidas de variabilidad entre repeticiones y los valores deben leerse como una única observación por configuración. Toda la experimentación se realizó en simulación sobre una sola máquina, sin latencias de red reales ni efectos propios de un despliegue distribuido. Por último, las métricas de consumo de recursos dependen del hardware empleado y las valoraciones de usabilidad y flexibilidad, aunque apoyadas en evidencias registradas, incorporan el juicio del autor como desarrollador, por lo que deben tomarse como una guía cualitativa y no como una medida absoluta.

8.5 Líneas de trabajo futuro

El trabajo abre varias líneas de continuación que prolongan de forma natural lo realizado.

- **Escenarios no IID.** Repetir la comparación con particiones no independientes ni idénticamente distribuidas permitiría observar cómo afecta la heterogeneidad de los datos al rendimiento y a la convergencia de cada *framework*, un factor deliberadamente aislado en este estudio.
 - **Recuperación de la precisión del modo concurrente de FedML.** Dado que las ejecuciones concurrentes de FedML completaron pero no registraron una precisión final extraíble, una evaluación a posteriori del modelo guardado, con el mismo procedimiento de evaluación centralizada del modo secuencial, permitiría incorporar esos resultados a la comparación.
 - **Análisis de variabilidad.** Ejecutar cada configuración con varias semillas haría posible reportar media y desviación típica y estimar la significación de las diferencias observadas, superando la limitación de la observación única.
 - **Ajuste de las estrategias adaptativas.** Profundizar en la configuración de la familia FedOpt —tasa de aprendizaje del servidor, factor de adaptabilidad y número de rondas— permitiría determinar en qué condiciones FedAdam y FedAdagrad resultan competitivos frente a FedAvg.
 - **Despliegue distribuido real.** Trasladar el experimento de la simulación a un despliegue con clientes en máquinas distintas mediría latencias de red y efectos de un entorno federado realista, especialmente relevante para NVIDIA FLARE y FedML, orientados a producción.
 - **Mecanismos de seguridad y privacidad.** Evaluar de forma experimental la privacidad diferencial, la agregación segura o el cifrado, capacidades que en este trabajo solo se verificaron documentalmente, completaría el análisis de flexibilidad con evidencia ejecutada.
-

8.6 Reflexión final

El desarrollo de este trabajo ha mostrado que comparar herramientas de aprendizaje federado exige mucho más que medir precisiones: obliga a implementar un mismo escenario sobre arquitecturas muy distintas, a instrumentar la ejecución con cuidado y a distinguir con honestidad entre lo que se ha verificado experimentalmente y lo que solo consta en la documentación. El resultado es una fotografía equilibrada de tres herramientas que, lejos de poder ordenarse en un ranking absoluto, responden a prioridades complementarias. La principal enseñanza, tanto para este estudio como para quien deba seleccionar un *framework* en un proyecto similar, es que la decisión debe guiarse por el contexto —el coste asumible, la experiencia de desarrollo buscada y las necesidades de flexibilidad y despliegue— y no por una supuesta superioridad en precisión que, en condiciones equivalentes, apenas distingue a unas herramientas de otras.

Bibliografía

- Beutel, D. J., Topal, T., Mathur, A., Qiu, X., Fernandez-Marques, J., Gao, Y., Sani, L., Li, K. H., Parcollet, T., Porto Buarque de Gusmão, P., & Lane, N. D. (2020). Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390*.
- Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C., Konečný, J., Mazzocchi, S., McMahan, H. B., Van Overveldt, T., Petrou, D., Ramage, D., & Roselander, J. (2019). Towards Federated Learning at Scale: System Design. *Proceedings of Machine Learning and Systems (MLSys)*, 1, 374-388.
- Caldas, S., Duddu, S. M. K., Wu, P., Li, T., Konečný, J., McMahan, H. B., Smith, V., & Talwalkar, A. (2018). LEAF: A Benchmark for Federated Settings. *arXiv preprint arXiv:1812.01097*.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- He, C., Li, S., So, J., Zeng, X., Zhang, M., Wang, H., Wang, X., Vepakomma, P., Singh, A., Qiu, H., Zhu, X., Wang, J., Shen, L., Zhao, P., Kang, Y., Liu, Y., Raskar, R., Yang, Q., Annavaram, M., & Avestimehr, S. (2020). FedML: A Research Library and Benchmark for Federated Machine Learning. *arXiv preprint arXiv:2007.13518*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778. <https://doi.org/10.1109/CVPR.2016.90>
- International Organization for Standardization. (2018). *ISO 9241-11:2018. Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts*. ISO. Geneva, Switzerland.
- Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., et al. (2021). Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning*, 14(1-2), 1-210. <https://doi.org/10.1561/22000000083>
- Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S. J., Stich, S. U., & Suresh, A. T. (2020). SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 119, 5132-5143.
- Li, T., Sahu, A. K., Talwalkar, A., & Smith, V. (2020). Federated Learning: Challenges, Methods, and Future Directions. *IEEE Signal Processing Magazine*, 37(3), 50-60. <https://doi.org/10.1109/MSP.2020.2975749>
- Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated Optimization in Heterogeneous Networks. *Proceedings of Machine Learning and Systems (MLSys)*, 2, 429-450.
- Liu, Y., Fan, T., Chen, T., Xu, Q., & Yang, Q. (2021). FATE: An Industrial Grade Platform for Collaborative Learning With Data Protection. *Journal of Machine Learning Research*, 22(226), 1-6.

- Luján Domenech, R. (2026). *Federated Learning Frameworks Comparative: código y experimentos del TFG*. GitHub. Consultado el 22 de junio de 2026, desde <https://github.com/rld8/Federated-Learning-Frameworks-Comparative>
- McMahan, B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 54, 1273-1282.
- Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A Distributed Framework for Emerging AI Applications. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 561-577.
- Morville, P. (2004). *User Experience Design*. Semantic Studios. Consultado el 1 de junio de 2026, desde https://semanticstudios.com/user_experience_design/
- Nielsen, J. (1993). *Usability Engineering*. Morgan Kaufmann.
- NVIDIA. (2026a). *Available Recipes*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/user_guide/data_scientist_guide/available_recipes.html
- NVIDIA. (2026b). *Client Controlled Workflows*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/programming_guide/controllers/client_controlled_workflows.html
- NVIDIA. (2026c). *Cyclic Workflow*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/programming_guide/controllers/cyclic_workflow.html
- NVIDIA. (2026d). *Differential Privacy in FLARE*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/user_guide/admin_guide/security/differential_privacy.html
- NVIDIA. (2026e). *Experiment Tracking*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/programming_guide/experiment_tracking.html
- NVIDIA. (2026f). *FL Algorithms*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/examples/fl_algorithms.html
- NVIDIA. (2026g). *FLARE Confidential Federated AI*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/user_guide/confidential_computing/index.html
- NVIDIA. (2026h). *NVFLARE Component Configuration and Event Handling*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/programming_guide/component_configuration.html
- NVIDIA. (2026i). *nvflare.app_opt.pt.fedopt module*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/apidocs/nvflare.app_opt.pt.fedopt.html
- NVIDIA. (2026j). *nvflare.edge.assessors.model_update module*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/apidocs/nvflare.edge.assessors.model_update.html
- NVIDIA. (2026k). *NVIDIA FLARE Security Overview*. Consultado el 14 de junio de 2026, desde https://nvflare.readthedocs.io/en/main/system_architecture/security_overview.html
- NVIDIA. (2026l). *Welcome to NVIDIA FLARE*. Consultado el 14 de junio de 2026, desde <https://nvflare.readthedocs.io/en/main/welcome.html>
-

-
- Reddi, S. J., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., & McMahan, H. B. (2021). Adaptive Federated Optimization. *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=LkFG3IB13U5>
- Reina, G. A., Gruzdev, A., Foley, P., Aono, O., Moorthy, A., Shah, S.-h., Edwards, B., Martin, J., Baid, U., & Bakas, S. (2021). OpenFL: An Open-Source Framework for Federated Learning. <https://arxiv.org/abs/2105.06413>
- Riedel, P., Schick, L., von Schwerin, R., Reichert, M., Schaudt, D., & Hafner, A. (2024). Comparative analysis of open-source federated learning frameworks – a literature-based survey and review. *International Journal of Machine Learning and Cybernetics*, 15(11), 5257-5278. <https://doi.org/10.1007/s13042-024-02234-z>
- Roth, H. R., Cheng, Y., Wen, Y., Yang, I., Xu, Z., Hsieh, Y.-T., Kersten, K., Harouni, A., Zhao, C., Lu, K., Zhang, Z., Li, W., Myronenko, A., Yang, D., Yang, S., Rieke, N., Quraini, A., Chen, C., Xu, D., ... Feng, A. (2022). NVIDIA FLARE: Federated Learning from Simulation to Real-World. *arXiv preprint arXiv:2210.13291*.
- Ryffel, T., Trask, A., Dahl, M., Wagner, B., Mancuso, J., Rueckert, D., & Passerat-Palmbach, J. (2018). A Generic Framework for Privacy Preserving Deep Learning. <https://arxiv.org/abs/1811.04017>
- TensorOpera. (2026a). *Cross-Silo Federated Learning: Overview*. Consultado el 14 de junio de 2026, desde <https://docs.tensoropera.ai/federate/cross-silo/overview>
- TensorOpera. (2026b). *Customize Trainer and Aggregator*. Consultado el 14 de junio de 2026, desde <https://docs.tensoropera.ai/federate/customize-trainer-and-aggregator>
- TensorOpera. (2026c). *Customize Your Own Dataloader*. Consultado el 14 de junio de 2026, desde https://docs.tensoropera.ai/federate/data_loader_customization
- TensorOpera. (2026d). *Experimental Tracking*. Consultado el 14 de junio de 2026, desde https://docs.tensoropera.ai/train/experimental_tracking
- TensorOpera. (2026e). *FedML Simulation API Reference*. Consultado el 14 de junio de 2026, desde <https://docs.tensoropera.ai/federate/simulation/api>
- TensorOpera. (2026f). *FedML Simulation Overview*. Consultado el 14 de junio de 2026, desde <https://docs.tensoropera.ai/federate/simulation/overview>
- TensorOpera. (2026g). *Pre-built Jobs and Examples*. Consultado el 14 de junio de 2026, desde <https://docs.tensoropera.ai/federate/examples>
- TensorOpera. (2026h). *Security in Federated Learning: Overview*. Consultado el 14 de junio de 2026, desde <https://docs.tensoropera.ai/federate/security/overview>
- The Flower Authors. (2026). *Flower Framework Documentation (v1.31)*. Flower Labs. Consultado el 22 de junio de 2026, desde <https://flower.ai/docs/framework/>
- The TensorFlow Federated Authors. (2019). *TensorFlow Federated: Machine Learning on Decentralized Data*. Google. Consultado el 16 de junio de 2026, desde <https://www.tensorflow.org/federated>
- World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C. Consultado el 1 de junio de 2026, desde <https://www.w3.org/TR/WCAG21/>
- Yang, Q., Liu, Y., Chen, T., & Tong, Y. (2019). Federated Machine Learning: Concept and Applications. *ACM Transactions on Intelligent Systems and Technology*, 10(2), 1-19. <https://doi.org/10.1145/3298981>
-

Lista de Acrónimos y Abreviaturas

API	Application Programming Interface.
CPU	Central Processing Unit.
CUDA	Compute Unified Device Architecture.
DP	Differential Privacy.
FedAvg	Federated Averaging.
FedOpt	Federated Optimization.
FedProx	Federated Proximal.
FL	Federated Learning.
GPU	Graphics Processing Unit.
gRPC	gRPC Remote Procedure Call.
IID	Independent and Identically Distributed.
ISO	International Organization for Standardization.
JSON	JavaScript Object Notation.
MPI	Message Passing Interface.
NVML	NVIDIA Management Library.
OE	Objetivo Específico.
PSI	Private Set Intersection.
RAM	Random Access Memory.
SDK	Software Development Kit.
SGD	Stochastic Gradient Descent.
TFG	Trabajo de Fin de Grado.
TLS	Transport Layer Security.
VRAM	Video Random Access Memory.
W3C	World Wide Web Consortium.

Glosario de Términos

A

agregación segura Protocolo criptográfico que permite combinar las actualizaciones de los clientes sin que el servidor observe cada una por separado.

aprendizaje federado Paradigma de aprendizaje automático que entrena un modelo de forma colaborativa entre varios clientes sin centralizar sus datos; en lugar de los datos, solo se intercambian las actualizaciones del modelo.

aprendizaje federado horizontal Variante en la que los clientes comparten el mismo espacio de características pero poseen ejemplos distintos. Es el escenario empleado en este trabajo.

aprendizaje federado vertical Variante en la que los clientes comparten parte de los individuos o entidades pero describen atributos distintos, lo que exige alinear registros.

B

backend Motor de ejecución subyacente de un framework; en FedML determina si el entrenamiento es secuencial (*sp*) o concurrente (MPI).

C

CIFAR-10 Conjunto de 60 000 imágenes en color de 32 por 32 píxeles distribuidas en diez clases, empleado en este trabajo como banco de pruebas de clasificación.

cifrado homomórfico Esquema de cifrado que permite operar sobre datos cifrados sin necesidad de descifrarlos previamente.

cross-device Escenario federado con un número muy elevado de dispositivos de usuario final, con disponibilidad intermitente, recursos limitados y conexiones inestables.

cross-silo Escenario federado con pocos clientes estables y con capacidad de cómputo elevada (por ejemplo hospitales o bancos), con participación habitualmente completa.

D

datos no IID Datos no independientes ni idénticamente distribuidos; situación en la que la distribución de los datos difiere de un cliente a otro, lo que dificulta la convergencia del modelo global.

E

época local Número de veces que cada cliente recorre su conjunto de datos local durante el entrenamiento de una ronda.

estrategia de agregación Procedimiento del servidor para combinar los modelos locales recibidos y obtener una nueva versión del modelo global.

F

FedAdagrad Variante de FedOpt que emplea el optimizador Adagrad en el lado del servidor.

FedAdam Variante de FedOpt que emplea el optimizador Adam en el lado del servidor.

FedAvg (algoritmo) Algoritmo de referencia del aprendizaje federado que agrega los modelos locales mediante un promedio ponderado por el número de ejemplos de cada cliente.

FedOpt (familia) Familia de optimización federada adaptativa que sustituye el promedio del servidor por un optimizador con momento y tasa de aprendizaje propios.

FedProx (algoritmo) Variante de FedAvg que añade un término proximal al entrenamiento local para penalizar la desviación de los pesos locales respecto al modelo global.

flexibilidad Propiedad arquitectónica que describe hasta qué punto un framework permite adaptarse a escenarios distintos de los previstos por defecto, modificando, sustituyendo o ampliando su comportamiento.

framework Conjunto de componentes reutilizables que proporciona la infraestructura para construir y ejecutar un tipo de sistema, en este caso de aprendizaje federado.

función de pérdida Función que cuantifica el error del modelo comparando su predicción con la salida esperada; en este trabajo se emplea la entropía cruzada.

M

modo concurrente Modo de ejecución en el que varios clientes o procesos permanecen activos simultáneamente, lo que permite observar el paralelismo y su impacto en los recursos.

modo secuencial Modo de ejecución en el que se mantiene una sola tarea de entrenamiento activa cada vez, de manera que los clientes de una ronda se procesan uno tras otro.

P

privacidad diferencial Técnica que añade ruido controlado a los datos o a las actualizaciones del modelo para limitar la información que puede inferirse sobre un individuo concreto.

R

Ray Framework de cómputo distribuido sobre el que Flower ejecuta su motor de simulación de clientes.

ResNet-18 Red neuronal residual de 18 capas; en este trabajo se adapta a las dimensiones de CIFAR-10 sustituyendo la primera convolución y eliminando el *maxpool* inicial.

ronda federada Iteración completa del ciclo de distribución del modelo global, entrenamiento local en los clientes, envío de los modelos locales y agregación en el servidor.

S

SCAFFOLD Algoritmo federado que corrige la deriva entre clientes mediante variables de control para mejorar la convergencia con datos heterogéneos.

T

tamaño de lote Número de ejemplos que se procesan antes de cada actualización de los parámetros del modelo (*batch size*).

tasa de aprendizaje Hiperparámetro que regula la magnitud de cada paso de actualización de los parámetros del modelo (*learning rate*).

U

usabilidad Eje de evaluación que valora la experiencia de desarrollo de cada framework: instalación, configuración, claridad de los registros y facilidad de depuración.
